

Solidum FEM

Manual de Referencia

Especificaciones físicas, formulaciones numéricas y contratos
de los componentes del programa

Generado automáticamente desde docs/specs/

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

20 de mayo de 2026

Índice general

Sobre este Manual

XII

1. Elementos 1D — Armaduras	1
1.1. Truss2D	1
1.2. Especificación física	1
1.2.1. 0. Descripción general	1
1.2.2. 1. Cinemática de desplazamientos	1
1.2.3. 2. Cinemática de deformaciones	1
1.2.4. 3. Ecuación constitutiva	1
1.2.5. 4. Equilibrio — forma fuerte	1
1.2.6. 5. Equilibrio — forma débil	2
1.3. Formulación numérica (FEM)	2
1.3.1. 6. Discretización	2
1.3.2. 7. Funciones de forma	2
1.3.3. 8. Matriz deformación-desplazamiento	2
1.3.4. 9. Rigidez elemental	2
1.3.5. 10. Fuerzas internas	2
1.3.6. 11. Cuadratura	2
1.4. Contrato de implementación	2
1.5. Implementación	3
1.6. Diálogo	4
1.7. Truss2DCorot	5
1.8. Especificación física	5
1.8.1. 0. Descripción general	5
1.8.2. 1. Cinemática	5
1.8.3. 2. Medida de deformación	5
1.8.4. 3. Ecuación constitutiva	5
1.8.5. 4. Equilibrio — forma débil incremental	5
1.9. Formulación numérica (FEM)	6
1.9.1. 6. Discretización	6
1.9.2. 7. Vector dirección actual	6
1.9.3. 8. Matriz B corotacional	6
1.9.4. 9. Rigidez tangente	6
1.9.5. 10. Fuerzas internas	6
1.9.6. 11. Cuadratura	6
1.10. Contrato de implementación	6
1.11. Implementación	7

1.12. Diálogo	8
1.13. Truss3D	9
1.14. Especificación física	9
1.14.1. 0. Descripción general	9
1.14.2. 1. Cinemática de desplazamientos	9
1.14.3. 2. Cinemática de deformaciones	9
1.14.4. 3. Ecuación constitutiva	9
1.14.5. 4. Equilibrio — forma fuerte	9
1.14.6. 5. Equilibrio — forma débil	9
1.15. Formulación numérica (FEM)	10
1.15.1. 6. Discretización	10
1.15.2. 7. Funciones de forma	10
1.15.3. 8. Matriz deformación-desplazamiento	10
1.15.4. 9. Rigidez elemental	10
1.15.5. 10. Fuerzas internas	10
1.15.6. 11. Cuadratura	10
1.16. Contrato de implementación	10
1.17. Implementación	11
1.18. Diálogo	12
1.19. Truss3DCorot	13
1.20. Especificación física	13
1.20.1. 0. Descripción general	13
1.20.2. 1. Cinemática	13
1.20.3. 2. Medida de deformación	13
1.20.4. 3. Ecuación constitutiva	13
1.20.5. 4. Equilibrio — forma débil incremental	13
1.21. Formulación numérica (FEM)	13
1.21.1. 6. Discretización	13
1.21.2. 7. Vector dirección actual	14
1.21.3. 8. Matriz B corotacional	14
1.21.4. 9. Rigidez tangente	14
1.21.5. 10. Fuerzas internas	14
1.21.6. 11. Cuadratura	14
1.22. Contrato de implementación	14
1.23. Implementación	15
1.24. Diálogo	16
2. Elementos 1D — Cables	17
2.1. Cable2DCorot	17
2.2. Especificación física	17
2.2.1. 0. Descripción general	17
2.2.2. 1. Cinemática	17
2.2.3. 2. Medida de deformación	17
2.2.4. 3. Ecuación constitutiva	18
2.2.5. 4. Equilibrio — forma débil incremental	18
2.3. Formulación numérica (FEM)	18
2.3.1. 6. Discretización	18
2.3.2. 7. Vectores de dirección (configuración corriente)	18

2.3.3.	8. Matriz B corotacional	18
2.3.4.	9. Rigidez tangente	18
2.3.5.	10. Fuerzas internas	19
2.3.6.	11. Cuadratura	19
2.4.	Contrato de implementación	19
2.5.	Implementación	20
2.6.	Diálogo	21
2.7.	Cable3DCorot	22
2.8.	Especificación física	22
2.8.1.	0. Descripción general	22
2.8.2.	1. Cinemática	22
2.8.3.	2. Medida de deformación	22
2.8.4.	3. Ecuación constitutiva	22
2.8.5.	4. Equilibrio — forma débil incremental	22
2.9.	Formulación numérica (FEM)	23
2.9.1.	6. Discretización	23
2.9.2.	7. Vector dirección actual	23
2.9.3.	8. Matriz B corotacional	23
2.9.4.	9. Rigidez tangente	23
2.9.5.	10. Fuerzas internas	23
2.9.6.	11. Cuadratura	23
2.10.	Contrato de implementación	23
2.11.	Implementación	25
2.12.	Diálogo	26
3.	Elementos 1D — Marcos / Vigas	27
3.1.	Frame2DEuler	27
3.2.	Especificación física	27
3.2.1.	0. Descripción general	27
3.2.2.	1. Hipótesis cinemática de Euler-Bernoulli	27
3.2.3.	2. Campos de desplazamiento	27
3.2.4.	3. Medidas de deformación	28
3.2.5.	4. Ecuaciones constitutivas	28
3.2.6.	5. Equilibrio — forma débil	28
3.3.	Formulación numérica (FEM)	28
3.3.1.	6. Discretización	28
3.3.2.	7. Sistema local y transformación	28
3.3.3.	8. Funciones de forma	29
3.3.4.	9. Rigidez local	29
3.3.5.	10. Fuerzas internas	29
3.3.6.	11. Ensamblaje global	29
3.3.7.	12. Cuadratura	29
3.4.	Contrato de implementación	30
3.5.	Implementación	31
3.6.	Diálogo	31
3.7.	Frame2DTimoshenko	33
3.8.	Especificación física	33
3.8.1.	0. Descripción general	33

3.8.2.	1. Hipótesis cinemática de Timoshenko	33
3.8.3.	2. Campos de desplazamiento	33
3.8.4.	3. Medidas de deformación	33
3.8.5.	4. Ecuaciones constitutivas	34
3.8.6.	5. Factor de Phi (shear locking)	34
3.8.7.	6. Equilibrio — forma débil	34
3.9.	Formulación numérica (FEM)	34
3.9.1.	7. Discretización	34
3.9.2.	8. Sistema local y transformación	34
3.9.3.	9. Rigidez local con corrección por shear locking	35
3.9.4.	10. Fuerzas internas	35
3.9.5.	11. Ensamblaje global	35
3.9.6.	12. Cuadratura	35
3.10.	Contrato de implementación	35
3.11.	Implementación	36
3.12.	Diálogo	37
3.13.	Frame2DEulerCorot	38
3.14.	Especificación física	38
3.14.1.	0. Descripción general	38
3.14.2.	1. Cinemática corotacional	38
3.14.3.	2. Medidas de deformación	38
3.14.4.	3. Ecuaciones constitutivas	38
3.14.5.	4. Equilibrio — forma débil	39
3.15.	Formulación numérica (FEM)	39
3.15.1.	5. DOFs locales deformacionales	39
3.15.2.	6. Rigidez local	39
3.15.3.	7. Matriz de transformación $\mathbf{B}$ (3×6)	39
3.15.4.	8. Rigidez tangente	39
3.15.5.	9. Cuadratura	40
3.16.	Contrato de implementación	40
3.17.	Implementación	41
3.18.	Diálogo	42
3.19.	Frame3D	43
3.20.	Especificación física	43
3.20.1.	0. Descripción general	43
3.20.2.	1. Hipótesis	43
3.20.3.	2. Sistema de ejes locales	43
3.20.4.	3. Medidas de deformación / acciones internas	43
3.20.5.	4. Equilibrio — forma débil	44
3.21.	Formulación numérica (FEM)	44
3.21.1.	5. Discretización	44
3.21.2.	6. Construcción de ejes locales	44
3.21.3.	7. Matriz de transformación global $\rightarrow$ local	44
3.21.4.	8. Matriz de rigidez local	45
3.21.5.	9. Fuerzas internas	45
3.21.6.	10. Ensamblaje global	45
3.21.7.	11. Cuadratura	45
3.22.	Contrato de implementación	45

3.23. Implementación	47
3.24. Diálogo	48
4. Elementos 2D — Sólidos	49
4.1. Quad4	49
4.2. Especificación física	49
4.2.1. 0. Descripción general	49
4.2.2. 1. Cinemática de desplazamientos	49
4.2.3. 2. Cinemática de deformaciones	49
4.2.4. 3. Ecuación constitutiva	49
4.2.5. 4. Equilibrio — forma fuerte	50
4.2.6. 5. Equilibrio — forma débil	50
4.3. Formulación numérica (FEM)	50
4.3.1. 6. Discretización	50
4.3.2. 7. Funciones de forma	50
4.3.3. 8. Matriz deformación-desplazamiento	50
4.3.4. 9. Rigidez elemental	50
4.3.5. 10. Fuerzas internas	51
4.3.6. 11. Cuadratura	51
4.3.7. 12. Cargas distribuidas consistentes	51
4.3.8. 13. Salida por punto de Gauss	51
4.4. Contrato de implementación	52
4.5. Implementación	53
4.6. Diálogo	53
4.7. Tri3	55
4.8. Especificación física	55
4.8.1. 0. Descripción general	55
4.8.2. 1. Cinemática de desplazamientos	55
4.8.3. 2. Cinemática de deformaciones	55
4.8.4. 3. Ecuación constitutiva	55
4.8.5. 4. Equilibrio — forma fuerte	55
4.8.6. 5. Equilibrio — forma débil	55
4.9. Formulación numérica (FEM)	55
4.9.1. 6. Discretización	55
4.9.2. 7. Funciones de forma	56
4.9.3. 8. Matriz deformación-desplazamiento	56
4.9.4. 9. Rigidez elemental	56
4.9.5. 10. Fuerzas internas	56
4.9.6. 11. Cuadratura	56
4.9.7. 12. Cargas distribuidas consistentes	56
4.9.8. 13. Salida por punto de Gauss	57
4.10. Limitación crítica — <i>shear locking</i>	57
4.11. Contrato de implementación	57
4.12. Implementación	58
4.13. Diálogo	58
4.14. Quad8	60
4.15. Especificación física	60
4.15.1. 0. Descripción general	60

4.15.2. 1. Cinemática de desplazamientos	60
4.15.3. 2. Cinemática de deformaciones	60
4.15.4. 3. Ecuación constitutiva	60
4.15.5. 4-5. Equilibrio	60
4.16. Formulación numérica	60
4.16.1. 6. Discretización	60
4.16.2. 7. Funciones de forma serendípitas	60
4.16.3. 8. Matriz $\mathbf{B}$	61
4.16.4. 9-10. Rigidez y fuerzas internas	61
4.16.5. 11. Cuadratura	61
4.16.6. 12. Cargas distribuidas consistentes	61
4.16.7. 13. Salida por punto de Gauss	61
4.17. Contrato de implementación	61
4.18. Implementación	62
4.19. Quad9	63
4.20. Especificación física	63
4.20.1. 7. Funciones de forma	63
4.20.2. 11. Cuadratura	63
4.20.3. 12. Cargas distribuidas	63
4.21. Contrato de implementación	63
4.22. Implementación	64
4.23. Tri6	65
4.24. Especificación física	65
4.24.1. 7. Funciones de forma	65
4.24.2. 11. Cuadratura	65
4.24.3. 12. Cargas distribuidas	65
4.25. Contrato de implementación	65
4.26. Implementación	66
5. Modelos Constitutivos (Materiales)	67
5.1. CableMaterial1D	67
5.2. Especificación física	67
5.2.1. 0. Descripción general	67
5.2.2. 1. Ecuación constitutiva	67
5.2.3. 2. Módulo tangente	67
5.2.4. 3. Variables internas	67
5.2.5. 4. Interpretación física	68
5.3. Contrato de implementación	68
5.4. Implementación	69
5.5. Diálogo	70
5.6. VonMises2D	71
5.7. Especificación física	71
5.7.1. 0. Descripción general	71
5.7.2. 1. Descomposición aditiva	71
5.7.3. 2. Ley elástica	71
5.7.4. 3. Superficie de fluencia (J2)	71
5.7.5. 4. Regla de flujo (asociada)	72
5.7.6. 5. Endurecimiento isótropo lineal	72

5.7.7.	6. Condiciones de Kuhn-Tucker	72
5.7.8.	7. Variables internas	72
5.7.9.	8. Notación Voigt y manejo de ε_{zz}^p	72
5.8.	Formulación numérica	73
5.8.1.	9. Esquema temporal — Backward Euler implícito	73
5.8.2.	10. Return mapping plane strain — algoritmo radial cerrado (existente)	73
5.8.3.	11. Return mapping plane stress — algoritmo proyectado (a implementar)	73
5.8.4.	12. Tolerancia de fluencia	75
5.9.	Contrato de implementación	75
5.10.	Implementación	78
5.10.1.	Bug fixes detectados al implementar	79
5.11.	Diálogo	79
5.12.	IsotropicDamage1D	81
5.13.	Especificación física	81
5.13.1.	Ecuaciones	81
5.14.	Formulación numérica	81
5.14.1.	Tangente algorítmica consistente	81
5.14.2.	Signo de la tangente consistente	82
5.14.3.	Por qué no tests de integración en esta spec	82
5.15.	Contrato de implementación	82
5.16.	Implementación	84
5.17.	Diálogo	85
5.18.	IsotropicDamage2D	86
5.19.	Especificación física	86
5.19.1.	0. Descripción general	86
5.19.2.	1. Cinemática	86
5.19.3.	2. Esfuerzo nominal vs esfuerzo efectivo	86
5.19.4.	3. Deformación equivalente	86
5.19.5.	4. Variable histórica y condición de carga (Kuhn-Tucker)	86
5.19.6.	5. Ley de evolución del daño	87
5.20.	Formulación numérica	87
5.20.1.	6. Algoritmo en un paso	87
5.20.2.	7. Tangente algorítmica consistente (ampliación)	87
5.20.3.	8. Pérdida de simetría	88
5.20.4.	9. Decisión de régimen durante el Newton	88
5.21.	Contrato de implementación	89
5.22.	Implementación	91
5.23.	Diálogo	92
5.24.	DruckerPrager2D	93
5.25.	Especificación física	93
5.25.1.	0. Descripción general	93
5.25.2.	1. Descomposición aditiva y elasticidad	93
5.25.3.	2. Criterio de fluencia (Drucker-Prager)	93
5.25.4.	3. Calibración con Mohr-Coulomb	94
5.25.5.	4. Regla de flujo (no asociada por defecto)	94
5.25.6.	5. Endurecimiento isótropo lineal	94
5.25.7.	6. Condiciones de Kuhn-Tucker	94
5.25.8.	7. Variables internas	95

5.26. Formulación numérica	95
5.26.1. 8. Esquema implícito — Backward Euler	95
5.26.2. 9. Return regular (cone surface)	95
5.26.3. 10. Return al ápice	96
5.26.4. 11. Detección del régimen de retorno	97
5.26.5. 12. Tangente algorítmica consistente	97
5.26.6. 13. Tolerancia de fluencia	97
5.27. Contrato de implementación	98
5.28. Implementación	101
5.29. Diálogo	101
6. Esquemas de Solución	102
6.1. ModalSolver	102
6.2. Especificación física	102
6.2.1. 0. Descripción general	102
6.2.2. 1. Problema físico	102
6.2.3. 2. Problema de autovalor generalizado	102
6.2.4. 3. Propiedades del par ($\mathbf{K}$, $\mathbf{M}$)	102
6.2.5. 4. Salidas	103
6.3. Formulación numérica	103
6.3.1. 5. Reducción por Dirichlet	103
6.3.2. 6. Algoritmo numérico: Lanczos con shift-invert	103
6.3.3. 7. Normalización	103
6.3.4. 8. Post-procesado	103
6.4. Contrato de implementación	104
6.5. Implementación	105
6.6. Diálogo	107
6.7. NewmarkSolver	108
6.8. Especificación física	108
6.8.1. 0. Descripción general	108
6.8.2. 1. Ecuación de movimiento semidiscreta	108
6.8.3. 2. Amortiguamiento Rayleigh	108
6.8.4. 3. Salidas	108
6.9. Formulación numérica	109
6.9.1. 4. Método de Newmark- β (a-form)	109
6.9.2. 5. Aceleración inicial	109
6.9.3. 6. Parámetros canónicos	109
6.9.4. 7. Imposición de Dirichlet con apoyos constantes	109
6.9.5. 8. Factorización reutilizable	109
6.10. Contrato de implementación	110
6.11. Implementación	112
6.12. Diálogo	113
6.13. NewtonNewmarkSolver	114
6.14. Qué cambia respecto a <code>NewmarkSolver</code>	114
6.14.1. Residuo dinámico en cada paso	114
6.14.2. Jacobiano dinámico tangente	114
6.14.3. Amortiguamiento Rayleigh — heredado, constante en el tiempo	114
6.14.4. Reducción y commit de estado	115

6.14.5. Convergencia (ADR 0007)	115
6.14.6. Factorización por iteración	115
6.14.7. Recuperación del comportamiento lineal	115
6.15. Contrato de implementación	115
6.16. Implementación	116
6.17. Diálogo	117
7. Elementos — catálogo	119
7.1. Truss2D — barra axial 2D	119
7.1.1. Cinemática	119
7.1.2. Formulación	119
7.1.3. Convenciones	119
7.1.4. Parámetros	119
7.1.5. Cargas distribuidas	120
7.1.6. Régimen de validez	120
7.1.7. Validación	120
7.2. Truss2DCorot — armadura 2D corotacional	120
7.2.1. Cinemática	120
7.2.2. Formulación	120
7.2.3. Cargas distribuidas	120
7.2.4. Régimen de validez	121
7.2.5. Validación	121
7.3. Truss3D — barra axial 3D	121
7.3.1. Cinemática	121
7.3.2. Formulación	121
7.3.3. Convenciones	121
7.3.4. Parámetros	122
7.3.5. Cargas distribuidas	122
7.3.6. Régimen de validez	122
7.3.7. Validación	122
7.4. Truss3DCorot — barra axial 3D corotacional	122
7.4.1. Cinemática	122
7.4.2. Formulación	122
7.4.3. Cargas distribuidas	123
7.4.4. Régimen de validez	123
7.4.5. Validación	123
7.5. Cable2DCorot — cable 2D corotacional	123
7.5.1. Cinemática	123
7.5.2. Formulación	123
7.5.3. Cargas distribuidas	123
7.5.4. Régimen de validez	124
7.5.5. Independencia del diseño	124
7.5.6. Validación	124
7.6. Cable3DCorot — cable 3D corotacional	124
7.6.1. Cinemática	124
7.6.2. Formulación	124
7.6.3. Cargas distribuidas	125
7.6.4. Régimen de validez	125

7.6.5.	Independencia del diseño	125
7.6.6.	Validación	125
7.7.	Frame2DEuler — marco/viga 2D Euler-Bernoulli	125
7.7.1.	Cinemática	125
7.7.2.	Formulación	125
7.7.3.	Cargas distribuidas	126
7.7.4.	Régimen de validez	126
7.7.5.	Independencia del diseño	126
7.7.6.	Validación	126
7.8.	Frame2DTimoshenko — marco/viga 2D Timoshenko	126
7.8.1.	Cinemática	126
7.8.2.	Formulación	127
7.8.3.	Parámetros	127
7.8.4.	Cargas distribuidas	127
7.8.5.	Régimen de validez	127
7.8.6.	Independencia del diseño	127
7.8.7.	Validación	127
7.9.	Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional	128
7.9.1.	Cinemática corotacional	128
7.9.2.	Formulación	128
7.9.3.	Cargas distribuidas	128
7.9.4.	Régimen de validez	128
7.9.5.	Dominios que abre	128
7.9.6.	Independencia del diseño	129
7.9.7.	Validación	129
7.10.	Frame3D — marco/viga 3D Euler-Bernoulli	129
7.10.1.	Cinemática	129
7.10.2.	Ejes locales y orientación de la sección	129
7.10.3.	Formulación	129
7.10.4.	Parámetros	130
7.10.5.	Cargas distribuidas	130
7.10.6.	Régimen de validez	130
7.10.7.	Masa lumped: limitación en orientación oblicua	130
7.10.8.	Independencia del diseño	130
7.10.9.	Validación	130
7.11.	Quad4 — cuadrilátero bilineal 2D	131
7.12.	Tri3 — triángulo lineal 2D (CST)	131
7.13.	Quad8 — cuadrilátero serendípito 2D de orden 2	132
7.14.	Quad9 — cuadrilátero Lagrangiano 2D de orden 2	132
7.15.	Tri6 — triángulo 2D cuadrático completo P_2	132
7.16.	Hex8 — hexaedro trilineal 3D	133
7.17.	Tet4 — tetraedro lineal 3D (CST 3D)	134
7.18.	CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS)	134
7.19.	Cómo añadir un elemento nuevo	135

8. Modelos constitutivos — catálogo	137
8.1. Elastic1D — elástico lineal 1D	137
8.2. Elastic2D — elástico lineal 2D	137
8.3. Elastic3D — elástico lineal isótropo 3D	138
8.4. IsotropicDamage1D — daño isótropo 1D con softening exponencial	138
8.5. IsotropicDamage2D — daño isótropo 2D con softening exponencial	139
8.6. Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal	140
8.7. CableMaterial1D — cable axial 1D con elasticidad unilateral	141
8.8. DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain	141
8.9. VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal	143
8.10. CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial	144
8.11. Cómo añadir un material nuevo	145
9. Solvers — catálogo	146
9.1. LinearSolver — solución lineal directa en un paso	146
9.2. NonlinearSolver — Newton-Raphson incremental con paso adaptativo	146
9.3. ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield)	147
9.4. DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004)	148
9.5. ModalSolver — análisis modal por autovalores generalizados (ADR 0009)	149
9.6. NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3)	150
9.7. NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4)	151
9.8. HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α)	152
9.9. NewtonHHTSolver — análisis dinámico transitorio no lineal HHT- α	153
9.10. CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5)	154
9.11. HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6)	155
9.12. ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)	155
9.13. Cómo añadir un solver nuevo	156
10. Anexos técnicos	158
10.1. Dos capas que se llaman ambas «solver»	158
10.2. Backends disponibles y criterio de selección	158
10.3. Fallback automático SPD $\rightarrow$ LU	159
10.4. Factorización reusable y Newton modificado	159
10.5. Override desde YAML — herramienta de diagnóstico	160
10.6. Activación de Cholesky (opcional)	160

Sobre este Manual

Este manual contiene la **referencia formal** de los componentes de Solidum FEM: cada elemento finito y cada modelo constitutivo se documenta con su especificación física (ecuaciones), su formulación numérica (matrices **B**, rigidez tangente, integración) y su contrato YAML.

El contenido se genera automáticamente desde los archivos `docs/specs/*.md` del repositorio, que constituyen la *fuentes única de verdad* de cada componente. No editar este PDF manualmente; cualquier corrección debe hacerse sobre la spec correspondiente y regenerar el manual mediante:

```
python manuals/build_reference_manual.py
```

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consulte el **Manual de Usuario** en `manuals/User_manual.pdf`.

Capítulo 1

Elementos 1D — Armaduras

1.1 Truss2D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.2 Especificación física

1.2.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento.

1.2.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s$$

1.2.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}$$

1.2.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}$$

1.2.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.2.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

1.3 Formulación numérica (FEM)

1.3.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en globales:

$$c = \frac{x_2 - x_1}{L}, \quad s_\theta = \frac{y_2 - y_1}{L}.$$

1.3.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.3.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global ($\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$):

$$\mathbf{B} = \frac{1}{L}[-c, -s_\theta, c, s_\theta], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.3.4 9. Rigidez elemental

Integración analítica exacta (EA constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d}^\top \mathbf{d}, \quad \mathbf{d} = [-c, -s_\theta, c, s_\theta].$$

1.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d}.$$

1.3.6 11. Cuadratura

No aplica (forma cerrada).

1.4 Contrato de implementación

```
name: Truss2D
kind: element
status: validated          # draft → implemented → validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
```

```

n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state')"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"

validity:
- "|| 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos en extremos
- grandes desplazamientos
- pandeo

acceptance:
- name: barra axial bajo carga puntual
setup: "empotrada en s=0, carga F axial en s=L"
expect: "u(L) = F·L / (E·A)"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"

```

1.5 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss2D (registrada vía `@ElementRegistry.register`)
- **Tests:**
 - tests/test_truss.py · `TestTruss2D` — verifica $L0$, cosenos directores, entradas de $\mathbf{K}_e$ (incluyendo simetría), y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos. Los valores numéricos esperados coinciden con $\mathbf{K}_e = (EA/L) \mathbf{d} \mathbf{d}^\top$ y $\mathbf{F}_{\text{int}} = N \mathbf{d}$ para geometría de prueba $L = 5$, $c = 0,6$, $s = 0,8$.
 - tests/test_integration.py · `TestSolversIntegration` — resuelve end-to-end el caso de aceptación (barra empotrada en un extremo, carga axial en el otro) con `NonlinearSolver` y `ArcLengthSolver`; en régimen elástico precedente a la fluencia reproduce $u(L) = FL/(EA)$.
- **Notas de traducción:**
 - $L0$, c , s se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen infinitesimal declarado.
 - El contrato con el material es el estándar del proyecto: `material.compute_state( $\epsilon$ , state) → ( $\sigma$ ,  $E_t$ , state')`.

- Para grandes desplazamientos/rotaciones usar `Truss2DCorot`, que hereda de esta clase.

1.6 Diálogo

- **2026-04-21** · Cierre de ciclo retroactivo. La clase `Truss2D` preexistía al flujo spec-first; la spec se creó documentando la formulación ya implementada. Los tests unitarios y de integración satisfacen el único criterio de **acceptance** declarado (analíticamente, vía los valores de $\mathbf{K}_e$ y end-to-end vía el solver). Promovido **status: validated**.

1.7 Truss2DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.8 Especificación física

1.8.1 0. Descripción general

Barra articulada 2D equivalente a **Truss2D** pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. La cinemática se actualiza en configuración corriente (corotacional / Updated Lagrangian): el eje de la barra gira con el movimiento y la deformación se mide sobre la longitud actual. Rigidez tangente = rigidez material + rigidez geométrica debida a la fuerza axial.

1.8.2 1. Cinemática

Configuraciones:

- Referencia: nodos $\mathbf{X}_1, \mathbf{X}_2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores en configuración corriente:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

1.8.3 2. Medida de deformación

Ingenieril corotacional (válida para pequeña deformación aunque la rotación sea grande):

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.8.4 3. Ecuación constitutiva

Material 1D: $\sigma = E \varepsilon$ (elástico lineal); el contrato admite cualquier material con **STRAIN_DIM** = 1.

1.8.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales completo:

$$\underbrace{\int_V \delta \varepsilon \sigma dV}_{\delta W_{\text{int}}} = \underbrace{\int_V \delta \mathbf{u}^\top \mathbf{b} dV + \int_{\partial V_t} \delta \mathbf{u}^\top \bar{\mathbf{t}} dA}_{\delta W_{\text{ext}}}.$$

El elemento calcula **solo el lado interno**:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}},$$

con la rigidez tangente consistente separada en parte material y geométrica (ver §9). Las cargas externas (nodales y, si las hubiera, de cuerpo o tracción) las ensambla el solver a partir del input del usuario; este elemento **no** produce vector nodal equivalente de fuerzas de cuerpo distribuidas a lo largo del eje.

1.9 Formulación numérica (FEM)

1.9.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$.

1.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top,$$

con c_θ, s_θ evaluados en la configuración corriente.

1.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.9.4 9. Rigidez tangente

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_M + \mathbf{K}_G, \\ \mathbf{K}_M &= \frac{EA}{L_0} \mathbf{d} \mathbf{d}^\top, \\ \mathbf{K}_G &= \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A. \end{aligned}$$

1.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.9.6 11. Cuadratura

No aplica (forma cerrada; un único punto conceptual en el eje).

1.10 Contrato de implementación

```
name: Truss2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }
```

```

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
  sign: " > 0 > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - c_, s_, l se recalculan en cada evaluación"

validity:
  - "|| 1e-2 (pequeña deformación axial)"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - pandeo por bifurcación (captura de snap-through requiere arc-length)
  - plasticidad con grandes deformaciones
  - "fuerzas de cuerpo distribuidas (peso propio sobre la barra): el elemento no
    calcula vector nodal equivalente; el usuario reparte masa a los nodos en el input"

acceptance:
  - name: equivalencia con Truss2D en régimen lineal
    setup: "barra axial empotrada, carga F pequeña ( ~ 1e-6)"
    expect: "u(L) = F·L/(E·A) con tol_rel 1e-8"
    tol_rel: 1.0e-8

  - name: invariancia bajo rotación de cuerpo rígido
    setup: "aplicar rotación rígida de 45° a ambos nodos (sin deformar)"
    expect: "F_int = 0 y = 0"
    tol_abs: 1.0e-10

  - name: rigidez geométrica bajo tensión
    setup: "barra en tensión con N > 0; verificar autovalor transversal"
    expect: "autovalor de K_T en dirección n = N/l"
    tol_rel: 1.0e-10

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
    .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.11 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss2DCorot — hereda de Truss2D (reutiliza `__init__`, `ClassVars` y metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar cosenos directores y longitud en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss2DCorot — tres tests, uno por cada acceptance:
- test_acceptance_linear_limit_matches_truss2d (criterio 1).
- test_acceptance_rigid_body_rotation (criterio 2).

- `test_acceptance_geometric_stiffness_under_traction` (criterio 3).
- **Notas de traducción:**
- La longitud corriente l y los cosenos c_θ, s_θ se recalculan en cada evaluación (no se cachean entre llamadas del solver, como exige Updated Lagrangian).
- El vector $\mathbf{d}$ de §7 se construye con signos $[-c_\theta, -s_\theta, c_\theta, s_\theta]$; el transverso $\mathbf{n}$ como $[-s_\theta, c_\theta, s_\theta, -c_\theta]$. Ambos con norma $\sqrt{2}$ — el factor se absorbe en los coeficientes EA/L_0 y N/l .
- La rigidez tangente se devuelve ya sumada $\mathbf{K}_M + \mathbf{K}_G$; el solver no distingue las dos contribuciones.
- El test 3 confronta $\mathbf{K}_T$ con la descomposición esperada y además verifica $\mathbf{K}_G \mathbf{n} = (2N/l)\mathbf{n}$ (autovalor sobre el vector sin normalizar, que es N/l sobre el versor — así queda la ambigüedad resuelta explícitamente).

1.12 Diálogo

- **2026-04-21** · Implementación inicial tras validar el diseño spec-first. Se optó por herencia `Truss2DCorot(Truss2D)` para reutilizar el constructor (L_0 , cosenos iniciales) y los `ClassVars` del registro; los métodos que dependen de configuración corriente se sobrescriben. La bandera `large_strains` que tenía `Truss2D` previamente se elimina en el mismo commit — el régimen corotacional vive ahora como elemento independiente.
- **2026-04-21** · Normalización del autovalor transversal (criterio 3): la fórmula del libro da N/l sobre el **versor** $\hat{\mathbf{n}} = \mathbf{n}/\sqrt{2}$; sobre el vector $\mathbf{n}$ directo (norma $\sqrt{2}$) el autovalor es $2N/l$. El test verifica la segunda forma para evitar raíces cuadradas y la nota de traducción documenta la equivalencia.

1.13 Truss3D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.14 Especificación física

1.14.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento. Régimen estrictamente de **linealidad geométrica**: pequeños desplazamientos, pequeñas rotaciones, pequeñas deformaciones.

1.14.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s.$$

1.14.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}.$$

1.14.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}.$$

El elemento admite cualquier material 1D con STRAIN_DIM = 1 (elástico lineal, plástico 1D, daño 1D).

1.14.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.14.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

El elemento calcula **solo el lado interno**; las cargas externas las ensambla el solver a partir del input del usuario.

1.15 Formulación numérica (FEM)

1.15.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en coordenadas globales:

$$c_x = \frac{x_2 - x_1}{L}, \quad c_y = \frac{y_2 - y_1}{L}, \quad c_z = \frac{z_2 - z_1}{L},$$

con $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$ (configuración inicial, fija).

1.15.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.15.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global con $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top$:

$$\mathbf{B} = \frac{1}{L}[-c_x, -c_y, -c_z, c_x, c_y, c_z], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.15.4 9. Rigidez elemental

Integración analítica exacta (EA constante, $\mathbf{B}$ constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top.$$

$\mathbf{K}_e$ es una matriz 6×6 de rango 1.

1.15.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d} = N \mathbf{d}.$$

1.15.6 11. Cuadratura

No aplica (forma cerrada, $\mathbf{B}$ y σ uniformes por elemento). El campo `n_integration_points: 1` se refiere al número de estados materiales conceptuales (uno por elemento), no a puntos de Gauss.

1.16 Contrato de implementación

```
name: Truss3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1
```

```

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state)"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"
configuration: "inicial fija - L, c_x, c_y, c_z no se actualizan con los
desplazamientos"

validity:
- "|| 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- grandes desplazamientos o rotaciones (para ese régimen usar `Truss3DCorot`)
- pandeo
- "fuerzas de cuerpo distribuidas sobre el eje: el usuario reparte masa a los nodos"

acceptance:
- name: barra axial bajo carga puntual (3D)
setup: "empotrada en s=0, carga F axial en s=L, orientación arbitraria en el
espacio"
expect: "u(L) = F·L / (E·A) proyectado sobre el eje"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"
- "Cook R.D., Concepts and Applications of FEA, §2.3"

```

1.17 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss3D — hereda directamente de Element (sin relación de herencia con Truss2D ni con Truss2DCorot).
- **Tests:** tests/test_truss.py · TestTruss3D — verifica L0, cosenos directores (c_x, c_y, c_z), entradas de $\mathbf{K}_e$ (incluyendo simetría, dimensiones 6×6) y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos con geometría de prueba $L = 13$, $(c_x, c_y, c_z) = (3/13, 4/13, 12/13)$.
- **Notas de traducción:**
 - L0, c_x , c_y , c_z se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen geoméricamente lineal declarado.
 - El constructor tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), para facilitar transición de problemas planos embebidos.

- El contrato con el material es el estándar del proyecto: `material.compute_state( $\varepsilon$ , state)  $\rightarrow$  ( $\sigma$ ,  $E_t$ , state')`.
- Para grandes desplazamientos/rotaciones en 3D usar `Truss3DCorot`, que hereda de esta clase.

1.18 Diálogo

- **2026-04-21** · Apertura de spec retroactiva. La clase `Truss3D` preexistía al flujo spec-first; la spec se crea documentando la formulación ya implementada y estableciendo explícitamente su alcance: régimen de linealidad geométrica, sin bandera ni variante corotacional. Los tests unitarios existentes cubren el criterio de **acceptance** (vía los valores de $\mathbf{K}_e = (EA/L)\mathbf{d}\mathbf{d}^\top$ que implican algebraicamente $u(L) = FL/(EA)$). Promovido **status: validated**.

1.19 Truss3DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.20 Especificación física

1.20.1 0. Descripción general

Barra articulada 3D equivalente a Truss3D pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. Updated Lagrangian / corotacional: el eje de la barra se redefine en cada iteración sobre la configuración corriente y la deformación se mide sobre la longitud actual. La rigidez tangente suma material + geométrica.

1.20.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

1.20.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.20.4 3. Ecuación constitutiva

Material 1D con STRAIN_DIM = 1: $\sigma = \sigma(\varepsilon)$ (el elemento no hardcodea la ley; la provee el material).

1.20.5 4. Equilibrio — forma débil incremental

PTV estándar. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

1.21 Formulación numérica (FEM)

1.21.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

1.21.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top,$$

con los cosenos evaluados en configuración **corriente**. $\|\mathbf{d}\|^2 = 2$.

1.21.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.21.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un **plano** (no un vector único). Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (matriz 3×3 , rango 2). La rigidez geométrica en el elemento de 6 DOFs:

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

1.21.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.21.6 11. Cuadratura

No aplica (forma cerrada).

1.22 Contrato de implementación

```
name: Truss3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state)"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
```

```

sign: " > 0    > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- pandeo por bifurcación
- plasticidad con grandes deformaciones
- "fuerzas de cuerpo distribuidas: el usuario reparte masa a los nodos"

acceptance:
- name: equivalencia con Truss3D en régimen lineal
  setup: "barra 3D con orientación arbitraria, carga axial pequeña ( ~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3D lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida 3D
  setup: "aplicar una rotación rígida arbitraria (p. ej. 30° sobre eje z, luego 45° sobre eje x)"
  expect: "F_int = 0 y    = 0"
  tol_abs: 1.0e-10

- name: rigidez geométrica transversa (plano perpendicular)
  setup: "barra en tensión con N > 0; aplicar K_G sobre dos vectores perpendiculares al eje linealmente independientes"
  expect: "K_G·v_perp = (N/l)·v_perp_struct, donde v_perp_struct es la extensión del vector transverso al espacio de 6 DOFs con signos opuestos en los dos nodos"
  tol_rel: 1.0e-10

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.23 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss3DCorot — hereda de Truss3D (reutiliza `__init__`, tolerancia a nodos 2D/3D, metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar longitud y cosenos directores en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss3DCorot — tres tests, uno por cada acceptance:
- `test_acceptance_linear_limit_matches_truss3d` (criterio 1).
- `test_acceptance_rigid_body_rotation` (criterio 2 — dos rotaciones rígidas distintas, verifica $\sigma=0$ y $F_{int}=0$ en ambas).

- `test_acceptance_geometric_stiffness_transverse_plane` (criterio 3 — aplica $\mathbf{K}_G$ a dos direcciones linealmente independientes del plano perpendicular al eje).
- **Notas de traducción:**
- La clave respecto a 2D es el proyector $\mathbf{P}_3 = \mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (3×3 , rango 2): el plano perpendicular al eje es bidimensional. $\mathbf{K}_G$ resulta entonces de rango 2, no rango 1.
- El proyector se construye con `solidum.math.geometry.perpendicular_projector( $\hat{\mathbf{e}}$ )`, helper compartido con `Cable3DCorot` (operación geométrica pura, no específica de armaduras ni de cables).
- La longitud corriente l y los cosenos c_x, c_y, c_z se recalculan en cada evaluación (Updated Lagrangian); no se cachean entre llamadas del solver.
- Autovalores de $\mathbf{K}_G$ sobre modos transversos (rotación de la barra alrededor de su centro): $2N/l$ sobre el vector sin normalizar — igual que en 2D, el factor 2 viene de que el vector de 6 componentes tiene norma $\sqrt{2}$. Sobre el versor el autovalor es N/l .

1.24 Diálogo

- **2026-04-21** · Implementación tras validar el diseño spec-first. Se optó por herencia `Truss3DCorot(Truss3D)` por paralelismo con `Truss2DCorot(Truss2D)` y reutilización del constructor. La diferencia estructural con el caso 2D es el proyector $\mathbf{P}$: en 2D tenía rango 1 y generaba un $\mathbf{K}_G$ de rango 1 (vector $\mathbf{n}$ único); en 3D tiene rango 2 y $\mathbf{K}_G$ rango 2 (plano perpendicular al eje). Los tests del criterio 3 verifican explícitamente **dos** direcciones transversas linealmente independientes, capturando este cambio dimensional.
- **2026-04-21** · Test de invariancia bajo rotación rígida 3D: se verifican dos rotaciones distintas (plano x-y y plano x-z) para descartar coincidencias accidentales en ejes canónicos.

Capítulo 2

Elementos 1D — Cables

2.1 Cable2DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.2 Especificación física

2.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada — se destensa. Cinemática **corotacional** (Updated Lagrangian): el eje rota con el movimiento, la longitud se mide sobre la configuración corriente.

La propiedad de unilateralidad vive en el **material**; el elemento es la maquinaria cinemática que aplica al material la deformación correcta. Para obtener el comportamiento propio de cable se empareja este elemento con un material unilateral como **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra articulada corotacional (útil para pruebas, no para modelado realista de cables).

2.2.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores del eje **corriente**:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

2.2.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.2.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento invoca `material.compute_state(ε, state)` y recibe $(\sigma, E_t, \text{nuevo estado})$ sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material — el elemento nunca filtra σ ni ε por signo.

2.2.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales, lado interno:

$$\delta W_{\text{int}} = \int_V \delta \varepsilon \sigma dV = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

Las cargas externas las ensambla el solver. El elemento no calcula vector nodal equivalente de fuerzas de cuerpo.

2.3 Formulación numérica (FEM)

2.3.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top.$$

2.3.2 7. Vectores de dirección (configuración corriente)

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top.$$

$\mathbf{d}$ apunta a lo largo del eje corriente (norma $\sqrt{2}$); $\mathbf{n}$ al perpendicular (norma $\sqrt{2}$). Son ortogonales: $\mathbf{d}^\top \mathbf{n} = 0$.

2.3.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.3.4 9. Rigidez tangente

$$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G, \\ \mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{K}_G = \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global, como físicamente corresponde a un cable flojo.

2.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

Cuando el cable está destensado ($N = 0$), $\mathbf{F}_{\text{int}} = \mathbf{0}$.

2.3.6 11. Cuadratura

No aplica (forma cerrada).

2.4 Contrato de implementación

```

name: Cable2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)
  expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
  sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - l, c_, s_ se recalculan en cada evaluación"

validity:
  - "|| 1e-2 en régimen tensado"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas axiales (las transversales se transmiten por el destensado-retensado del cable)

out_of_scope:
  - flexión, cortante, momentos
  - dinámica con inercia distribuida (sin matriz de masa)
  - pandeo (irrelevante: el cable no transmite compresión)
  - "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
  - pretensión mediante parámetro del elemento (modelar con longitud inicial ajustada)

numerical_caveats:

```



```

- "Un cable completamente destensado aporta  $K_T = 0$  al sistema global. Si otros
  elementos no garantizan rigidez suficiente en los DOFs compartidos, la matriz
  tangente global puede ser singular o mal condicionada."
- "En transiciones tensado destensado ( cruza 0), Newton-Raphson puede oscilar por
  el salto discontinuo en  $E_t$ . Usar arc-length o pasos de carga finos si el
  problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional
  setup: "cable horizontal con material lineal (Elastic1D, E=1000),  $u_x$  nodo 2 =  $1e-3$ 
        =  $1e-3 > 0$ "
  expect: " =  $E$  ,  $E_t = E$ ,  $K_T = K_M + K_G$  con valores de barra corotacional"
  tol_rel: 1.0e-10

- name: cable destensado produce aportación nula
  setup: "cable horizontal con CableMaterial1D,  $u_x$  nodo 2 =  $-1e-3$  < 0"
  expect: " = 0,  $F_{int} = 0$ ,  $K_T =$  matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida
  setup: "cable inicialmente horizontal, rotación rígida de  $45^\circ$  de ambos nodos"
  expect: " = 0 y  $F_{int} = 0$  (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero
  setup: " $u_e$  tal que = 0 exactamente, material unilateral"
  expect: " = 0,  $E_t = 0$ ,  $F_{int} = 0$ ,  $K_T = 0$ "
  tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.5 Implementación

- **Archivo:** solidum/elements/cable.py — archivo propio, no referencia a los módulos de armadura ni a ningún otro elemento.
- **Clase:** Cable2DCorot — hereda directamente de Element (no de Truss2DCorot ni de ningún elemento de armadura). La maquinaria cinemática corotacional se reimplementa íntegra dentro de la clase.
- **Tests:** tests/test_cable_elements.py · TestCable2DCorot — los cuatro criterios de acceptance más un test de registro:
 - test_acceptance_cable_tensado_como_barra_corot (criterio 1).
 - test_acceptance_cable_destensado_aportacion_nula (criterio 2).
 - test_acceptance_rotacion_rigida (criterio 3).
 - test_acceptance_cruce_por_cero (criterio 4).
 - test_registro_en_registry — autodiscover.

■ **Notas de traducción:**

- La clase duplica deliberadamente la lógica de `Truss2DCorot`. No hay herencia. Es el precio de la independencia: si mañana se toca la armadura corotacional, este cable no se mueve.
- `_current_geometry(u_e)` devuelve $(1, c_\theta, s_\theta)$ — método privado para aislar la única operación que depende de configuración corriente.
- En estado destensado, $\mathbf{K}_M = \mathbf{0}$ (porque $E_t = 0$ viene del material) y $\mathbf{K}_G = \mathbf{0}$ (porque $N = 0$); el elemento aporta literalmente ceros al ensamblaje, como debe ser físicamente un cable flojo. Los tests lo verifican con tolerancia absoluta.
- El elemento no valida que el material sea unilateral: es responsabilidad del usuario emparejar con `CableMaterial1D`. Emparejar con `Elastic1D` produce respuesta de barra corotacional y se usa internamente en el test del criterio 1 para comparar contra valores analíticos.

2.6 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente independiente de `Truss2DCorot` por decisión explícita del usuario: la maquinaria cinemática se duplica dentro de la clase de cable para desacoplar su evolución futura del elemento de armadura. El coste es 40 líneas duplicadas; la ganancia es que ninguna refactorización de las armaduras puede contaminar silenciosamente el comportamiento de los cables.
- **2026-04-21** · Validación de material unilateral: se optó por **no** imponerla en `__init__`. Motivo: el contrato elemento↔material del proyecto es genérico sobre `STRAIN_DIM=1`; introducir una verificación específica crearía un acoplamiento entre la clase del elemento y la identidad del material. Se documenta en `material_contract.expected_behaviour` del YAML para orientar al usuario.

2.7 Cable3DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.8 Especificación física

2.8.1 0. Descripción general

Elemento 1D inmerso en el espacio 3D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada. Cinemática **corotacional** (Updated Lagrangian): el eje rota libremente en el espacio, la longitud se mide sobre la configuración corriente.

La unilateralidad vive en el **material**; el elemento es la maquinaria cinemática 3D. Para obtener respuesta física de cable, emparejar con **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra corotacional 3D.

2.8.2 1. Cinemática

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

2.8.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.8.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento llama `material.compute_state( $\varepsilon$ , state)` y recibe (σ , E_t , nuevo estado) sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material.

2.8.5 4. Equilibrio — forma débil incremental

PTV estándar; el elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^T \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

2.9 Formulación numérica (FEM)

2.9.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

2.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top, \quad \|\mathbf{d}\|^2 = 2,$$

con los cosenos evaluados en configuración **corriente**.

2.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.9.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un plano bidimensional. Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (3×3 , rango 2):

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global.

2.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

2.9.6 11. Cuadratura

No aplica (forma cerrada).

2.10 Contrato de implementación

```
name: Cable3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
```

```

n_nodes: 2
strain_dim: 1
n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state')"
strain_kind: axial escalar (deformación ingenieril corotacional)
expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
voigt: "N/A (escalar)"
node_orientation: "libre; K_T y F_int invariantes bajo permutación"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2 en régimen tensado"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas axiales (las transversales se transmiten por destensado-retensado del cable)

out_of_scope:
- flexión, cortante, momentos, torsión
- dinámica con inercia distribuida (sin matriz de masa)
- pandeo (irrelevante: el cable no transmite compresión)
- "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
- pretensión mediante parámetro del elemento

numerical_caveats:
- "Cable destensado K_T = 0 en los DOFs del elemento. Si otros elementos no garantizan rigidez en esos DOFs, la matriz tangente global puede ser singular."
- "En transiciones tensado destensado (cruza 0), Newton-Raphson puede oscilar. Usar arc-length o pasos finos si el problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional 3D
  setup: "cable con orientación arbitraria, material lineal (Elastic1D), desplazamiento axial pequeño ( ~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3DCorot a tolerancia rtol = 1e-4"
  tol_rel: 1.0e-4

- name: cable destensado produce aportación nula
  setup: "cable sobre eje x con CableMaterial1D, u_x nodo 2 = -1e-3 < 0"
  expect: " = 0, F_int = 0, K_T = matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida 3D
  setup: "cable inicial sobre eje x, rotación rígida de 45° en plano x-z"
  expect: " = 0 y F_int = 0 (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero

```

```

setup: "u_e = 0 con material unilateral"
expect: " = 0, E_t = 0, F_int = 0, K_T = 0"
tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.11 Implementación

- **Archivo:** solidum/elements/cable.py — comparte archivo con `Cable2DCorot` por convención temática del proyecto; no comparte código.
- **Clase:** `Cable3DCorot` — hereda directamente de `Element` (no de `Cable2DCorot`, no de `Truss3DCorot`, no de ningún otro elemento). Maquinaria cinemática 3D autónoma.
- **Tests:** tests/test_cable_elements.py · `TestCable3DCorot` — los 4 criterios de `acceptance` más un test de registro:
 - `test_acceptance_cable_tensado_como_barra_corot_3d` (criterio 1).
 - `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
 - `test_acceptance_rotacion_rigida_3d` (criterio 3).
 - `test_acceptance_cruce_por_cero` (criterio 4).
 - `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
 - La clase duplica la maquinaria corotacional 3D de `Truss3DCorot`. Duplicación deliberada por la misma razón que en 2D: desacoplar la evolución futura de armaduras y cables.
 - El proyector $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ se construye con `solidum.math.geometry.perpendicular_projector( $\hat{\mathbf{e}}$ )`, helper compartido con `Truss3DCorot` (operación geométrica pura, ortogonal al hecho de que el material sea unilateral o no).
 - `_current_geometry` tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), heredando la flexibilidad de `Truss3D` para problemas embebidos.
 - En estado destensado, $\mathbf{K}_M = \mathbf{0}$ y $\mathbf{K}_G = \mathbf{0}$ simultáneamente; el elemento aporta literalmente ceros al ensamblaje.
 - El test del criterio 1 compara directamente contra `Truss3DCorot` con material `Elastic1D`: si ambos producen $\mathbf{K}$ y $\mathbf{F}_{\text{int}}$ iguales a tolerancia, la cinemática del cable y de la barra son consistentes entre sí en el régimen tensado.

2.12 Diálogo

- **2026-04-21** · Elemento creado como pieza totalmente autónoma por decisión del usuario: no hereda de `Cable2DCorot` ni de `Truss3DCorot`. Compartir archivo con `Cable2DCorot` (`solidum/elements/cable.py`) es convención temática del proyecto (elementos del mismo dominio conviven en un archivo, como las 4 armaduras y los 2 frames 2D en `structural.py`), no implica dependencia de código.
- **2026-04-21** · La única diferencia estructural con `Cable2DCorot` es el proyector 3D: en 2D la perpendicular es un vector único ($\mathbf{n}$); en 3D es un plano (dimensión 2). Esto se refleja en el rango de $\mathbf{K}_G$: 1 en 2D, 2 en 3D.

Capítulo 3

Elementos 1D — Marcos / Vigas

3.1 Frame2DEuler

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.2 Especificación física

3.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga esbelta** según la teoría de **Euler-Bernoulli**. Dos nodos rígidamente conectados (transmiten momento), 3 grados de libertad por nodo (u_x, u_y, r_z) . Captura tres acciones internas: **fuerza axial**, **cortante transverso** y **momento flector**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.2.2 1. Hipótesis cinemática de Euler-Bernoulli

- Las secciones transversales se mantienen **planas** tras la deformación.
- Las secciones permanecen **perpendiculares al eje neutro deformado** (cizalladura transversal nula: $\gamma_{xy} = 0$).

Consecuencia: la rotación de la sección es igual a la pendiente de la elástica:

$$\theta(s) = \frac{dv}{ds}.$$

Válido solo para vigas **esbeltas** ($L/h \gtrsim 10$). En vigas peraltadas la cizalladura no es despreciable
→ Frame2DTimoshenko.

3.2.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s) = dv/ds$ (derivada de la flecha).

3.2.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$ (constante para interpolación lineal en u).
- Curvatura: $\kappa(s) = d^2v/ds^2$.

La deformación axial en un punto genérico de la sección es $\varepsilon(s, y) = \varepsilon_{axial}(s) - y \kappa(s)$; al integrar sobre la sección se obtienen axial (N) y momento flector (M) desacoplados.

3.2.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión: $M = EI \kappa$.

El material delega la respuesta axial; la rigidez a flexión escala con el **módulo tangente** E_t devuelto por el material (aproximación: toda la sección entra en régimen no-elástico simultáneamente — no hay plasticidad distribuida).

3.2.6 5. Equilibrio — forma débil

Principio de trabajos virtuales. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.3 Formulación numérica (FEM)

3.3.1 6. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.3.2 7. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Las rotaciones r_z son invariantes ante rotación del sistema (escalares en el plano). $\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e$.

3.3.3 8. Funciones de forma

- Axial: lineal, $N_1^u(s) = 1 - s/L$, $N_2^u(s) = s/L$.
- Transverso (Hermite cúbicos, cuatro funciones que interpolan v y θ):

$$\begin{aligned} N_1^v &= 1 - 3\xi^2 + 2\xi^3, & N_1^\theta &= L(\xi - 2\xi^2 + \xi^3), \\ N_2^v &= 3\xi^2 - 2\xi^3, & N_2^\theta &= L(-\xi^2 + \xi^3), \end{aligned}$$

con $\xi = s/L$.

3.3.4 9. Rigidez local

En el sistema local, la matriz 6×6 tiene forma cerrada por integración analítica:

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix}.$$

En no-linealidad material, $E \rightarrow E_t(\varepsilon_{axial})$ devuelto por el material — escala **todos** los términos de la matriz por igual. Esta es una aproximación: supone que la plasticidad/daño afecta uniformemente a la sección.

3.3.5 10. Fuerzas internas

En el sistema local:

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se **corrige** con el esfuerzo axial verdadero del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$. Esto permite que el material aporte σ no-lineal sin que la formulación de flexión interfiera.

3.3.6 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.3.7 12. Cuadratura

No aplica (forma cerrada para la matriz; integración analítica de los polinomios de Hermite). El campo `n_integration_points`: 1 se refiere al único estado material conceptual del elemento (la deformación axial media).

3.4 Contrato de implementación

```

name: Frame2DEuler
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano (z)" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (deformación media axial = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local - no hay distinción axial vs flexión en régimen no-elástico"

conventions:
  sign: " > 0 > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario (convención matemática estándar)"
  node_orientation: "el eje local va del nodo 1 al nodo 2; permutar nodos invierte el signo de las fuerzas internas"
  configuration: "inicial fija - L, c, s, T no se actualizan con los desplazamientos"

validity:
  - "vigas esbeltas: L/h > 10 (h = canto de la sección)"
  - "pequeños desplazamientos y pequeñas rotaciones"
  - "|_axial| < 1e-2"

out_of_scope:
  - vigas peraltadas con deformación por cortante significativa (usar Frame2DTimoshenko)
  - grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo actual)
  - plasticidad distribuida en la sección (fibras); el modelo escala rigidez global por E_t
  - pandeo (requiere rigidez geométrica, no implementada en esta viga lineal)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
  - name: respuesta axial pura
    setup: "viga horizontal, empotrada en extremo izquierdo, carga axial F en extremo derecho"
    expect: "u_x (extremo libre) = F·L/(E·A); momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: voladizo con carga transversal (flecha analítica)
    setup: "viga horizontal, empotrada en extremo izquierdo, carga vertical P en extremo derecho"
    expect: "v (extremo libre) = P·L³/(3·E·I); rotación = P·L²/(2·E·I)"
    tol_rel: 1.0e-10

```

```

- name: simetría de K
  setup: "cualquier configuración"
  expect: "K_global = K_global (elemento conservativo en régimen elástico)"
  tol_abs: 1.0e-12

references:
- "Bathe K.-J., Finite Element Procedures, §4.2 (formulación isoparamétrica) y cap. 5 (vigas)"
- "Cook R.D., Concepts and Applications of FEA, §2.7 (elementos de viga)"

```

3.5 Implementación

- **Archivo:** `solidum/elements/frame/euler.py` — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DTimoshenko` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DEuler` — hereda directamente de `Element`. La construcción de la matriz de transformación **T** se delega a `build_geometry_2d` en `solidum/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye **T** desde `alpha0` por su lógica propia).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerAcceptance` — los tres criterios físicos y el registro:
 - `test_acceptance_respuesta_axial_pura` (criterio 1) — resuelve el voladizo con `solver`, verifica $u_x(L) = FL/(EA)$.
 - `test_acceptance_voladizo_carga_transversal` (criterio 2) — carga transversal P , verifica flecha $v = PL^3/(3EI)$ y rotación $\theta = PL^2/(2EI)$.
 - `test_acceptance_simetria_K` (criterio 3) — viga oblicua ($c=0.6$, $s=0.8$) para evitar ejes canónicos, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — `autodiscover`.
- **Notas de traducción:**
 - La matriz **T** se calcula una sola vez en `__init__` sobre la configuración inicial y se guarda como atributo — coherente con el régimen geoméricamente lineal.
 - En `compute_element_state`, la componente axial de $\mathbf{F}_{\text{int,local}}$ se sobrescribe tras el producto $\mathbf{K}_{\text{local}}\mathbf{u}_{\text{local}}$ con el valor σA devuelto por el material. Esto permite usar materiales no-lineales en la rama axial sin que la parte lineal de flexión interfiera.
 - La aproximación del modelo no-lineal es que E_t escala **toda** la matriz local: no hay distinción entre rigidez axial y de flexión en régimen no-elástico. Documentado en `material_contract.nonlinearity_mode`.

3.6 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper compartido `_frame_geometry`. El método estático `_build_geometry` reimplementa la construcción de **T** dentro de la clase. `Frame2DTimoshenko` sigue en `structural.py` usando su

propio acceso al helper compartido; si mañana también se independiza, duplicará o internalizará su propia versión.

- **2026-04-21** · Tests de aceptación cubren la flecha analítica del voladizo a 8 decimales ($v = PL^3/(3EI)$) usando el solver completo del proyecto — validan no solo la cinemática del elemento sino también su integración con ensamblador y solver.

- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/{euler,timoshenko,euler_corot,_,_}`

La duplicación de `_build_geometry` entre Euler y Timoshenko se elimina: la construcción de **T** vuelve a vivir en un único lugar (`build_geometry_2d` en `_shared.py`), pero esta vez sin función helper externa flotante — pertenece al paquete `frame/`. `Frame2DEulerCorot` mantiene su propia reconstrucción de **T** desde `alpha0` porque su cinemática corotacional no se beneficia del helper común.

3.7 Frame2DTimoshenko

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.8 Especificación física

3.8.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga** según la teoría de **Timoshenko**. Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Transmite **fuerza axial, cortante transversal y momento flector**. Incluye explícitamente la **deformación por cortante transversal**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.8.2 1. Hipótesis cinemática de Timoshenko

- Las secciones transversales se mantienen **planas** tras la deformación (igual que Euler-Bernoulli).
- Las secciones **no** están obligadas a ser perpendiculares al eje neutro: se permite una **rotación adicional** respecto a la tangente.

Consecuencia: la rotación de la sección $\theta(s)$ y la pendiente de la elástica dv/ds son **variables independientes**:

$$\gamma(s) = \frac{dv}{ds} - \theta(s) \neq 0,$$

donde γ es la distorsión angular (deformación por cortante transversal). Euler-Bernoulli corresponde al caso $\gamma = 0$.

Apropiado para vigas **gruesas, cortas o peraltadas** ($L/h \lesssim 10$) donde la deformación por cortante no es despreciable.

3.8.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s)$, **independiente** de v .

3.8.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$.
- Curvatura (flexión): $\kappa(s) = d\theta/ds$.
- Cortante: $\gamma(s) = dv/ds - \theta(s)$.

3.8.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (el material entrega σ y E_t).
- Flexión: $M = EI \kappa$.
- Cortante: $V = GA_s \gamma$, con $G = E/[2(1 + \nu)]$ (módulo de corte) y A_s el **área efectiva de cortante** (menor que A por la distribución no uniforme de la tensión tangencial en la sección).

3.8.6 5. Factor de Phi (shear locking)

La formulación estándar con interpolaciones lineales de v y θ sufre **shear locking** para vigas esbeltas (la rigidez por cortante domina numéricamente aunque γ físicamente sea pequeño). La matriz local incorpora un factor correctivo:

$$\Phi = \frac{12 EI}{G A_s L^2}.$$

Para $\Phi \rightarrow 0$ (viga esbelta) la matriz se reduce a la de Euler-Bernoulli.

3.8.7 6. Equilibrio — forma débil

Principio de trabajos virtuales (PTV). El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa + V \delta \gamma) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.9 Formulación numérica (FEM)

3.9.1 7. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.9.2 8. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.9.3 9. Rigidez local con corrección por shear locking

Coefficientes:

$$a = \frac{12 EI}{L^3(1 + \Phi)}, \quad b = \frac{6 EI}{L^2(1 + \Phi)}, \quad c_m = \frac{(4 + \Phi) EI}{L(1 + \Phi)}, \quad d = \frac{(2 - \Phi) EI}{L(1 + \Phi)}.$$

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} EA/L & 0 & 0 & -EA/L & 0 & 0 \\ 0 & a & b & 0 & -a & b \\ 0 & b & c_m & 0 & -b & d \\ -EA/L & 0 & 0 & EA/L & 0 & 0 \\ 0 & -a & -b & 0 & a & -b \\ 0 & b & d & 0 & -b & c_m \end{bmatrix}.$$

Para $\Phi \rightarrow 0$: $a \rightarrow 12EI/L^3$, $b \rightarrow 6EI/L^2$, $c_m \rightarrow 4EI/L$, $d \rightarrow 2EI/L$ — se recupera Euler-Bernoulli.

3.9.4 10. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se corrige con el esfuerzo axial del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$ (permite materiales no-lineales en la rama axial).

3.9.5 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.9.6 12. Cuadratura

No aplica (forma cerrada).

3.10 Contrato de implementación

```
name: Frame2DTimoshenko
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }
  - { name: As, type: float, required: true, desc: "Área efectiva de cortante" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
```



```

material_contract:
    signature: "compute_state(, state) -> (, E_tangent, state')"
    strain_kind: "axial escalar (deformación media = (u_2 - u_1)/L en sistema local)"
    nonlinearity_model: "E_tangent escala toda la matriz local (axial, flexión y cortante por igual)"
    poisson_source: "si material.nu existe, se usa; en su defecto el parámetro `nu` del elemento con default 0.3 y advertencia por consola"

conventions:
    sign: " > 0 > 0 (elongación) en el eje axial"
    rotation_sign: "r_z positivo antihorario"
    node_orientation: "eje local del nodo 1 al nodo 2"
    configuration: "inicial fija - L, c, s, T,  $\phi$  (en forma funcional) se calculan sin actualizar"

validity:
    - "vigas peraltadas o cortas (L/h > 10) donde el cortante no es despreciable"
    - "pequeños desplazamientos y pequeñas rotaciones"
    - " $|_{axial}| < 1e-2$ "

out_of_scope:
    - "grandes rotaciones o desplazamientos (no hay variante corrotacional en el catálogo actual)"
    - "plasticidad distribuida en la sección"
    - "pandeo"
    - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
    - name: convergencia a Euler-Bernoulli para viga esbelta
      setup: "voladizo con carga transversal P, viga esbelta (L/h > 100)  $\phi \rightarrow 0$ "
      expect: " $v(L) = PL^3/(3EI)$  (solución Euler-Bernoulli) con tolerancia relativa  $1e-3$ "
      tol_rel: 1.0e-3

    - name: respuesta axial pura desacoplada
      setup: "voladizo con carga axial F"
      expect: " $u_x(L) = F \cdot L / (E \cdot A)$ ; momentos y cortantes nulos"
      tol_rel: 1.0e-10

    - name: simetría de K
      setup: "viga oblicua"
      expect: " $K_{global} = K_{global}$ "
      tol_abs: 1.0e-12

references:
    - "Reddy J.N., An Introduction to the Finite Element Method, cap. 5 (vigas de Timoshenko)"
    - "Bathe K.-J., Finite Element Procedures, §5.4 (formulación con factor  $\phi$ )"

```

3.11 Implementación

- **Archivo:** `solidum/elements/frame/timoshenko.py` — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DEuler` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DTimoshenko` — hereda directamente de `Element`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `solidum/elements/frame/_shared.py`;

además, los helpers libres de carga de cuerpo, masa consistente y traducción a `ElementForces` también viven en `_shared.py`.

- **Tests:** `tests/test_frame.py` · `TestFrame2DTimoshenkoAcceptance`:
- `test_acceptance_convergencia_euler_en_viga_esbelta` (criterio 1) — viga con L/h muy grande, verifica que la flecha tiende a $PL^3/(3EI)$ con tolerancia relativa 10^{-3} .
- `test_acceptance_respuesta_axial_pura` (criterio 2) — carga axial pura, verifica $u_x = FL/(EA)$.
- `test_acceptance_simetria_K` (criterio 3) — viga oblicua, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
- Para no colisionar con la variable cinemática c (coseno director) dentro de `compute_element_state`, el coeficiente c_m de la matriz local se llama `c_coef` en el código y `d_coef` su análogo (antes eran `c` y `d` inline, ambiguos).
- El Poisson ν se toma de `material.nu` si el material lo expone; en su defecto, del parámetro `nu` del elemento con default 0.3 y una advertencia por consola la primera vez que se construye con ese default — atajo a errores silenciosos de usuarios que olvidan especificarlo.
- La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` y se guarda como atributo (régimen geométricamente lineal).
- El factor Φ se recalcula en cada llamada porque depende de E_t devuelto por el material (en régimen no-lineal Φ cambia con el estado).

3.12 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper `_frame_geometry`. Con este movimiento, el helper compartido se elimina completamente del repositorio: `Frame2DEuler` y `Frame2DTimoshenko` replican la construcción de $\mathbf{T}$ como método estático `_build_geometry`, cada una en su clase. La duplicación (15 líneas) es el precio aceptado de la independencia mutua.
- **2026-04-21** · Test del criterio 1: se eligió L/h grande en lugar de reproducir un caso específico de libro porque la convergencia Timoshenko $\rightarrow$ Euler es el comportamiento **esperado por construcción** (factor $\Phi \rightarrow 0$). Verificarlo directamente con el solver completo valida simultáneamente la cinemática de Timoshenko, el tratamiento del shear locking y la integración con ensamblador + solver.
- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. La duplicación de `_build_geometry` ya no se justifica: ambas vigas comparten `build_geometry_2d` en `_shared.py`, helper interno al paquete (no flotante). Las clases siguen sin herencia entre sí.

3.13 Frame2DEulerCorot

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.14 Especificación física

3.14.1 0. Descripción general

Viga 2D esbelta según Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Captura grandes desplazamientos y grandes rotaciones del elemento como un todo con **pequeñas deformaciones axiales** ($|\varepsilon_{axial}| \lesssim 1\%$) y rotaciones nodales deformacionales moderadas.

Abre dominios que la viga lineal no captura: **pandeo por flexión, post-pandeo, snap-through** de arcos planos, brazos flexibles esbeltos.

3.14.2 1. Cinemática corotacional

Se separa el movimiento del elemento en:

1. **Rotación rígida** α_e : el ángulo que el eje de la barra ha girado desde la configuración inicial.
2. **Rotación deformacional de cada sección** $\bar{\theta}_i$: la rotación nodal menos la rotación rígida (es lo que genera momento flector).

Sean:

$$\alpha_0 = \arctan 2(Y_2 - Y_1, X_2 - X_1) \quad (\text{ángulo del eje en config. inicial, fijo}),$$

$$\alpha = \arctan 2(y_2 - y_1, x_2 - x_1) \quad (\text{ángulo corriente}),$$

$$\alpha_e = \alpha - \alpha_0 \quad (\text{rotación rígida del elemento}).$$

Rotaciones deformacionales:

$$\bar{\theta}_1 = \theta_1 - \alpha_e, \quad \bar{\theta}_2 = \theta_2 - \alpha_e,$$

donde $\theta_i = r_z^{(i)}$ son las rotaciones totales de los nodos.

3.14.3 2. Medidas de deformación

- Axial: $\varepsilon_{axial} = (l - L_0)/L_0$ con l longitud corriente.
- Curvatura efectiva: $\kappa = (\bar{\theta}_2 - \bar{\theta}_1)/L_0$ (constante si se usa interpolación cúbica).

3.14.4 3. Ecuaciones constitutivas

- Axial: delegado al material (σ, E_t) .
- Flexión: $M = E_t I \kappa$ (el mismo E_t del material escala la rigidez de flexión, como en **Frame2DEuler**).

3.14.5 4. Equilibrio — forma débil

PTV en configuración corriente. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma.$$

3.15 Formulación numérica (FEM)

3.15.1 5. DOFs locales deformacionales

Se reduce la descripción del estado deformacional del elemento a **3 magnitudes escalares**:

$$\mathbf{q}_l = [u_l, \bar{\theta}_1, \bar{\theta}_2]^\top, \quad u_l = l - L_0.$$

3.15.2 6. Rigidez local

En el sistema de coordenadas corrotado (eje de la barra corriente), la rigidez local 3×3 :

$$\mathbf{K}_{ll} = \begin{bmatrix} \frac{E_t A}{L_0} & 0 & 0 \\ 0 & \frac{4E_t I}{L_0} & \frac{2E_t I}{L_0} \\ 0 & \frac{2E_t I}{L_0} & \frac{4E_t I}{L_0} \end{bmatrix}.$$

Fuerzas locales $\mathbf{F}_l = \mathbf{K}_{ll} \mathbf{q}_l = [N, M_1, M_2]^\top$, con N sustituido por σA directamente del material para permitir respuestas no-lineales.

3.15.3 7. Matriz de transformación $\mathbf{B}$ (3×6)

Relación entre variaciones de $\mathbf{q}_l$ y de los DOFs globales $\delta \mathbf{u}_e$:

$$\mathbf{B} = \begin{bmatrix} -c & -s & 0 & c & s & 0 \\ -s/l & c/l & 1 & s/l & -c/l & 0 \\ -s/l & c/l & 0 & s/l & -c/l & 1 \end{bmatrix}, \quad c = \cos \alpha, \quad s = \sin \alpha.$$

c, s y l se evalúan en **configuración corriente** — esta es la esencia corrotacional.

Fuerzas internas globales:

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \mathbf{F}_l.$$

3.15.4 8. Rigidez tangente

Se descompone en contribución material más geométrica:

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma, \\ \mathbf{K}_{\text{material}} &= \mathbf{B}^\top \mathbf{K}_{ll} \mathbf{B}, \\ \mathbf{K}_\sigma &= \frac{N}{l} \mathbf{z} \mathbf{z}^\top + \frac{M_1 + M_2}{l^2} (\mathbf{r} \mathbf{z}^\top + \mathbf{z} \mathbf{r}^\top), \end{aligned}$$

con vectores geométricos de 6 componentes:

$$\begin{aligned} \mathbf{r} &= [-c, -s, 0, c, s, 0]^\top \quad (\text{dirección axial}), \\ \mathbf{z} &= [-s, c, 0, s, -c, 0]^\top \quad (\text{perpendicular}). \end{aligned}$$

La parte $(N/l)\mathbf{z}\mathbf{z}^\top$ es análoga al $\mathbf{K}_G$ de **Truss2DCorot**: captura la rigidización por tensión (y el ablandamiento precrítico por compresión, base del pandeo). La parte con momentos acopla rotaciones con traslaciones — crítica para problemas de flexión con desplazamientos significativos.

3.15.5 9. Cuadratura

No aplica (forma cerrada con Hermite cúbicos).

3.16 Contrato de implementación

```

name: Frame2DEulerCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar corotacional ( = (1 - L)/L)"
  nonlinearity_model: "E_tangent escala K_material local; el K_ geométrico depende solo de N, M, M corrientes"

conventions:
  sign: " > 0 > 0 (elongación); r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "Updated Lagrangian - l, c, s, se recalculan en cada evaluación"

validity:
  - "|_axial| 1e-2 (pequeña deformación axial)"
  - rotaciones rígidas del elemento de cualquier magnitud, pero acumuladas en |_e| < entre commits (rotaciones continuas > requerirían tracking de vueltas - fuera de alcance)
  - rotaciones deformacionales de las secciones moderadas (~|_i| 30°); para valores mayores la interpolación Hermite cúbica pierde precisión
  - vigas esbeltas (L/h 10)

out_of_scope:
  - rotaciones continuas del eje > sin pasar por commit (pendulums; aliasing en atan2)
  - deformaciones axiales grandes (requiere medida logarítmica)
  - plasticidad distribuida en la sección (fibras)
  - cortante transverso significativo (usar versión Timoshenko, pendiente)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "El ángulo _e se unwrappea a (-, ] por llamada. Si el problema tiene saltos de rotación > entre iteraciones consecutivas del solver, la formulación produce resultados incorrectos. Usar pasos de carga finos."
  - "En problemas con pandeo, la rigidez tangente se hace singular al cruzar el punto crítico. Usar `ArcLengthSolver` para seguir la rama post-crítica."

acceptance:

```

```

- name: límite lineal coincide con Frame2DEuler
  setup: "voladizo horizontal, cargas pequeñas ( ~ 1e-8, v/L ~ 1e-6)"
  expect: "K_T y F_int coinciden con Frame2DEuler lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida
  setup: "viga inicialmente horizontal, rotada rígidamente 45° (ambos nodos)"
  expect: "F_int = 0, = 0 (ningún esfuerzo interno bajo rotación rígida pura)"
  tol_abs: 1.0e-10

- name: coherencia tangente/fuerza interna (chequeo por diferencias finitas)
  setup: "configuración arbitraria no trivial; K_T analítica vs derivada numérica de
        F_int"
  expect: "||K_T_analítica - K_T_numérica|| / ||K_T_analítica|| 1e-5"
  tol_rel: 1.0e-5

- name: simetría de K
  setup: "configuración arbitraria"
  expect: "K_T = K_T"
  tol_abs: 1.0e-6

- name: Bathe-Bolourchi - cuarto de círculo # añadido 2026-05-19
  setup: "voladizo recto, 10 elementos, momento puro M = (/2)·EI/L en la punta, 8
        pasos"
  expect: "tip se mueve a (R·sin (/2)-L, R·(1-cos (/2))) con R=EI/M; _tip = /2"
  tol_abs: 0.005 # sobre L (cuerda ~0.2% en 10 elementos rectos)

- name: Bathe-Bolourchi - medio círculo # añadido 2026-05-19
  setup: "voladizo recto, 20 elementos, momento puro M = ·EI/L en la punta, 20 pasos
        "
  expect: "tip a (-L, 2L/) y _tip = - la viga se enrosca en semicírculo cerrado"
  tol_abs: 0.01 # sobre L

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §7.3 (corotational beams)"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.11"
- "Wriggers P., Nonlinear Finite Element Methods, cap. 4"

```

3.17 Implementación

- **Archivo:** `solidum/elements/frame/euler_corot.py` — convive con `Frame2DEuler` y `Frame2DTimoshenko` en el paquete `solidum/elements/frame/`, sin heredar de ninguno. Comparte con ellos los helpers libres de `_shared.py` (carga de cuerpo, masa consistente, traducción a `ElementForces`), pero no `build_geometry_2d`: reconstruye `T` desde `alpha0` por su lógica corotacional propia.
- **Clase:** `Frame2DEulerCorot` — hereda directamente de `Element`. Métodos internos `_current_geometry` (longitud, cosenos, ángulo corriente y rigid-body α_e) y `_unwrap` (reducción a $(-\pi, \pi]$).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerCorotAcceptance` — 4 criterios + registro:
- `test_acceptance_limite_lineal_coincide_con_frame2deuler`
- `test_acceptance_invariancia_rotacion_rigida`

- `test_acceptance_coherencia_tangente_fuerza_interna` — **finite-difference check** de $\mathbf{K}_T$ contra derivada numérica de $\mathbf{F}_{\text{int}}$, red de seguridad clave en esta formulación.
- `test_acceptance_simetria_K`
- `test_registro_en_registry`
- **Notas de traducción:**
- La formulación se basa en 3 DOFs deformacionales $[u, \bar{\theta}_1, \bar{\theta}_2]$ y una matriz $\mathbf{B}$ 3×6 que los relaciona con los DOFs globales. Evaluada en configuración corriente.
- **Signo de la contribución de momentos en $\mathbf{K}_\sigma$:** tras derivar $\partial(\mathbf{z}/l)/\partial \mathbf{u}_e$ analíticamente se obtiene $-(1/l^2)(\mathbf{r}\mathbf{z}^\top + \mathbf{z}\mathbf{r}^\top)$. El signo es **negativo** — un error común es ponerlo positivo. El test de diferencias finitas lo cazó en la primera implementación.
- `compute_internal_forces` proyecta $\mathbf{F}_{\text{int}}$ al sistema local corriente para reportar axial/cortante separados, utilizando la submatriz 2×2 de rotación del eje corriente.
- `_unwrap` lleva los ángulos al intervalo $(-\pi, \pi]$; la formulación es válida mientras la rotación acumulada por paso sea menor que π .

3.18 Diálogo

- **2026-04-21** · Implementación siguiendo Crisfield §7.3. Convive en `frame.py` con las vigas lineales por familia temática (todas son vigas 2D), sin herencia ni helpers compartidos.
- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. EulerCorot vive ahora en `euler_corot.py`, sin herencia con las otras vigas 2D. Pasan a compartirse los helpers libres de carga de cuerpo, masa y traducción de fuerzas a través de `_shared.py`; la lógica corotacional propia (reconstrucción de T desde `alpha0`, cinemática actual) se mantiene íntegra dentro de la clase.
- **2026-04-21** · La primera versión tenía el signo equivocado en la parte de momentos de $\mathbf{K}_\sigma$ (+ en lugar de -). El test de coherencia tangente/fuerza interna por diferencias finitas lo detectó inmediatamente: $\|\mathbf{K}_{T,\text{analítica}} - \mathbf{K}_{T,\text{FD}}\|_{\text{rel}}$ del orden de $1e-1$ en lugar de $1e-5$. Tras derivar $\partial(\mathbf{z}/l)/\partial \mathbf{u}_e$ cuidadosamente (ver notas) el signo correcto es negativo. Este episodio justifica retrospectivamente incluir un test FD como criterio de aceptación en toda formulación no-lineal geométrica futura.
- **2026-04-21** · El test del límite lineal usa $P = 10$ N para obtener $v/L \sim 10^{-4}$: suficientemente pequeño para estar en régimen lineal pero suficientemente grande para que el residuo del solver converja por debajo de `tol=1e-8` (cargas menores producen desplazamientos del orden del ruido numérico y el Newton oscila en la tolerancia relativa).

3.19 Frame3D

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.20 Especificación física

3.20.1 0. Descripción general

Elemento 1D inmerso en el espacio tridimensional que modela una **viga esbelta 3D**. Dos nodos rígidamente conectados, 6 DOFs por nodo ($u_x, u_y, u_z, r_x, r_y, r_z$). Captura seis acciones internas:

- **Axial** (fuerza normal N a lo largo del eje local x).
- **Cortante en dos planos** (V_y en el plano xy , V_z en el plano xz).
- **Flexión en dos planos** (M_z alrededor del eje local z , M_y alrededor del eje local y).
- **Torsión** (M_x alrededor del eje longitudinal).

Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones). Hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje deformado (sin cizalladura transversal).

3.20.2 1. Hipótesis

- Secciones planas y perpendiculares al eje neutro deformado (igual que Frame2DEuler).
- Flexiones en los dos planos **desacopladas** entre sí (ejes principales de inercia).
- Torsión de Saint-Venant pura (sin alabeo). Válida para secciones macizas o cerradas; abiertas de pared delgada requerirían formulación con alabeo (fuera de alcance).
- Material isótropo 1D; $G = E/[2(1 + \nu)]$.

3.20.3 2. Sistema de ejes locales

Tres ejes ortonormales en cada elemento:

- **$\hat{\mathbf{x}}$ local**: dirección del eje de la barra ($\hat{\mathbf{x}} = (\mathbf{x}_2 - \mathbf{x}_1)/L$).
- **$\hat{\mathbf{y}}$ local y $\hat{\mathbf{z}}$ local**: ejes principales de inercia de la sección, ortogonales al eje.

La orientación de $\hat{\mathbf{y}}, \hat{\mathbf{z}}$ no está determinada solo por la dirección del eje — hace falta un **vector de referencia** proporcionado por el usuario (o un default). Es la diferencia estructural respecto al caso 2D, donde el plano era único.

3.20.4 3. Medidas de deformación / acciones internas

- Axial: $\varepsilon_{axial} = du/ds$, tensión $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión en plano xy : curvatura $\kappa_z = d^2v/ds^2$, momento $M_z = EI_z \kappa_z$.
- Flexión en plano xz : curvatura $\kappa_y = d^2w/ds^2$, momento $M_y = EI_y \kappa_y$.
- Torsión: $d\theta_x/ds$, momento torsor $M_x = GJ d\theta_x/ds$.

3.20.5 4. Equilibrio — forma débil

PTV. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon + M_z \delta \kappa_z + M_y \delta \kappa_y + M_x \delta \theta'_x) ds.$$

3.21 Formulación numérica (FEM)

3.21.1 5. Discretización

Dos nodos, 6 DOFs por nodo:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, r_x^{(1)}, r_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}, r_x^{(2)}, r_y^{(2)}, r_z^{(2)}]^\top.$$

Vector de 12 componentes $\rightarrow$ matrices 12×12 .

3.21.2 6. Construcción de ejes locales

Dados $\mathbf{X}_1, \mathbf{X}_2$ y un vector de referencia $\mathbf{v}_{\text{ref}}$ proporcionado:

1. $\hat{\mathbf{x}} = (\mathbf{X}_2 - \mathbf{X}_1)/L$.
2. Proyectar $\mathbf{v}_{\text{ref}}$ al plano perpendicular a $\hat{\mathbf{x}}$:

$$\tilde{\mathbf{y}} = \mathbf{v}_{\text{ref}} - (\mathbf{v}_{\text{ref}} \cdot \hat{\mathbf{x}}) \hat{\mathbf{x}}.$$

1. $\hat{\mathbf{y}} = \tilde{\mathbf{y}}/\|\tilde{\mathbf{y}}\|$ (falla si $\|\tilde{\mathbf{y}}\| \approx 0$: $\mathbf{v}_{\text{ref}}$ paralelo al eje).
2. $\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$.

Default de $\mathbf{v}_{\text{ref}}$ (cuando el usuario no lo proporciona): $[0, 0, 1]$ (eje z global como "vertical"). Si el eje del elemento es cercanamente paralelo a z global (viga vertical), se usa $[1, 0, 0]$ como fallback y se emite una advertencia.

3.21.3 7. Matriz de transformación global $\rightarrow$ local

$$\boldsymbol{\lambda} = \begin{bmatrix} \hat{\mathbf{x}}^\top \\ \hat{\mathbf{y}}^\top \\ \hat{\mathbf{z}}^\top \end{bmatrix} \quad (3 \times 3, \text{ ortogonal}).$$

$$\mathbf{T} = \text{bloques diag}(\boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}) \quad (12 \times 12).$$

Aplicada a los 4 bloques de 3 DOFs: desplazamientos nodo 1, rotaciones nodo 1, desplazamientos nodo 2, rotaciones nodo 2. Las rotaciones infinitesimales en 3D transforman como vectores.

$$\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.21.4 8. Matriz de rigidez local

Desacoplada en cuatro contribuciones: axial, torsión, flexión xy (plano con u_y y r_z , inercia I_z), flexión xz (plano con u_z y r_y , inercia I_y).

Definiciones:

$$a_z = \frac{12EI_z}{L^3}, \quad b_z = \frac{6EI_z}{L^2}, \quad c_z = \frac{4EI_z}{L}, \quad d_z = \frac{2EI_z}{L},$$

$$a_y = \frac{12EI_y}{L^3}, \quad b_y = \frac{6EI_y}{L^2}, \quad c_y = \frac{4EI_y}{L}, \quad d_y = \frac{2EI_y}{L}.$$

La rigidez $\mathbf{K}_{\text{local}}$ es simétrica, con los bloques desacoplados (0s fuera de su plano correspondiente). Los signos del bloque de flexión xz se invierten respecto al xy porque u_z y r_y forman una pareja dextrógira opuesta (consecuencia del producto vectorial $\hat{\mathbf{x}} \times \hat{\mathbf{y}} = \hat{\mathbf{z}}$).

Forma concreta en términos de los 12 DOFs locales (la matriz completa vive en la implementación).

3.21.5 9. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}},$$

con la componente axial corregida por el σ que devuelve el material: $F_1 = -\sigma A$, $F_7 = +\sigma A$. El resto de componentes viene del producto lineal (permite materiales no-lineales en la rama axial).

3.21.6 10. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.21.7 11. Cuadratura

No aplica (forma cerrada; integración analítica de los polinomios de Hermite).

3.22 Contrato de implementación

```
name: Frame3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz, rx, ry, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: Iy, type: float, required: true, desc: "Momento de inercia alrededor del eje local y" }
  - { name: Iz, type: float, required: true, desc: "Momento de inercia alrededor del eje local z" }
  - { name: J, type: float, required: true, desc: "Constante torsional de Saint-Venant" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
  - { name: ref_vector, type: list, required: false, default: "[0, 0, 1]",
```

```

    desc: "Vector 3D de referencia que fija la orientación de los ejes locales y, z
          de la sección. Default: eje z global." }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (= (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local; G se recalcula como E_t
                      /[2(1+)]"
  poisson_source: "material.nu si existe; en su defecto parámetro `nu` del elemento"

conventions:
  sign: " > 0 > 0 (elongación); momentos siguen regla de la mano derecha respecto a
        los ejes locales"
  local_axes: "x_local = eje de la barra; y_local, z_local = ejes principales de
              inercia definidos por ref_vector"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, T, ejes locales no se actualizan"

validity:
  - "vigas esbeltas (L/h > 10 en ambos planos de flexión)"
  - "pequeños desplazamientos y pequeñas rotaciones en el espacio"
  - "|_axial| < 1e-2"
  - "sección con ejes principales de inercia bien definidos por ref_vector"

out_of_scope:
  - grandes rotaciones espaciales (requeriría Frame3DCorot; pendiente)
  - torsión con alabeo restringido (secciones abiertas de pared delgada)
  - plasticidad distribuida en la sección (fibras)
  - pandeo por bifurcación
  - deformación por cortante transversal significativa (Timoshenko 3D, fuera de alcance)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "Si ref_vector es paralelo al eje de la barra, la proyección al plano perpendicular
    se anula y el constructor aborta con mensaje claro - el usuario debe elegir otro
    vector."
  - "Para barras verticales (eje casi paralelo a z global) con ref_vector default, el
    constructor usa [1, 0, 0] como fallback y emite advertencia por consola."

acceptance:
  - name: respuesta axial pura
    setup: "voladizo alineado con eje x, carga F axial en extremo libre"
    expect: "u_x = F·L/(E·A); demás componentes nulas"
    tol_rel: 1.0e-10

  - name: flexión en plano xy (coherencia con Frame2D)
    setup: "voladizo alineado con eje x, carga P en dirección y, ref_vector = [0,0,1]"
    expect: "v_y = P·L³/(3·E·Iz), _z = P·L²/(2·E·Iz)"
    tol_rel: 1.0e-10

  - name: flexión en plano xz
    setup: "voladizo alineado con eje x, carga P en dirección z, ref_vector = [0,1,0]"
    expect: "v_z = P·L³/(3·E·Iy), _y = -P·L²/(2·E·Iy) (signo por mano derecha)"
    tol_rel: 1.0e-10

  - name: torsión pura
    setup: "voladizo con momento T aplicado alrededor del eje de la barra en el extremo
          libre"

```

```

expect: "_x = T·L/(G·J); demás componentes nulas"
tol_rel: 1.0e-10

- name: simetría de K
  setup: "viga en orientación arbitraria con ref_vector genérico"
  expect: "K_global = K_global"
  tol_abs: 1.0e-8

references:
- "Przemieniecki J.S., Theory of Matrix Structural Analysis, Tabla 11.3"
- "Cook R.D., Concepts and Applications of FEA, §2.8"
- "Bathe K.-J., Finite Element Procedures, cap. 5"

```

3.23 Implementación

- **Archivo:** solidum/elements/frame3d.py — archivo propio, sin dependencia de ningún otro elemento.
- **Clase:** Frame3D — hereda directamente de Element. Métodos privados `_build_local_frame` (longitud, matriz λ 3×3 y $\mathbf{T}$ 12×12) y `_build_local_stiffness` (matriz $\mathbf{K}_{\text{local}}$ 12×12 con bloques axial, torsión y flexión en ambos planos).
- **Tests:** tests/test_frame3d.py · TestFrame3DAcceptance — cubre los 5 criterios + validación de `ref_vector` paralelo al eje + registro:
- test_acceptance_respuesta_axial_pura
- test_acceptance_flexion_xy_coincide_con_frame2d
- test_acceptance_flexion_xz
- test_acceptance_torsion_pura
- test_acceptance_simetria_K
- test_rechazo_ref_vector_paralelo
- test_registro_en_registry
- **Notas de traducción:**
- La matriz λ 3×3 se construye como $[\hat{\mathbf{x}}^\top; \hat{\mathbf{y}}^\top; \hat{\mathbf{z}}^\top]$ — las filas son los ejes locales expresados en coordenadas globales. $\mathbf{T}$ 12×12 es un `blkdiag` de cuatro copias de λ (traslaciones y rotaciones transforman como vectores en 3D infinitesimal).
- El default de `ref_vector` es $[0, 0, 1]$; el fallback $[1, 0, 0]$ se activa cuando $|\hat{\mathbf{x}} \cdot \hat{\mathbf{z}}_{\text{global}}| > 0,99$ (barra cercanamente vertical). Ambas rutas emiten advertencia por consola la primera vez que se usan con default.
- La matriz $\mathbf{K}_{\text{local}}$ se construye rellenando entradas no nulas en sus 4 bloques desacoplados (axial, torsión, flexión xy , flexión xz). Los signos de la flexión xz están invertidos respecto a la flexión xy porque la pareja (u_z, r_y) tiene orientación dextrógira opuesta a (u_y, r_z) : este punto se verifica explícitamente en `test_acceptance_flexion_xz`.
- `compute_internal_forces` devuelve cortantes, torsión y momentos en ambos extremos separados — útil para post-proceso estructural (diagramas de fuerzas internas).

3.24 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente autónomo: no hereda de `Frame2DEuler` ni de ninguna otra viga. La maquinaria cinemática 3D se implementa íntegra dentro de `frame3d.py`, coherente con la filosofía aplicada a cables.
- **2026-04-21** · Decisión sobre `ref_vector` vs "tercer nodo de orientación". Se optó por `ref_vector` (vector 3D proyectado al plano perpendicular al eje) por ser más compacto en YAML y no requerir nodos fantasma en la malla. El default `[0, 0, 1]` cubre la mayoría de casos de ingeniería estructural (estructuras terrestres con gravedad en $-z$); el fallback para barras verticales usa `[1, 0, 0]` con advertencia explícita.
- **2026-04-21** · Verificación cruzada con `Frame2DEuler`: el criterio 2 aplica una viga alineada con el eje x global bajo carga en y ; los desplazamientos resultantes deben coincidir con la solución 2D clásica $v = PL^3/(3EI_z)$, $\theta = PL^2/(2EI_z)$. El test lo verifica a 8 decimales. Esto blindo la coherencia dimensional: el 3D se reduce al 2D sin error acumulado cuando el problema es efectivamente plano.

Capítulo 4

Elementos 2D — Sólidos

4.1 Quad4

Documentación retroactiva del elemento ya implementado. Cubre la formulación, el contrato de implementación y la trazabilidad con tests existentes.

4.2 Especificación física

4.2.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, cuadrilátero de cuatro nodos. Aproximación bilineal C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

4.2.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_y^{(i)}.$$

4.2.3 2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}.$$

4.2.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). El elemento llama `material.compute_state( $\varepsilon$ , state)  $\rightarrow$  ( $\sigma$ , C_tangent, state')` por punto de Gauss y soporta cualquier ley constitutiva 2D (Elastic2D, IsotropicDamage2D, VonMises2D).

4.2.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.2.6 5. Equilibrio — forma débil

$$\int_{\Omega} \delta \boldsymbol{\varepsilon}^{\top} \boldsymbol{\sigma} dV = \int_{\Omega} \delta \mathbf{u}^{\top} \mathbf{b} dV + \int_{\partial\Omega_N} \delta \mathbf{u}^{\top} \bar{\mathbf{t}} dS, \quad \forall \delta \mathbf{u} \in V_0.$$

4.3 Formulación numérica (FEM)

4.3.1 6. Discretización

Cuatro nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el cuadrado de referencia $[-1, 1]^2$:

$$x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) x^{(i)}, \quad y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) y^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x, y)/\partial(\xi, \eta)$ y su determinante $\det \mathbf{J}$ entran en el cambio de variable de la integración. El código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (elemento degenerado o nodos en orden invertido).

4.3.2 7. Funciones de forma

Producto tensorial bilineal:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta), \quad (\xi_i, \eta_i) = (\pm 1, \pm 1).$$

Convención de numeración (antihorario): $(\xi_1, \eta_1) = (-1, -1)$, $(\xi_2, \eta_2) = (+1, -1)$, $(\xi_3, \eta_3) = (+1, +1)$, $(\xi_4, \eta_4) = (-1, +1)$.

4.3.3 8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i / \partial x = \mathbf{J}^{-1} \partial N_i / \partial \xi$. La matriz $\mathbf{B}$ de tamaño 3×8 se ensambla por nodo:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial y \\ \partial N_i / \partial y & \partial N_i / \partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

4.3.4 9. Rigidez elemental

$$\mathbf{K}_e = \int_{\Omega} \mathbf{B}^{\top} \mathbf{D} \mathbf{B} t dA = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^{\top} \mathbf{D}_p \mathbf{B}(\xi_p, \eta_p) \det \mathbf{J}_p w_p t,$$

donde t es el espesor (**thickness**) y $\mathbf{D}_p$ es el tensor constitutivo tangente devuelto por el material en cada punto.

4.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^\top \boldsymbol{\sigma}_p \det \mathbf{J}_p w_p t.$$

4.3.6 11. Cuadratura

Por defecto Gauss–Legendre 2×2 (cuatro puntos): orden de exactitud 3 — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelogramo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro `quadrature` (lista nombrada en `QuadratureRegistry`).

Aviso sobre integración reducida: el esquema 1×1 está disponible pero el código emite advertencia explícita; un único punto central no detecta los modos de hourglass (energía nula con desplazamientos de signo alternado), que pueden contaminar la respuesta sin estabilización adicional. No se implementa estabilización tipo Flanagan-Belytschko.

4.3.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, -\rho g)$, fuerza centrífuga). Vector nodal equivalente:

$$\mathbf{f}_e = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA = \sum_{p=1}^{n_g} \mathbf{N}(\xi_p, \eta_p)^\top \mathbf{b} \det \mathbf{J}_p w_p t,$$

donde $\mathbf{N}$ es la matriz 2×8 de funciones de forma. Se reutiliza la cuadratura del elemento (Gauss 2×2 por defecto) — exacta para $\mathbf{b}$ uniforme y geometrías de paralelogramo, y adecuada para geometrías irregulares moderadas.

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde $\Gamma_e \subset \partial\Omega_N$:

$$\mathbf{f}_e = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}} t dS.$$

Cada borde es una recta entre dos nodos; sobre él las funciones de forma son lineales y, para $\bar{\mathbf{t}}$ constante, la integral se reduce a un reparto exacto de $\frac{L}{2} \bar{\mathbf{t}} t$ a cada uno de los dos nodos del borde, donde L es la longitud del segmento. Bordes numerados según la conectividad nodal:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 3)
3	(3, 0)

$\bar{\mathbf{t}}$ se especifica en **coordenadas globales** (t_x, t_y) ; presiones normales se obtienen multiplicando previamente por la normal exterior del borde. Tracción variable a lo largo del borde no está soportada en este paso.

4.3.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento, junto con sus coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, extrapolación tipo Barlow). `compute_internal_forces` queda como atajo de promedio.

4.4 Contrato de implementación

```

name: Quad4
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 4
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 4 # default Gauss 2x2

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
    estado plano }
  - { name: quadrature, type: tuple, required: false, default: "2x2", desc: Regla de
    cuadratura desde QuadratureRegistry }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state')"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy (engineering shear strain)"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales Elastic2D, IsotropicDamage2D, VonMises2D (este último
    solo plane_strain)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales o totales lagrangianas"
  - "estabilización contra hourglass para integración reducida"
  - "elementos de alto orden (Q8, Q9)"

acceptance:
  verification:
    - name: patch_test_macneal_harder
      setup: "5 Quad4 distorsionados (4 nodos exteriores + 4 interiores)
        con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
        impuesto en los 4 nodos del contorno"
      expect: "nodos interiores adoptan el campo lineal y = (1e-3, 1e-3,
        1e-3) en todos los puntos de Gauss; uniforme"
      tol_rel: 1.0e-10
  specific:
    - name: parche de tracción uniaxial sobre malla regular
      setup: "malla regular con campo de desplazamiento lineal y material Elastic2D"
      expect: "_xx constante en todos los puntos de Gauss"
      tol_rel: 1.0e-12
    - name: jacobiano degenerado abortado
      setup: "elemento con nodos colineales o en orden inverso"
      expect: "ValueError en _compute_kinematics"
    - name: cargas consistentes - body load uniforme
      setup: "Quad4 regular y distorsionado con b uniforme; sumar componentes nodales"
      expect: "Σf_x = b_x·A·t y Σf_y = b_y·A·t (invariante de geometría)"

```

```

    tol_rel: 1.0e-10
- name: cargas consistentes - tracción uniforme en un borde
  setup: "Quad4 con tracción constante en un borde"
  expect: " $\Sigma f = t \cdot L \cdot t$  y reparto  $L/2$  a cada nodo del borde, 0 en los demás"
  tol_rel: 1.0e-12
- name: patch físico de tracción uniaxial
  setup: "Quad4 cuadrado  $L \times H$ , borde izquierdo con  $u_x=0$  (n0,n3) +  $u_y=0$  (n0); borde
    derecho con tracción uniforme (p,0)"
  expect: "u reproduce  $u_x=p \cdot x/E$ ,  $u_y=-p \cdot y/E$  (plane stress);  $_{xx}=p$ ,  $_{yy}=_{xy}=0$ "
  tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §5.3 (isoparametric elements)"
- "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6 (plane stress/
  strain)"

```

4.5 Implementación

- **Archivo:** solidum/elements/solid_2d/quad4.py
- **Clase:** Quad4 (registrada vía `@ElementRegistry.register`)
- **Funciones núcleo:** `_compute_kinematics(xi, eta, coords)` y `_compute_integrands(B, C,  $\sigma$ , detJ, w, t)`, ambas decoradas con `@njit` (Numba) para evaluar **B**, **det J** y los integrandos a velocidad cercana a Fortran. Viven en solidum/elements/solid_2d/_shared.py y se comparten con Tri3.
- **Tests:**
- tests/test_solid_2d.py — ensamblaje de **B**, simetría de **K_e**, modos de cuerpo rígido y tracción uniaxial sobre malla regular.
- tests/test_patch_solid_2d.py — patch test de MacNeal-Harder sobre 5 Quad4 distorsionados (NAFEMS, V&V).

4.6 Diálogo

- **2026-04-30** · Spec retroactiva. Quad4 precedía al protocolo spec-first; la spec se creó documentando la formulación ya implementada y verificada. Promovido **status: validated**.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en **acceptance.verification**. Cinco Quad4 distorsionados con cuatro nodos interiores libres reproducen exactamente un campo lineal impuesto en el contorno, con ε constante e igual al gradiente analítico en todos los puntos de Gauss.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` con σ y ε por punto de Gauss y coordenadas naturales/globales. `compute_internal_forces` queda como promedio derivado. El `VtkExporter` añade campos nodales `Sigma_XX_nodal`, `Sigma_YY_nodal`, `Tau_XY_nodal` y `Von_Mises_nodal` por promedio simple sobre los elementos contiguos a cada nodo (suavizado de orden 0).

- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`). Para tracciones uniformes en bordes rectos el reparto es exacto ($L/2$ a cada nodo del borde) y se evita la cuadratura 1D; las fuerzas de cuerpo se integran con la cuadratura del elemento. Solo tracciones **constantes por borde** y en **coordenadas globales** en este paso; tracción variable y presión normal quedan para una iteración posterior.

4.7 Tri3

Documentación retroactiva del elemento ya implementado.

4.8 Especificación física

4.8.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, triangular de tres nodos. Aproximación lineal C^0 del campo de desplazamientos en coordenadas naturales (ξ, η) . Conocido en la literatura como elemento *constant strain triangle* (CST): la matriz $\mathbf{B}$ es constante sobre el elemento, por lo que la deformación $\boldsymbol{\varepsilon}$ y la tensión $\boldsymbol{\sigma}$ son uniformes dentro de cada Tri3. Régimen de pequeños desplazamientos y deformaciones.

4.8.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_y^{(i)}.$$

4.8.3 2. Cinemática de deformaciones

Notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica a la de Quad4. Por la linealidad de N_i , las derivadas $\partial N_i / \partial x$ son constantes y $\boldsymbol{\varepsilon}$ es uniforme en el elemento.

4.8.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). Soporta Elastic2D, IsotropicDamage2D, VonMises2D (este último solo plane_strain).

4.8.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.8.6 5. Equilibrio — forma débil

Idéntica a la del Quad4; el principio variacional no depende de la forma del elemento.

4.9 Formulación numérica (FEM)

4.9.1 6. Discretización

Tres nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el triángulo de referencia con vértices en $(0, 0)$, $(1, 0)$, $(0, 1)$. Jacobiano $\mathbf{J} = \partial(x, y) / \partial(\xi, \eta)$ es constante sobre el elemento; el código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (nodos colineales o invertidos).

4.9.2 7. Funciones de forma

Coordenadas baricéntricas:

$$N_1 = 1 - \xi - \eta, \quad N_2 = \xi, \quad N_3 = \eta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\partial N_1 / \partial \xi = -1, \quad \partial N_2 / \partial \xi = 1, \quad \partial N_3 / \partial \xi = 0;$$

$$\partial N_1 / \partial \eta = -1, \quad \partial N_2 / \partial \eta = 0, \quad \partial N_3 / \partial \eta = 1.$$

4.9.3 8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 3×6 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial y \\ \partial N_i / \partial y & \partial N_i / \partial x \end{bmatrix}.$$

4.9.4 9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D} \mathbf{B} A_e t,$$

con $A_e = \frac{1}{2} \det \mathbf{J}$ el área del triángulo y t el espesor. La integración es exacta con un único punto central porque $\mathbf{B}$ es constante.

4.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma} A_e t.$$

4.9.6 11. Cuadratura

Un punto central (en el baricentro) con peso $w = 1/2$ sobre el triángulo de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

4.9.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA \implies \text{cada nodo recibe } \frac{1}{3} \mathbf{b} A_e t.$$

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde recto. Con $\bar{\mathbf{t}}$ constante el reparto es $\frac{L}{2} \bar{\mathbf{t}} t$ en cada uno de los dos nodos del borde. Bordes numerados según la conectividad:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 0)

$\bar{\mathbf{t}}$ se especifica en coordenadas globales (t_x, t_y) . Tracción variable o presión normal no soportadas en este paso.

4.9.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que en `Quad4` pero con un único punto (centroide). Para `Tri3` ε y σ son constantes sobre el elemento, así que el dato del punto coincide con el promedio.

4.10 Limitación crítica — *shear locking*

El `Tri3` es notoriamente rígido en flexión: tres DOFs de desplazamiento no bastan para representar el modo de flexión puro de una viga discretizada en triángulos, por lo que el elemento responde con cortante espurio (*shear locking*) y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema. Recomendación operativa: usar `Quad4` por defecto; reservar `Tri3` para zonas de transición de malla en regiones donde el campo de tensiones varía suavemente.

4.11 Contrato de implementación

```
name: Tri3
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 3
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 1

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
      estado plano }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "campos de tensión que varían suavemente sobre el elemento"

out_of_scope:
  - "flexión dominante (shear locking severo)"
  - "elementos de alto orden (Tri6)"
  - "grandes deformaciones"

acceptance:
  verification:
    - name: patch_test_macneal_harder
      setup: "4 Tri3 distorsionados alrededor de un nodo interior descentrado
              con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
              impuesto en los 4 nodos del contorno del cuadrado unitario"
      expect: "nodo interior adopta el campo lineal y = (1e-3, 1e-3,"
```

```

        1e-3) en cada elemento;  uniforme"
    tol_rel: 1.0e-10
specific:
- name: parche de deformación constante sobre malla triangular regular
  setup: "malla triangular con campo de desplazamiento lineal y material Elastic2D"
  expect: " constante (CST reproduce campos lineales por construcción)"
  tol_rel: 1.0e-12
- name: jacobiano degenerado abortado
  setup: "tres nodos colineales"
  expect: "ValueError en _compute_kinematics_tri3"
- name: cargas consistentes - body load uniforme
  setup: "Tri3 con b uniforme"
  expect: "cada nodo recibe (1/3)·b·A_e·t;  $\Sigma = b \cdot A_e \cdot t$ "
  tol_rel: 1.0e-12
- name: cargas consistentes - tracción uniforme en un borde
  setup: "Tri3 con tracción constante en un borde"
  expect: " $\Sigma_f = t \cdot L \cdot t$  y reparto L/2 a cada nodo del borde, 0 en el opuesto"
  tol_rel: 1.0e-12

references:
- "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §3.6 (CST)"
- "Zienkiewicz O.C., The Finite Element Method, vol. 1, §5 (limitations of CST)"

```

4.12 Implementación

- **Archivo:** solidum/elements/solid_2d/tri3.py
- **Clase:** Tri3 (registrada vía `@ElementRegistry.register`)
- **Función núcleo:** `_compute_kinematics_tri3(coords)`, decorada con `@njit`, en `_shared.py`. Reutiliza `_compute_integrands` (también en `_shared`, compartido con Quad4) con peso $w = 0.5$.
- **Tests:**
- `tests/test_solid_2d.py` — patch test sobre malla triangular regular y modos de cuerpo rígido.
- `tests/test_patch_solid_2d.py` — patch test de MacNeal-Harder con nodo interior libre (NAFEMS, V&V).

4.13 Diálogo

- **2026-04-30** · Spec retroactiva. Tri3 precedía al protocolo spec-first; la spec se creó documentando la formulación ya implementada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verificaton`. Cuatro triángulos alrededor de un nodo interior descentrado reproducen un campo lineal impuesto en el contorno con ε constante en cada elemento.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` (1 punto central). El `VtkExporter` consume σ por elemento y promedia a nodos (`Sigma_*_nodal`).

- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`) con la misma semántica que `Quad4`. Para `Tri3` ambos integrandos son exactos analíticamente: body load reparte $1/3$ por nodo, tracción de borde reparte $L/2$ a cada uno de los dos nodos del borde.

4.14 Quad8

Elemento sólido 2D de orden cuadrático con interpolación serendípita.

*Reproduce exactamente campos hasta Q_2^{seren} (todos los polinomios de grado total ≤ 2 más algunos de orden 3 sin el término $\xi^2\eta^2$). Mucho más preciso que **Quad4** en flexión y geometrías curvas con el mismo grado de malla.*

4.15 Especificación física

4.15.1 0. Descripción general

Cuadrilátero plano de 8 nodos: 4 vértices + 4 nodos en los puntos medios de los bordes. Pequeñas deformaciones, isoparamétrico.

4.15.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^8 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^8 N_i(\xi, \eta) u_y^{(i)}.$$

4.15.3 2. Cinemática de deformaciones

Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica al Quad4.

4.15.4 3. Ecuación constitutiva

Material 2D (STRAIN_DIM = 3).

4.15.5 4-5. Equilibrio

Idénticas a Quad4.

4.16 Formulación numérica

4.16.1 6. Discretización

Nodos en orden estándar: 0-3 vértices antihorarios desde $(-1, -1)$; 4 medio del borde 0-1, 5 medio del 1-2, 6 medio del 2-3, 7 medio del 3-0. Mapeo isoparamétrico.

4.16.2 7. Funciones de forma serendípitas

Vértices ($i = 0, 1, 2, 3$ con $(\xi_i, \eta_i) \in \{\pm 1\}^2$):

$$N_i = \frac{1}{4}(1 + \xi_i\xi)(1 + \eta_i\eta)(\xi_i\xi + \eta_i\eta - 1).$$

Medios de borde:

- Bordes con $\xi = 0$ (nodos 4, 6): $N_i = \frac{1}{2}(1 - \xi^2)(1 + \eta_i\eta)$ con $\eta_i = \mp 1$.
- Bordes con $\eta = 0$ (nodos 5, 7): $N_i = \frac{1}{2}(1 + \xi_i\xi)(1 - \eta^2)$ con $\xi_i = \pm 1$.

4.16.3 8. Matriz $\mathbf{B}$

Ensamblaje estándar a partir de $\partial N_i / \partial x = \mathbf{J}^{-1} \partial N_i / \partial \xi$. Tamaño 3×16 .

4.16.4 9-10. Rigidez y fuerzas internas

$$\mathbf{K}_e = \int \mathbf{B}^\top \mathbf{D} \mathbf{B} t dA, \quad \mathbf{F}_{\text{int}}^e = \int \mathbf{B}^\top \boldsymbol{\sigma} t dA.$$

Cuadratura Gauss 3×3 por defecto.

4.16.5 11. Cuadratura

Gauss-Legendre 3×3 (9 puntos) — exacta para polinomios hasta grado 5, suficiente para integrar $\mathbf{B}^\top \mathbf{D} \mathbf{B}$ del Quad8 sobre Jacobiano constante. La opción 2x2 queda disponible (reducida) pero introduce modos espurios; emite advertencia.

4.16.6 12. Cargas distribuidas consistentes

- **Body load:** $\int N^\top \mathbf{b} t dA$ con la cuadratura del elemento (3×3).
- **Edge traction** uniforme: en un borde recto con N cuadráticas la integral analítica reparte $L/6$ a cada nodo extremo y $4L/6$ al nodo medio (regla 1-4-1, Simpson). Bordes:

Edge	Nodos extremos	Nodo medio
0	(0, 1)	4
1	(1, 2)	5
2	(2, 3)	6
3	(3, 0)	7

4.16.7 13. Salida por punto de Gauss

`compute_gauss_state(U)` con la misma estructura que Quad4: 9 puntos por defecto.

4.17 Contrato de implementación

```
name: Quad8
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 8
  strain_dim: 3
  n_integration_points: 9      # default Gauss 3x3

parameters:
  - { name: thickness, type: float, required: false, default: 1.0 }
  - { name: quadrature, type: tuple, required: false, default: "3x3" }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
```

```

strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy"
  node_orientation: "vértices antihorarios; medios en el orden de los bordes"

validity:
  - "pequeños desplazamientos y deformaciones"

out_of_scope:
  - "grandes deformaciones"
  - "estabilización contra modos espurios bajo integración reducida"

acceptance:
  verification:
    - name: patch_lineal
      setup: "campo u = a + a·x + a·y impuesto en el contorno"
      expect: "constante en todos los Gauss e igual al gradiente analítico"
      tol_rel: 1.0e-10
    - name: patch_cuadratico
      setup: "campo u_x = x2·1e-3 impuesto en los 8 nodos"
      expect: "_xx(x, y) = 2e-3·x exacto en todos los puntos de Gauss"
      tol_rel: 1.0e-10
  specific:
    - name: cargas consistentes - body load uniforme
      setup: "Σf = b·A·t (regular y distorsionado)"
      tol_rel: 1.0e-10
    - name: cargas consistentes - tracción uniforme en un borde
      setup: "Σf = t·L·t y reparto 1/6, 4/6, 1/6 en (vértice, medio, vértice)"
      tol_rel: 1.0e-12
    - name: Cook's membrane (Cook 1974) # 2026-05-19
      setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, ν=1/3, plane stress,
             cortante total F=1 distribuido en borde derecho; malla 4×4 Q8"
      expect: "u_y en (48,52) 23.91 ± 0.5; refinamiento 2×→24×→46×6 reduce error monot
             ónicamente"
      tol_abs: 0.5

references:
  - "Bathe K.-J., Finite Element Procedures, §5.3"
  - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6"
  - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §8"

```

4.18 Implementación

- **Archivo:** solidum/elements/solid_2d/quad8.py · clase Quad8 (subclase de la base interna _HigherOrderSolid2D en _shared.py, compartida con Quad9 y Tri6).
- **Tests:** tests/test_higher_order_solid_2d.py.

4.19 Quad9

Variante Lagrangiana del Quad8: añade un noveno nodo central interior.

Espacio polinómico completo Q_2 (incluye el término $\xi^2\eta^2$ que el serendípito omite).

4.20 Especificación física

Idéntica a Quad8 salvo por las funciones de forma.

4.20.1 7. Funciones de forma

Producto tensorial de los polinomios de Lagrange 1D de orden 2:

$$L_0(\xi) = \frac{1}{2}\xi(\xi - 1), \quad L_1(\xi) = 1 - \xi^2, \quad L_2(\xi) = \frac{1}{2}\xi(\xi + 1).$$

$N_i(\xi, \eta) = L_a(\xi) L_b(\eta)$ con (a, b) asociado al nodo i según la numeración:

- 0..3: vértices $(-1, -1)$, $(+1, -1)$, $(+1, +1)$, $(-1, +1)$.
- 4: medio del borde 0-1 ($\eta = -1$).
- 5: medio del borde 1-2 ($\xi = +1$).
- 6: medio del borde 2-3 ($\eta = +1$).
- 7: medio del borde 3-0 ($\xi = -1$).
- 8: nodo central interior $(\xi, \eta) = (0, 0)$ — *bubble*.

4.20.2 11. Cuadratura

Gauss 3×3 por defecto.

4.20.3 12. Cargas distribuidas

Body load con la cuadratura del elemento. Tracción en bordes idéntica a Quad8 (1/6, 4/6, 1/6 — el nodo 8 no participa, no toca bordes).

4.21 Contrato de implementación

```
name: Quad9
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 9
  strain_dim: 3
  n_integration_points: 9

acceptance:
  verification:
    - name: patch_cuadratico
```

```
setup: "u = (x2, 0) impuesto en los 9 nodos"
expect: "_xx = 2x exacto en todos los Gauss"
tol_rel: 1.0e-10
- name: Cook's membrane (Cook 1974) # 2026-05-19
  setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, =1/3, plane stress,
         cortante total F=1 distribuido en borde derecho; malla 4x4 Q9"
  expect: "u_y en (48,52) 23.91 ± 0.5"
  tol_abs: 0.5
```

4.22 Implementación

- **Archivo:** solidum/elements/solid_2d/quad9.py · clase `Quad9` (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con `Quad8` y `Tri6`).
- **Tests:** tests/test_higher_order_solid_2d.py.

4.23 Tri6

*Triángulo isoparamétrico de 6 nodos. Cura el shear locking severo del **Tri3** y reproduce campos cuadráticos exactamente. Útil cuando la malla es triangular por geometría.*

4.24 Especificación física

Idéntica al **Tri3** salvo por la cinemática y la cuadratura.

4.24.1 7. Funciones de forma

Coordenadas baricéntricas $L_1 = 1 - \xi - \eta$, $L_2 = \xi$, $L_3 = \eta$:

$$N_i^{vert} = L_i(2L_i - 1), \quad i = 1, 2, 3$$

$$N_4 = 4L_1L_2, \quad N_5 = 4L_2L_3, \quad N_6 = 4L_3L_1.$$

Numeración: 0,1,2 vértices en (0,0), (1,0), (0,1); 3 medio del borde 0-1, 4 medio del 1-2, 5 medio del 2-0.

4.24.2 11. Cuadratura

Cuadratura triangular de 3 puntos (puntos medios de los lados; peso 1/6 cada uno) — exacta para polinomios hasta grado 2 sobre el triángulo de referencia.

4.24.3 12. Cargas distribuidas

- Body load con la cuadratura del elemento.
- Edge traction uniforme: reparto 1/6, 4/6, 1/6 a (vértice, medio, vértice).

4.25 Contrato de implementación

```
name: Tri6
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 6
  strain_dim: 3
  n_integration_points: 3

acceptance:
  verification:
    - name: patch_cuadratico
      setup: "u = (x2, 0) impuesto en los 6 nodos"
      expect: "_xx = 2x exacto en cada Gauss"
      tol_rel: 1.0e-10
    - name: cooks_membrane_4x4
      setup: "Trapezoide de Cook triangulado 4x4 (32 Tri6, diagonal →c1c3 con center node compartido)"
      expect: "u_y(48, 52) 23.91 (Bathe; Hughes)"
```

```
tol_abs: 1.5
- name: cooks_membrane_refinamiento
  setup: "Mallas 2x2 → 4x4 → 6x6 con la misma triangulación"
  expect: "Error |u_y - 23.91| decreciente monótonamente"
  tol_rel: 0.0
```

4.26 Implementación

- **Archivo:** solidum/elements/solid_2d/tri6.py · clase `Tri6` (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con `Quad8` y `Quad9`). Declara `_MASS_QUADRATURE = "tri_6"` (Dunavant 6 puntos, orden 4) porque la cuadratura del elemento (`tri_3`, orden 2) subintegra el producto cuadrático×cuadrático de la masa consistente.
- **Tests:** tests/test_higher_order_solid_2d.py (patch cuadrático), tests/test_cooks_membrane.py (benchmark Bathe/Hughes con malla triangulada 4×4 + refinamiento monótono, añadido Fase B 2026-05-19).

Capítulo 5

Modelos Constitutivos (Materiales)

5.1 CableMaterial1D

*Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

5.2 Especificación física

5.2.1 0. Descripción general

Material 1D memoryless que modela la respuesta axial de un cable: **elástico lineal en tensión y nulo en compresión**. No tiene variables internas ni historia — la respuesta depende exclusivamente de la deformación instantánea. Captura la propiedad esencial de un cable: que solo transmite esfuerzo cuando está tensado.

5.2.2 1. Ecuación constitutiva

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E \varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Rampa lineal de pendiente E en tensión ($\varepsilon > 0$), cero en compresión o estado sin deformación ($\varepsilon \leq 0$). $\langle \cdot \rangle^+$ denota la parte positiva (operador de MacAuley).

5.2.3 2. Módulo tangente

$$E_t(\varepsilon) = \frac{d\sigma}{d\varepsilon} = \begin{cases} E & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Discontinuo en $\varepsilon = 0$. Por convención el valor en el punto de corte se asigna al régimen compresivo ($E_t = 0$), lo que evita generar rigidez espuria cuando el cable está exactamente sin tensión.

5.2.4 3. Variables internas

No hay. La respuesta es puramente función de ε actual; el estado del material es el vacío.

5.2.5 4. Interpretación física

- $\varepsilon > 0$: cable tensado — se comporta como barra elástica.
- $\varepsilon < 0$: cable destensado — no transmite nada, la carga pasa por otros caminos del modelo.
- $\varepsilon = 0$: frontera neutra — no hay tensión, no hay rigidez.

La no linealidad es **constitutiva** (en la respuesta del material), no geométrica. Un elemento con este material puede tener cualquier cinemática (lineal o corotacional); la unilateralidad solo vive aquí.

5.3 Contrato de implementación

```

name: CableMaterial1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null      # memoryless, no exporta variable principal

parameters:
  - { name: E, type: float, required: true, desc: "Módulo de Young (en tensión)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_tangent: float, state_vars)"
  strain_kind: axial escalar

conventions:
  sign: " > 0      > 0 (elongación);      0      = 0"
  state_passthrough: "state_vars se devuelve intacto (el material no almacena historia)"

validity:
  - "E > 0"
  - "|| 1e-2 en el régimen tensado (elasticidad lineal)"
  - "IS_UNILATERAL = True solo admitido en elementos con ACCEPTS_UNILATERAL = True (Cable2DCorot, Cable3DCorot). La base Element rechaza la combinación con Truss* en construcción."

out_of_scope:
  - plasticidad, fluencia, fatiga
  - viscoelasticidad / dependencia temporal
  - pretensión inicial (el usuario la modela con una longitud de referencia ajustada)
  - regularización numérica en compresión (no hay rigidez residual en 0)
  - anisotropía direccional; la respuesta es puramente axial escalar

numerical_caveats:
  - "En transiciones tensado destensado (cruza 0), E_tangent salta de E a 0 y Newton-Raphson puede oscilar. Mitigación: usar arc-length o pasos de carga finos si el problema tiene destensiones reales."
  - "Un cable completamente destensado tiene K_M = 0 y K_G = 0 matriz tangente singular en los DOFs del elemento. El usuario debe garantizar que otros elementos del modelo proporcionen rigidez suficiente, o pretensar el cable antes de aplicar cargas que puedan destensarlo."

```

```

acceptance:
- name: respuesta en tensión pura
  setup: " = 1e-4"
  expect: " = E · 1e-4, E_tangent = E"
  tol_rel: 1.0e-12

- name: respuesta en compresión pura
  setup: " = -1e-4"
  expect: " = 0, E_tangent = 0"
  tol_abs: 1.0e-15

- name: punto de corte
  setup: " = 0"
  expect: " = 0, E_tangent = 0 (régimen compresivo por convención)"
  tol_abs: 1.0e-15

- name: state_vars se devuelve intacto
  setup: "state_vars = {'dummy': 42}, arbitrario"
  expect: "el tercer retorno es idéntico al state_vars de entrada"

references:
- "Irvine H.M., Cable Structures, MIT Press, §1.2 (modelo elástico del cable)"
- "MacAuley M.A., on the deflection of beams, 1919 (operador parte positiva)"

```

5.4 Implementación

- **Archivo:** `solidum/materials/cable_1d.py` — archivo propio, sin dependencia de otros módulos de material salvo la clase base `Material`.
- **Clase:** `CableMaterial1D` — hereda directamente de `Material` (no de `Elastic1D` ni de ningún otro material), registrada vía `@MaterialRegistry.register`.
- **Tests:** `tests/test_cable_material.py · TestCableMaterial1D` — seis tests:
 - `test_acceptance_tension_pura` (criterio 1).
 - `test_acceptance_compresion_pura` (criterio 2).
 - `test_acceptance_punto_de_corte` (criterio 3).
 - `test_acceptance_state_vars_passthrough` (criterio 4).
 - `test_registro_en_registry` — verifica autodiscover.
 - `test_validacion_E_positivo` — rechaza $E \leq 0$.
- **Notas de traducción:**
 - La ramificación en `compute_state` es un único `if strain > 0.0`. El `>` estricto implementa la convención del punto de corte (el cero se asigna al régimen compresivo). El `else` cubre tanto $\varepsilon < 0$ como $\varepsilon = 0$ con las mismas salidas.
 - `state_vars` se devuelve como llega (identidad, no copia). El test `test_acceptance_state_vars_passthrough` usa `assertIs` para confirmar identidad.

- Validación de $E > 0$ al construir, con mensaje claro. Atrapa typos del input YAML temprano.
- El material acepta ****kwargs** en `compute_state` por compatibilidad con elementos que pudieran pasar argumentos adicionales (convención del proyecto).

5.5 Diálogo

- **2026-04-21** · Material creado como pieza independiente para la futura implementación de elementos de cable (`Cable2DCorot`, `Cable3DCorot`). Deliberadamente **no hereda** de `Elastic1D` aunque su rama en tensión sea idéntica: mantener el material autocontenido simplifica su lectura y evita acoplamientos futuros si la rama elástica cambia en otro archivo.
- **2026-04-21** · Discontinuidad en $\varepsilon = 0$: documentada explícitamente en `out_of_scope: regularización numérica`. Si aparece un caso de uso con destensiones frecuentes donde Newton-Raphson oscile, se añadirá una variante `CableMaterial1DRegularized` con $E_c \ll E$ en compresión — **no** se modificará este material (contrato limpio).
- **2026-05-12** · Restricción de emparejamiento elevada a contrato: `IS_UNILATERAL = True` en el material; `ACCEPTS_UNILATERAL = True` en `Cable2DCorot/Cable3DCorot`. La base `Element._validate_material` rechaza la combinación con `Truss*` (cuyo tangente colapsa la rigidez global sin `K_G` que lo compense). Antes el contrato lo permitía y solo se desaconsejaba en docstring; ahora falla limpio al construir.

5.6 VonMises2D

Orden de trabajo. El usuario escribe *especificación física, formulación numérica y contrato*; la IA rellena *implementación* y responde en *diálogo*.

>Spec *retroactiva con ampliación*: la hipótesis *plane_strain* está implementada y testada desde sesiones anteriores; esta spec la documenta y añade *plane_stress* como hipótesis a implementar en esta sesión.

5.7 Especificación física

5.7.1 0. Descripción general

Modelo elastoplástico independiente de la velocidad (rate-independent”) basado en el criterio **J2 / Von Mises** con regla de flujo **asociada** y **endurecimiento isótropo lineal**. Captura plasticidad de metales dúctiles en régimen de pequeñas deformaciones, sin efectos cinemáticos (Bauschinger), térmicos ni viscosos.

Soporta dos hipótesis cinemáticas 2D mutuamente excluyentes seleccionadas en construcción:

- **plane_strain** — $\varepsilon_{zz} = \varepsilon_{xz} = \varepsilon_{yz} = 0$. Cuerpos prismáticos largos cargados transversalmente (presa, túnel, cilindro largo).
- **plane_stress** — $\sigma_{zz} = \sigma_{xz} = \sigma_{yz} = 0$. Láminas o membranas delgadas en su plano (chapa traccionada, placa fina).

5.7.2 1. Descomposición aditiva

En pequeñas deformaciones, el tensor de deformación se descompone aditivamente:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$$

La parte elástica $\boldsymbol{\varepsilon}^e$ gobierna el esfuerzo vía la ley elástica isótropa; la parte plástica $\boldsymbol{\varepsilon}^p$ es **histórica** y evoluciona según la regla de flujo.

5.7.3 2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e : \boldsymbol{\varepsilon}^e = \mathbf{C}_e : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

con $\mathbf{C}_e$ tensor de rigidez elástica isótropa parametrizado por (E, ν) o equivalentemente (K, G) :

$$K = \frac{E}{3(1-2\nu)}, \quad G = \frac{E}{2(1+\nu)}$$

5.7.4 3. Superficie de fluencia (J2)

$$f(\boldsymbol{\sigma}, \alpha) = \|\mathbf{s}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha) \leq 0$$

donde $\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3} \text{tr}(\boldsymbol{\sigma}) \mathbf{I}$ es el tensor desviador, $\|\mathbf{s}\| = \sqrt{\mathbf{s} : \mathbf{s}}$, $\sigma_y > 0$ es la fluencia inicial y $H \geq 0$ el módulo de endurecimiento isótropo lineal. El factor $\sqrt{2/3}$ alinea σ_y con la fluencia uniaxial estándar.

5.7.5 4. Regla de flujo (asociada)

$$\dot{\epsilon}^p = \dot{\gamma} \frac{\partial f}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \mathbf{N}, \quad \mathbf{N} = \frac{\mathbf{s}}{\|\mathbf{s}\|}$$

El flujo es puramente desviador ($\text{tr}(\mathbf{N}) = 0$) — **plasticidad incompresible**, propiedad central de J2.

5.7.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \sqrt{\frac{2}{3}} \dot{\gamma}$$

α es la **deformación plástica acumulada equivalente** (escalar, monótona no decreciente). La superficie de fluencia se expande uniformemente con α manteniendo su centro en el origen del espacio de tensiones desviadoras.

5.7.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

Garantizan que $\dot{\gamma} > 0$ solo si el estado intenta salir de la superficie, y que el estado real siempre cumple $f \leq 0$.

5.7.8 7. Variables internas

Símbolo	Tipo	Significado
ϵ^p	tensor (4 componentes Voigt extendido en plane strain, ver §8)	deformación plástica
α	escalar	deformación plástica acumulada equivalente

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`); `eps_p` es interna al algoritmo de return mapping.

5.7.9 8. Notación Voigt y manejo de ϵ_{zz}^p

Convención del proyecto para 2D: $\boldsymbol{\epsilon} = [\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\epsilon_{xy}$. Pero la **descomposición desviadora** requiere todas las componentes 3D, así que internamente el material maneja $\boldsymbol{\epsilon}^p = [\epsilon_{xx}^p, \epsilon_{yy}^p, \epsilon_{zz}^p, \epsilon_{xy}^p]$ (4 componentes, ϵ_{xy}^p tensorial sin factor 2).

- **Plane strain:** $\epsilon_{zz} = 0$ se impone *en deformación total*. La componente ϵ_{zz}^p es no nula y evoluciona libremente (la incompresibilidad plástica acopla $\epsilon_{zz}^p = -(\epsilon_{xx}^p + \epsilon_{yy}^p)$). Por simetría, σ_{zz} resulta no nulo pero no se expone en el API público 2D — vive solo en el cómputo interno.
- **Plane stress:** $\sigma_{zz} = 0$ se impone *en esfuerzo*. La componente ϵ_{zz} es no nula (extensión transversal de Poisson + Poisson plástico), y aparece como variable adicional en el problema local de retorno (ver §10).

5.8 Formulación numérica

5.8.1 9. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\varepsilon_n^p, \alpha_n\}$ al paso n y la deformación total prescrita ε_{n+1} , resolver para $\{\sigma_{n+1}, \varepsilon_{n+1}^p, \alpha_{n+1}\}$ con discretización implícita:

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta\gamma \mathbf{N}_{n+1}, \quad \alpha_{n+1} = \alpha_n + \sqrt{\frac{2}{3}} \Delta\gamma$$

más $f(\sigma_{n+1}, \alpha_{n+1}) \leq 0$ con la condición de complementariedad.

5.8.2 10. Return mapping plane strain — algoritmo radial cerrado (existente)

Descomposición volumétrica-desviadora del estado de prueba:

$$\varepsilon_{n+1}^{\text{trial}} = \varepsilon_{n+1} - \varepsilon_n^p, \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}^{\text{trial}}, \quad p^{\text{trial}} = K \operatorname{tr}(\varepsilon_{n+1})$$

donde $\mathbf{e}^{\text{trial}}$ es la parte desviadora de $\varepsilon_{n+1}^{\text{trial}}$. Función de fluencia trial:

$$f^{\text{trial}} = \|\mathbf{s}^{\text{trial}}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol} \rightarrow$ paso elástico, $\sigma_{n+1} = \mathbf{C}_e : \varepsilon_{n+1}$, estado intacto.

Si no, el flujo es radial (la dirección no cambia entre trial y final por isotropía + asociado en J2):

$$\Delta\gamma = \frac{f^{\text{trial}}}{2G + \frac{2}{3}H} \quad (\text{forma cerrada})$$

$$\mathbf{N} = \frac{\mathbf{s}^{\text{trial}}}{\|\mathbf{s}^{\text{trial}}\|}, \quad \mathbf{s}_{n+1} = \mathbf{s}^{\text{trial}} - 2G \Delta\gamma \mathbf{N}, \quad \sigma_{n+1} = \mathbf{s}_{n+1} + p^{\text{trial}} \mathbf{I}$$

Tangente consistente algorítmica (derivada del algoritmo, no de la ley continua):

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G(1 - \beta) \mathbf{I}_{\text{dev}} - 2G \bar{\gamma} \mathbf{N} \otimes \mathbf{N}$$

$$\text{con } \beta = \frac{2G \Delta\gamma}{\|\mathbf{s}^{\text{trial}}\|}, \quad \bar{\gamma} = \frac{1}{1 + H/(3G)} - \beta.$$

5.8.3 11. Return mapping plane stress — algoritmo proyectado (a implementar)

Referencia: Simó & Hughes 1998, *Computational Inelasticity*, §3.4.1 ("Plane stress projected algorithm").

La restricción $\sigma_{zz} = 0$ se impone **eliminando** ε_{zz} del problema mediante un operador de proyección, no como ecuación extra. Resultado: el problema local sigue siendo escalar en $\Delta\gamma$, sin nested Newton.

Matriz elástica plane stress (3×3 , base Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$):

$$\mathbf{C}_e^{ps} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1 - \nu)/2 \end{bmatrix}$$

Predictor elástico en el subespacio plane stress:

$$\sigma^{\text{trial}} = \mathbf{C}_e^{ps} (\varepsilon_{n+1} - \mathbf{P}_\varepsilon \varepsilon_n^p)$$

con $\mathbf{P}_\varepsilon$ extrayendo las componentes plane $[\varepsilon_{xx}^p, \varepsilon_{yy}^p, 2\varepsilon_{xy}^p]$ del tensor 4D.

Operador $\mathbf{P}$ — norma desviadora bajo $\sigma_{zz} = 0$ (Simó-Hughes Box 3.1):

$$\mathbf{P} = \frac{1}{3} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 6 \end{bmatrix}$$

Esta matriz codifica el producto escalar desviador en el subespacio plane stress; reemplaza $\|\mathbf{s}\|^2$ por $\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}$ con todas las componentes 3D ya eliminadas analíticamente.

Función de fluencia plane stress proyectada:

$$\bar{f}(\boldsymbol{\sigma}, \alpha) = \frac{1}{2} \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma} - \frac{1}{3} (\sigma_y + H \alpha)^2 \leq 0$$

Esquema de retorno:

$$\boldsymbol{\sigma}_{n+1} = [\mathbf{C}_e^{ps}]^{-1} (\boldsymbol{\sigma}^{\text{trial}}) - \Delta\gamma \mathbf{P} \boldsymbol{\sigma}_{n+1}$$

reordenando:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{A}(\Delta\gamma)^{-1} \boldsymbol{\sigma}^{\text{trial}}, \quad \mathbf{A}(\Delta\gamma) = \mathbf{I} + \Delta\gamma \mathbf{C}_e^{ps} \mathbf{P}$$

La matriz $\mathbf{A}$ es 3×3 , diagonalizable en los autovectores ortonormales de $\mathbf{C}_e^{ps} \mathbf{P}$. Los autovalores son cerrados:

$$\mu_1 = \frac{E}{3(1-\nu)}, \quad \mu_2 = 2G, \quad \mu_3 = 2G$$

con autovectores $\mathbf{v}_1 = \frac{1}{\sqrt{2}}[1, 1, 0]$ (hidrostático plano), $\mathbf{v}_2 = \frac{1}{\sqrt{2}}[1, -1, 0]$ (desviador plano), $\mathbf{v}_3 = [0, 0, 1]$ (cortante). Diagonalizan simultáneamente $\mathbf{C}_e^{ps}$ y $\mathbf{P}$, con $\mathbf{v}_i^\top \mathbf{P} \mathbf{v}_i \in \{1/3, 1, 2\}$.

Actualización de la deformación plástica acumulada equivalente. La regla $\dot{\alpha} = \sqrt{2/3} \dot{\gamma}$ usada en plane strain **no** es válida aquí: en plane stress projected $\dot{\gamma}$ tiene unidades [1/esfuerzo] (porque $\dot{\varepsilon}^p = \dot{\gamma} \mathbf{P} \boldsymbol{\sigma}$ con $\mathbf{P} \boldsymbol{\sigma}$ en unidades de esfuerzo). La fórmula físicamente correcta y dimensionalmente consistente, obtenida de $\dot{\alpha} = \sqrt{2/3} \|\dot{\varepsilon}^p\|_{\text{Frob}}$:

$$\alpha_{n+1} = \alpha_n + \Delta\gamma \sqrt{\frac{2}{3} \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \boldsymbol{\sigma}_{n+1}}$$

Newton local sobre $\Delta\gamma$. En base autovectores $\sigma_i(\Delta\gamma) = a_i/(1 + \Delta\gamma \mu_i)$, así $\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}$ se expresa cerrado:

$$\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}(\Delta\gamma) = \frac{a_1^2}{3(1 + \Delta\gamma \mu_1)^2} + \frac{a_2^2}{(1 + \Delta\gamma \mu_2)^2} + \frac{2 a_3^2}{(1 + \Delta\gamma \mu_3)^2}$$

donde a_i son las proyecciones de $\boldsymbol{\sigma}^{\text{trial}}$ en los autovectores. La ecuación residual:

$$\bar{f}(\Delta\gamma) = \frac{1}{2} \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}(\Delta\gamma) - \frac{1}{3} (\sigma_y + H \alpha(\Delta\gamma))^2 = 0$$

con $\alpha(\Delta\gamma) = \alpha_n + \Delta\gamma \sqrt{(2/3) \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}}$ es monótona decreciente; tangente cerrada con regla de la cadena. **Newton local converge** típicamente en 3-6 iteraciones desde $\Delta\gamma = 0$. Coste: $O(1)$ por punto de Gauss, comparable a plane strain.

Tangente consistente plane stress. Sea $\mathbf{D} = \mathbf{A}^{-1} \mathbf{C}_e^{ps}$, $\mathbf{q} = \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \mathbf{D} \mathbf{P} \boldsymbol{\sigma}_{n+1}$, $w = \sqrt{(2/3) \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \boldsymbol{\sigma}_{n+1}}$, $m = (2/3) R H$, $n = 2\Delta\gamma/(3w)$. De la condición de consistencia ($\dot{f} = 0$) con la regla correcta de α :

$$\mathbf{C}_{ps}^{\text{alg}} = \mathbf{D} - \frac{(\mathbf{D} \mathbf{P} \boldsymbol{\sigma}_{n+1}) (\boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \mathbf{D})}{q + m w / (1 - m n)}$$

Si $H = 0$ (plasticidad perfecta) el término m se anula y el denominador se reduce a q . Forma cerrada — sin diferenciación numérica.

Predictor elástico con $\varepsilon^p \neq 0$. Tanto en plane strain como en plane stress, el predictor debe construirse como $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_n^p)$ (o equivalente desviador + presión), **no** como $\mathbf{C}_e \boldsymbol{\varepsilon}$, que ignoraría la deformación plástica acumulada. Importante en descargas elásticas desde estado plástico y al reevaluar el material con `compute_gauss_state(U_final)` tras un análisis converged.

5.8.4 12. Tolerancia de fluencia

Vía ADR 0006 (`Material.admissibility_tol`), patrón `atol + rtol·escala`:

- **Plane strain:** `admissibility_scale =  $\sqrt{(2/3)(\sigma_y + H \cdot \alpha)}$` (norma desviadora característica en la frontera).
- **Plane stress:** `admissibility_scale =  $(\sigma_y + H \cdot \alpha)^2/3$` (escala de $\bar{f}$ proyectada; los dos términos son del mismo orden).

5.9 Contrato de implementación

```
name: VonMises2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha    # deformación plástica acumulada equivalente

parameters:
  - { name: E,           type: float, required: true,  desc: "Módulo de Young" }
  - { name: nu,          type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: sigma_y,     type: float, required: true,  desc: "Tensión de fluencia inicial" }
  - { name: H,           type: float, required: false, default: 0.0, desc: "Módulo de endurecimiento isótropo lineal (0). H=0 plasticidad perfecta" }
  - { name: hypothesis,  type: str,   required: false, default: "plane_strain", desc: "plane_strain | plane_stress" }
  - { name: density,     type: float, required: false, default: null, desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: " (: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_alg: ndarray(3,3), state_vars )"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  eps_p: "ndarray(4,) - [^p_xx, ^p_yy, ^p_zz, ^p_xy] (xy tensorial, sin factor 2)"
  alpha: "float 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "> 0 tracción (Reglas §5)"
```



```

voigt: "_xy = 2·_xy en deformación total; ^p_xy almacenada en componente tensorial (
    sin factor 2)"
hypothesis_dispatch: "selección por kwarg al construir; mutuamente excluyentes"

validity:
- "E > 0, _y > 0, H = 0"
- "    (-1, 0.5) en plane_strain;    (-1, 0.5) en plane_stress (=0.5 no admitido por
    singularidad)"
- "|| 1e-2 (régimen de pequeñas deformaciones)"
- "hypothesis {'plane_strain', 'plane_stress'}"

out_of_scope:
- "endurecimiento cinemático (back-stress); usar variante futura VonMises2DKinematic"
- "endurecimiento isótropo no lineal (Voce, exponencial)"
- "endurecimiento por ablandamiento (H<0); requiere regularización (longitud caracter
    ística, no-local)"
- "regla de flujo no asociada"
- "acoplamiento con daño; usar modelo plástico-dañado dedicado"
- "viscoplasticidad y dependencia de la velocidad de deformación"
- "grandes deformaciones (descomposición multiplicativa F = F~e·F~p)"
- "anisotropía (Hill, Barlat) - el modelo es estrictamente J2 isótropo"

numerical_caveats:
- "Plane strain: incompresibilidad plástica genera locking volumétrico en elementos
    de bajo orden (Quad4, Tri3) cuando →0.5 o el campo plástico domina. Mitigaciones
    futuras: B-bar, F-bar, elementos mixtos u-p. No incluidas en esta spec."
- "Plane stress: el Newton local sobre Δ asume H = 0; con H = 0 (perfectamente plá
    stico) la ecuación residual sigue siendo monótona pero su pendiente al final del
    paso plástico tiende a 0 - convergencia más lenta. Mitigación: bisección de
    respaldo si Newton local diverge."
- "Para  cercano a 0.5 en plane strain, K →∞ y el predictor elástico pierde precisió
    n numérica. Recomendado 0.49 en metales (típico =0.3)."
```

acceptance:

```

# Cubierto por tests/test_materials_unit.py · TestVonMises2D y
  TestVonMises2DPlaneStress
unit_plane_strain:
- name: paso_elastico_bajo_fluencia
  expect: "[1e-5,0,0]: =C_e· exacto, alpha=0, tangente=C_e"
  tol_rel: 1.0e-10
- name: fluencia_traccion_uniaxial_confinada
  expect: "fluencia activa con =[0.01,-0.01,0]: alpha>0 tras un solo paso"
- name: monotonicidad_alpha
  expect: " no decrece bajo carga creciente con eps_yy=-; _final > 0"
- name: no_further_yield_on_frontier
  expect: "repetir en frontera de fluencia no incrementa (return mapping
    consistente)"
  tol_rel: 1.0e-10
- name: hypothesis_validation
  expect: "constructor rechaza hypothesis distinto de 'plane_strain'/'plane_stress'
    con ValueError"

unit_plane_stress:
- name: paso_elastico_bajo_fluencia
  expect: " pequeño: =C_e^ps·, tangente=C_e^ps, alpha=0, eps_p=0"
  tol_rel: 1.0e-10
- name: degeneracion_a_1d_traccion_pura_elastico
  expect: "=(eps,-·eps,0): _xx=E·eps, _yy0 (regimen elástico, no E/(1-²))"
  tol_rel: 1.0e-4

```

```

- name: yield_uniaxial_at_sigma_y
  expect: "primer yield en _xx=_y bajo tracción uniaxial pura (no E/(1-2))."
  tol_rel: 1.0e-2
- name: traccion_biaxial_isotropa
  expect: "fluencia biaxial isotropa exactamente en _xx=_yy=_y (||s||=·√(2/3)
    iguala √(2/3) _y); _yield=_y (1-)/E"
  tol_rel: 1.0e-2
- name: incompresibilidad_plastica
  expect: "tr (^p) = ^p_xx + ^p_yy + ^p_zz = 0 tras cualquier paso plástico"
  tol_abs: 1.0e-12
- name: alpha_monotonic
  expect: " no decrece bajo carga monotónica creciente"
- name: descarga_recupera_C_e_ps
  expect: "tras estado plástico, paso siguiente con menor produce tangente=C_e^ps
    y intacto"
  tol_rel: 1.0e-10
- name: tangente_simetrica
  expect: "C_alg simétrica (J2 asociado IS_SYMMETRIC=True)"
  tol_abs: 1.0e-8
- name: unit_invariance_MPa_vs_Pa
  expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) y ^p idénticos"
  tol_rel: 1.0e-12

# Cubierto por tests/test_solid_2d_plasticity.py
integration:
- name: pipeline_plane_strain_elastico
  setup: "Quad4 unitario, confinamiento uy=0; tracción horizontal por debajo del
    yield"
  expect: "u_x = _xx_target exacto; =0"
  tol_rel: 1.0e-8

- name: pipeline_plane_strain_plastico_vs_material_directo
  setup: "mismo setup, carga 1.5×_xx_yield → régimen plástico; integrar 10 pasos"
  expect: "_xx FEM (gauss avg) coincide con material standalone en _xx_final bajo
    trayectoria monótona; coincide"
  tol_rel: 1.0e-3

- name: pipeline_plane_stress_elastico_uniaxial
  setup: "Quad4 unitario, lado izq apoyado en ux; nodo (0,0) y (1,0) en uy; nodo
    (1,1) libre en y. Tracción horizontal por debajo del yield"
  expect: "u_x = _xx_target; contracción transversal u_y(0,1) = -·_xx (Poisson);
    =0"
  tol_rel: 1.0e-6

- name: pipeline_plane_stress_plastico_vs_1d
  setup: "mismo setup, carga 1.2_y para entrar plástico; comparar contra
    Elastoplastic1D bajo mismo path _xx"
  expect: "_xx FEM = _1D, _yy FEM 0 (uniaxial puro emergente); _FEM = _1D"
  tol_rel: 1.0e-3

- name: pipeline_converges_few_iter_plane_strain
  setup: "carga plástica significativa, 4 incrementos, max_iter=8"
  expect: "solver converge en todos los pasos sin agotar max_iter; alpha final > 0"

- name: pipeline_converges_few_iter_plane_stress
  setup: "tracción uniaxial pura plane stress 1.5_y, 4 incrementos, max_iter=8"
  expect: "solver converge en todos los pasos; alpha final > 0"

references:

```

- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §3.3 (plane strain return mapping), §3.4 (plane stress projected algorithm), Box 3.1 (operador P)."
- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures, Vol. 1. Wiley. Cap. 6 (J2 plasticity)."
- "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. Cap. 5 (cilindro a presión interna, sol. cerrada Hill)."
- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for Plasticity. Wiley. §9.4 (plane stress projection alternativa)."

5.10 Implementación

- **Archivo:** solidum/materials/von_mises_2d.py.
- **Clase:** VonMises2D, registrada vía `@MaterialRegistry.register`. Despacho por `hypothesis` en construcción (sin coste runtime).
- **Kernels Numba:**
- `_compute_j2_plane_strain`: descomposición volumétrica-desviadora 3D (ε_{zz}^p libre), return mapping radial cerrado, tangente algorítmica consistente cerrada. Predictor elástico construido como $\mathbf{s}_{\text{trial}} + p\mathbf{I}$ con $\mathbf{s}_{\text{trial}} = 2G(\mathbf{e}^{\text{dev}} - \varepsilon_n^p \mathbf{e}_n)$ — respeta ε_n^p en descargas/reevaluaciones.
- `_compute_j2_plane_stress`: predictor $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e^{ps}(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_n^p)$, función de fluencia $\bar{f}$ proyectada, Newton local sobre $\Delta\gamma$ con tangente cerrada y máximo `_PLANE_STRESS_MAX_LOCAL_ITER` = 25 iteraciones. Construcción de $\mathbf{A}^{-1}$ y $\boldsymbol{\sigma}_{\text{new}}$ vía base de autovectores conocida ($\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$); incompresibilidad plástica cierra $\varepsilon_{zz}^p = -(\varepsilon_{xx}^p + \varepsilon_{yy}^p)$.
- **State schema:** `eps_p` ndarray(4), [xx, yy, zz, xy_tensorial], alpha escalar. Las cuatro componentes de `eps_p` son obligatorias en ambas hipótesis para preservar invariantes (incompresibilidad, normas tensoriales).
- **admissibility_scale:**
- `plane_strain`: $\sqrt{(2/3)} \cdot (\sigma_y + H \cdot \alpha)$
- `plane_stress`: $(\sigma_y + H \cdot \alpha)^2 / 3$
- **Tests:**
- `tests/test_materials_unit.py` · `TestVonMises2D` (5 tests plane strain pre-existentes, no regresión) + `TestVonMises2DPlaneStress` (9 tests plane stress).
- `tests/test_solid_2d_plasticity.py` · 6 tests de integración del pipeline Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis.
- `tests/test_density_self_weight.py` · `test_vonmises2d_density` — densidad ADR 0008.
- **Notas de traducción:**
- El kernel plane stress trabaja con $\boldsymbol{\sigma}$ en Voigt mixto: las componentes $\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]$ son las "engineering" mientras que $\boldsymbol{\varepsilon}$ usa $\gamma_{xy} = 2\varepsilon_{xy}$. El operador $\mathbf{P}$ en esta convención tiene la forma con 6 en la entrada (3,3); $\mathbf{P}\boldsymbol{\sigma}$ devuelve la componente cortante como $2\sigma_{xy}$ (convención engineering), que se convierte a tensorial dividiendo por 2 antes de acumular en ε_{xy}^p .

- La actualización $\alpha_{n+1} = \alpha_n + \Delta\gamma\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ se evalúa **con $\boldsymbol{\sigma}$ final** del paso plástico (tras converger Newton local). Durante las iteraciones se usa la misma fórmula con el $\boldsymbol{\sigma}$ corriente.
- Validación de parámetros en constructor: $E>0$, $\sigma_y>0$, $H\geq 0$, $\nu\in(-1,0.5)$, `hypothesis` $\in$ {`plane_strain`, `plane_stress`}. Mensajes en español describiendo la violación.

5.10.1 Bug fixes detectados al implementar

- **Predictor elástico plane strain ignoraba $\boldsymbol{\varepsilon}^p$.** Versión previa devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$ cuando $f^{\text{trial}} \leq 0$ — correcto solo si $\boldsymbol{\varepsilon}^p = 0$. En descargas elásticas desde estado plástico, o al reevaluar el material vía `compute_gauss_state(U_final)` tras un análisis converged, devolvía $\boldsymbol{\sigma}$ inconsistente con el estado interno. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. Detectado por el test de integración `test_plastic_regime_matches_standalone_material`: $\boldsymbol{\sigma}$ FEM reportado coincidía con el valor de equilibrio (F/A) en Newton (donde sí entra a return mapping), pero `compute_gauss_state` post-converged daba el valor incorrecto del predictor elástico, divergiendo del material standalone.

5.11 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (J2 plane strain) y extendida con J2 plane stress. Decisión de algoritmo plane stress: **proyección de Simó-Hughes §3.4.1** en lugar de nested Newton sobre ε_{zz} . Justificación: forma cerrada del problema local (Newton local escalar sobre $\Delta\gamma$ con tangente cerrada, 3-6 iteraciones, $O(1)$ por Gauss); Numba-compatible (kernel plano); tangente consistente cerrada vía operador $\mathbf{P}$. Nested Newton implicaría 5-10× el coste por anidación de return mapping completo dentro de cada iteración externa sobre $\sigma_{zz} = 0$, justificable solo con endurecimiento no lineal complejo (Voce, plasticidad mixta) que no es el caso. Registrado aquí, no en ADR — afecta a un material, no a contratos arquitecturales transversales.
- **2026-05-14** · Corrección durante la implementación: la regla heurística $\alpha_{n+1} = \alpha_n + \sqrt{2/3}\Delta\gamma$ heredada de plane strain rompe la invariancia bajo cambio de unidades en plane stress, porque allí $\Delta\gamma$ tiene unidades [1/esfuerzo] (la regla de flujo $\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\mathbf{P}\boldsymbol{\sigma}$ tiene $\mathbf{P}\boldsymbol{\sigma}$ con dimensión de esfuerzo). La fórmula físicamente correcta es $\alpha_{n+1} = \alpha_n + \Delta\gamma\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ — adimensional. Detectado por el test unitario `test_unit_invariance_MPa_vs_Pa` que daba un factor $1e6$ entre los dos sistemas. Implica recalcular la tangente consistente con $\partial\alpha/\partial\Delta\gamma \neq \sqrt{2/3}$ (ver §11).
- **2026-05-14** · Corrección plane strain: el predictor elástico devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$, ignorando $\boldsymbol{\varepsilon}_n^p$. Bug latente preexistente sin tests que lo cubrieran (los unitarios solo probaban primer paso con $\boldsymbol{\varepsilon}^p = 0$). Detectado al validar el pipeline FEM: `compute_gauss_state(U_final)` reportaba $\boldsymbol{\sigma}$ del predictor elástico (incorrecto) en lugar del $\boldsymbol{\sigma}$ post return-mapping. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. La rama plane stress ya lo hacía bien desde el principio (predictor incluía la sustracción de $\boldsymbol{\varepsilon}^p$).
- **2026-05-14** · Validación numérica de plane strain queda implícita en los tests unitarios existentes; los tests de integración `tests/test_solid_2d_plasticity.py` abren por primera vez el pipeline completo Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis — hasta hoy no había cobertura de integración sólido 2D + material plástico.

- **2026-05-14** · No se introduce ADR. La spec no rompe contratos: extiende un material existente con una rama nueva del despacho por `hypothesis`. Sí se actualiza `docs/catalogo_materiales.md` al validar.

5.12 IsotropicDamage1D

Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.

>Spec **retroactiva con ampliación**: modelo implementado desde sesiones anteriores con tangente secante; esta spec lo documenta y añade la tangente algorítmica consistente, replicando la derivación de `[[IsotropicDamage2D]]` reducida a 1D escalar.

5.13 Especificación física

Versión 1D de `IsotropicDamage2D`. Modelo de daño escalar isótropo en pequeñas deformaciones para barras, cables y elementos axiales. Captura degradación progresiva de la rigidez axial con ablandamiento exponencial.

5.13.1 Ecuaciones

- **Constitutiva**: $\sigma = (1 - d) E \varepsilon$, con $\sigma^{\text{eff}} = E \varepsilon$.
- **Deformación equivalente**: $\varepsilon_{\text{eq}} = |\varepsilon|$.
- **Variable histórica**: $\kappa_{n+1} = \max(\kappa_n, |\varepsilon_{n+1}|)$, con κ_0 inicial.
- **Daño**: $d(\kappa) = \begin{cases} 0 & \kappa \leq \kappa_0 \\ 1 - (\kappa_0/\kappa) e^{-\alpha(\kappa-\kappa_0)} & \kappa > \kappa_0 \end{cases}$, saturado a `DAMAGE_MAX`.

Todas las propiedades físicas (irreversibilidad, asintótica $d \rightarrow 1$, papel de α) son las del modelo 2D restringidas a un escalar. No distingue tracción de compresión.

5.14 Formulación numérica

5.14.1 Tangente algorítmica consistente

$$\frac{\partial \sigma}{\partial \varepsilon} = (1 - d) E - E \varepsilon \frac{\partial d}{\partial \kappa} \frac{\partial \kappa}{\partial \varepsilon}$$

con:

- $\frac{\partial d}{\partial \kappa} = (1 - d) \left(\frac{1}{\kappa} + \alpha \right)$ (idéntico a 2D).
- $\frac{\partial \kappa}{\partial \varepsilon} = \text{sign}(\varepsilon)$ en **carga activa** ($|\varepsilon| > \kappa_n$); 0 en descarga.

Tangente final:

$$E^{\text{alg}} = \begin{cases} E & \kappa \leq \kappa_0 \text{ (sin daño)} \\ (1 - d) E & \text{descarga} \\ (1 - d) E [1 - (1/\kappa + \alpha) \kappa] = -(1 - d) E \alpha \kappa & \text{carga activa, } d < d_{\text{max}} \\ (1 - d_{\text{max}}) E & d = d_{\text{max}} \text{ (saturación)} \end{cases}$$

donde se ha usado $(1/\kappa + \alpha)\kappa = 1 + \alpha\kappa$ y $E \varepsilon \text{sign}(\varepsilon) = E|\varepsilon| = E\kappa$ en carga activa.

5.14.2 Signo de la tangente consistente

En carga activa **siempre es negativa**: $E^{\text{alg}} = -(1 - d) E \alpha \kappa < 0$ porque $1 - d > 0$, $E > 0$, $\alpha > 0$, $\kappa > 0$. Refleja físicamente el régimen de **softening** (ablandamiento): la curva σ - ε desciende tras el umbral.

Implicaciones numéricas:

- La rigidez tangente del elemento ($\mathbf{K}_e = \int B^\top E^{\text{alg}} B dV$) puede tener autovalores negativos. La matriz **sigue siendo simétrica** (escalar $\times$ forma cuadrática) — `IS_SYMMETRIC = True` se mantiene.
- La matriz global puede dejar de ser **positiva definida**. El despachador algebraico (ADR 0003) detecta el fallo de Cholesky y degrada a LU automáticamente — no requiere cambio declarativo.
- Control de carga puede no converger en problemas con suficiente daño para producir snap-back; en ese régimen se requiere control de longitud de arco (`ArcLengthSolver`).

5.14.3 Por qué no tests de integración en esta spec

A diferencia de `[[IsotropicDamage2D]]`, no se añade benchmark de integración `Truss2D + IsotropicDamage1D + NonlinearSolver`: con un solo elemento axial el comportamiento global está dominado por la inestabilidad de softening ($E_{\text{alg}} < 0$) y la única vía limpia de seguir la respuesta post-pico es arc-length. La validación de la tangente consistente se cubre con tests unitarios (incluyendo diferencia finita centrada como verificación de la derivada cerrada). Si en el futuro aparece un caso real de problema con daño 1D ramificado, se añadirá un benchmark dedicado con `ArcLengthSolver`.

5.15 Contrato de implementación

```
name: IsotropicDamage1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: damage
  is_symmetric: true # tangente escalar contribución elemento simétrica (puede
                    ser indefinida pero simétrica)

parameters:
- { name: E, type: float, required: true, desc: "Módulo de Young intacto" }
- { name: kappa_0, type: float, required: true, desc: "Umbral de deformación
  equivalente ||" }
- { name: alpha, type: float, required: true, desc: "Velocidad de degradación
  exponencial (>0)" }
- { name: density, type: float, required: false, default: null, desc: "Densidad (ADR
  0008)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_tan: float, state_vars)"
  strain_kind: axial escalar

state_schema:
```

```

kappa: "float    _0 - máxima || histórica"
damage: "float    [0, DAMAGE_MAX]"

conventions:
  sign: " > 0   tracción; el modelo no distingue tracción de compresión en la activaci
        ón del daño"
  damage_irreversibility: " monótono no decreciente; d() no decreciente"

validity:
  - "E > 0, _0 > 0,   > 0"
  - "|| 1e-2 (régimen de pequeñas deformaciones)"

out_of_scope:
  - "split tracción/compresión"
  - "fatiga / acumulación cíclica"
  - "regularización para mesh-dependency en problemas con varios elementos en serie"
  - "test de integración con NonlinearSolver - requiere arc-length para superar el snap
    -back, fuera de scope hasta caso real"

numerical_caveats:
  - "Tangente consistente NEGATIVA en carga activa (softening). La rigidez global puede
    no ser PD; el despachador algebraico (ADR 0003) degrada a LU automáticamente al
    detectar fallo de Cholesky."
  - "Control de carga con un elemento aislado tras alcanzar el pico se vuelve inestable
    . Usar arc-length para seguir la rama post-pico."
  - "Transición cargadescarga discontinua en la tangente (rama loading vs unloading):
    aceptable para Newton convergente, problemático cerca del umbral en problemas
    patológicos."

acceptance:
  unit_existing_no_regression:
    - name: secant_response_below_threshold
      expect: " pequeño,    _0  d=0,  =E·, tangente = E"
      tol_rel: 1.0e-12

    - name: damage_evolution_exponential
      expect: "ley exponencial reproducida con la fórmula"
      tol_rel: 1.0e-10

  unit_consistent_tangent:
    - name: tangent_equals_E_below_threshold
      setup: " < _0"
      expect: "E_tan = E (sin daño)"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_on_unloading
      setup: "cargar a  > _0; descargar a || < "
      expect: "E_tan = (1-d)·E exacto"
      tol_rel: 1.0e-12

    - name: tangent_negative_on_loading
      setup: "carga activa con  > _0, d  (0, DAMAGE_MAX)"
      expect: "E_tan = -(1-d)·E· · < 0"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_at_saturation
      setup: " enorme tal que d = DAMAGE_MAX"
      expect: "E_tan = (1-DAMAGE_MAX)·E (sin término consistente al saturar)"
      tol_rel: 1.0e-12

```



```

- name: tangent_matches_finite_difference
  setup: "carga activa; / por FD centrada (h=1e-7) vs fórmula cerrada"
  expect: "error relativo < 1e-5"
  tol_rel: 1.0e-5

- name: tangent_consistent_with_2D_when_uniaxial
  setup: "evaluar IsotropicDamage1D(E, _0, ) en =eps y comparar contra
         IsotropicDamage2D(E, =0, _0, ) plane stress en =[eps, 0, 0]; la primera
         componente de y la entrada [0,0] de C_alg deben coincidir"
  expect: "_1D = _xx_2D; E_tan_1D = C_alg_2D[0,0]"
  tol_rel: 1.0e-10

references:
- "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
  Cap. 7."
- "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
  Int. J. Solids Struct. 23, 821-840."
- "Spec hermana: [docs/specs/IsotropicDamage2D.md](IsotropicDamage2D.md) - modelo 2D,
  derivación detallada de la tangente consistente."

```

5.16 Implementación

- **Archivo:** solidum/materials/damage_1d.py.
- **Clase:** IsotropicDamage1D, registrada vía @MaterialRegistry.register. IS_SYMMETRIC queda True (default heredado de Material) — la contribución elemental es simétrica aunque pueda ser indefinida.
- **Algoritmo compute_state:**
 1. Recuperar κ_n y calcular $\varepsilon_{eq} = |\varepsilon|$.
 2. Si $\varepsilon_{eq} > \kappa_n$: carga activa, $\kappa_{n+1} = \varepsilon_{eq}$. Si no: descarga, $\kappa_{n+1} = \kappa_n$.
 3. Calcular d por la ley exponencial, saturar a DAMAGE_MAX.
 4. $\sigma = (1 - d) E \varepsilon$.
 5. Tangente: ramas (sin daño / descarga / saturación / carga activa) idénticas en estructura al modelo 2D, con la fórmula escalar resultante. En carga activa con daño efectivo no saturado, $E_{tan} = -(1-d) \cdot E \cdot \alpha \cdot \kappa$.
- **Tests:**
 - tests/test_materials_unit.py · TestIsotropicDamage1D (existente, no regresión) + nueva clase TestIsotropicDamage1DConsistentTangent.
 - Sin tests de integración (justificado arriba).

5.17 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Réplica directa de `[[IsotropicDamage2D]]` reducida a escalar. Sin ADR — extensión local al material, no afecta a contratos.
- **2026-05-14** · `IS_SYMMETRIC` permanece `True`: la rigidez axial es escalar y la contribución elemental $\mathbf{B}^T \mathbf{E}_{\text{tan}} \mathbf{B}$ es simétrica aunque pueda ser indefinida cuando $\mathbf{E}_{\text{tan}} < 0$. El sistema global puede dejar de ser PD; el fallback automático de Cholesky a LU del despachador algebraico (ADR 0003) lo gestiona sin cambios declarativos.
- **2026-05-14** · No se añade test de integración. Justificación: con tangente consistente < 0 en carga activa, un benchmark `Truss2D` bajo control de carga se vuelve inestable tras el pico; el seguimiento de la rama post-pico requiere arc-length que cae fuera del scope inmediato. Tests unitarios (incluyendo FD numérica y consistencia con la versión 2D reducida) cubren la corrección de la fórmula.

5.18 IsotropicDamage2D

Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.

>Spec **retroactiva con ampliación**: el modelo está implementado desde sesiones anteriores con **tangente secante**. Esta spec lo documenta y añade la **tangente algorítmica consistente** para recuperar convergencia cuadrática del Newton global.

5.19 Especificación física

5.19.1 0. Descripción general

Modelo de **daño isótropo escalar** en pequeñas deformaciones para sólidos 2D. Captura degradación progresiva e isótropa de la rigidez en respuesta a la deformación máxima histórica, con ablandamiento exponencial. Pensado para hormigón en tracción uniforme, cerámicos, materiales cuasi-frágiles donde la microfisuración distribuida puede modelarse mediante un escalar de daño en lugar de microestructura explícita.

5.19.2 1. Cinemática

Pequeñas deformaciones, descomposición elástica pura: la deformación plástica no existe en este modelo (sin plasticidad). El daño se manifiesta como degradación de la rigidez efectiva.

5.19.3 2. Esfuerzo nominal vs esfuerzo efectivo

$$\boldsymbol{\sigma} = (1 - d) \boldsymbol{\sigma}^{\text{eff}}, \quad \boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$$

con $d \in [0, d_{\text{max}}]$ la variable de daño isótropa y $\mathbf{C}_e$ el tensor constitutivo elástico isótropo (2D, plane stress o plane strain según **hypothesis** heredada del **Elastic2D** interno).

El esfuerzo efectivo $\boldsymbol{\sigma}^{\text{eff}}$ es el que el material intacto soportaría con la deformación corriente. El esfuerzo nominal $\boldsymbol{\sigma}$ es lo que efectivamente transmite considerando la degradación.

5.19.4 3. Deformación equivalente

Cantidad escalar que cuantifica la “intensidad” de la deformación. Convención del proyecto, norma simétrica simple en notación Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$:

$$\varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}) = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{M} \boldsymbol{\varepsilon}}$$

con $\mathbf{M} = \text{diag}(1, 1, 1/2)$. Equivalente a la norma de Frobenius del tensor deformación 2D plana ($\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}$ con $\varepsilon_{xy} = \gamma_{xy}/2$).

Limitación: no distingue tracción de compresión. Para hormigón a compresión se necesitaría un split tipo Mazars o de Vree; aquí se diferencia explícitamente como *out-of-scope*.

5.19.5 4. Variable histórica y condición de carga (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}_{n+1})), \quad \kappa_0 \text{ inicial}$$

Esto codifica:

- **Carga activa** ($\dot{\kappa} > 0$): κ crece con ε_{eq} ; el daño puede aumentar.
- **Descarga / recarga** ($\dot{\kappa} = 0$): κ queda congelado; el daño no cambia.
- **Irreversibilidad**: κ es monótono no decreciente.

5.19.6 5. Ley de evolución del daño

$$d(\kappa) = \begin{cases} 0 & \kappa \leq \kappa_0 \\ 1 - \frac{\kappa_0}{\kappa} e^{-\alpha(\kappa - \kappa_0)} & \kappa > \kappa_0 \end{cases}$$

con saturación numérica $d \leq d_{\max} = \text{DAMAGE_MAX}$ (`solidum.constants.DAMAGE_MAX`, evita rigidez nula).

Propiedades:

- $d(\kappa_0) = 0$ (continuidad).
- $d'(\kappa) > 0$ para $\kappa > \kappa_0$ (irreversibilidad).
- $\lim_{\kappa \rightarrow \infty} d = 1$ (asintótico).
- α controla la velocidad de degradación (ablandamiento más abrupto si α grande).

5.20 Formulación numérica

5.20.1 6. Algoritmo en un paso

Dado $\{\kappa_n\}$ y ε_{n+1} :

1. $\varepsilon_{eq} = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2}$.
2. Si $\varepsilon_{eq} > \kappa_n$: **carga activa**, $\kappa_{n+1} = \varepsilon_{eq}$. Si no: **descarga**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar ley de daño con κ_{n+1} ; saturar a $d_{\max}$.
4. $\sigma_{n+1} = (1 - d) \mathbf{C}_e \varepsilon_{n+1}$.
5. Tangente: **secante** en descarga / sin daño / saturación; **algorítmica consistente** en carga activa con daño no saturado (ver §7).

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y d es directa.

5.20.2 7. Tangente algorítmica consistente (ampliación)

$$\mathbf{C}^{\text{alg}} = \frac{\partial \sigma_{n+1}}{\partial \varepsilon_{n+1}}$$

Derivando $\sigma = (1 - d) \mathbf{C}_e \varepsilon$:

$$\mathbf{C}^{\text{alg}} = (1 - d) \mathbf{C}_e - \sigma^{\text{eff}} \otimes \frac{\partial d}{\partial \varepsilon}$$

con $\sigma^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$. La derivada del daño respecto a la deformación, por regla de la cadena:

$$\frac{\partial d}{\partial \boldsymbol{\varepsilon}} = \frac{\partial d}{\partial \kappa} \frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$$

donde:

- $\frac{\partial d}{\partial \kappa} = (1-d) \left(\frac{1}{\kappa} + \alpha \right)$ para la ley exponencial. (Demostración: derivando $d = 1 - (\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$ y reagrupando vía $1-d = (\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$.)
- $\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$ depende del régimen:
- **Carga activa** ($\varepsilon_{\text{eq}} > \kappa_n$, $\kappa = \varepsilon_{\text{eq}}$):

$$\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}} = \frac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \frac{1}{\varepsilon_{\text{eq}}} \begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \frac{1}{2}\gamma_{xy} \end{bmatrix}$$

- **Descarga** ($\varepsilon_{\text{eq}} \leq \kappa_n$): $\partial \kappa / \partial \boldsymbol{\varepsilon} = 0$.

Tangente final:

$$\mathbf{C}^{\text{alg}} = \begin{cases} \mathbf{C}_e & \kappa \leq \kappa_0 \text{ (sin daño)} \\ (1-d) \mathbf{C}_e & \text{descarga} \\ (1-d) \mathbf{C}_e - \frac{(1-d)(1/\kappa + \alpha)}{\varepsilon_{\text{eq}}} (\mathbf{C}_e \boldsymbol{\varepsilon})(\mathbf{M} \boldsymbol{\varepsilon})^\top & \text{carga activa, } d < d_{\text{max}} \\ (1-d_{\text{max}}) \mathbf{C}_e & d = d_{\text{max}} \text{ (saturación)} \end{cases}$$

con $\mathbf{M} = \text{diag}(1, 1, 1/2)$.

5.20.3 8. Pérdida de simetría

El producto externo $(\mathbf{C}_e \boldsymbol{\varepsilon})(\mathbf{M} \boldsymbol{\varepsilon})^\top$ es una matriz 3×3 que **no es simétrica en general**. $\mathbf{C}_e \boldsymbol{\varepsilon}$ y $\mathbf{M} \boldsymbol{\varepsilon}$ no son proporcionales salvo en estados de deformación muy particulares (e.g. uniaxial puro alineado con ejes).

Esto rompe la simetría heredada de la tangente secante $(1-d)\mathbf{C}_e$:

- **Tangente secante:** simétrica. `IS_SYMMETRIC = True` sería válido.
- **Tangente consistente:** no simétrica en general. `IS_SYMMETRIC = False` obligatorio.

Consecuencia algebraica (ADR 0003): el despachador elige LU en lugar de Cholesky/LDL[⊤] para sistemas globales que incluyan este material. El coste por paso aumenta 30-50 % respecto a Cholesky, pero la convergencia cuadrática del Newton global con tangente consistente recupera ese coste con creces (típicamente 2-3 iteraciones en lugar de 5-10 con tangente secante).

5.20.4 9. Decisión de régimen durante el Newton

Durante una iteración del Newton global, el material recibe un $\boldsymbol{\varepsilon}_{\text{trial}}$ y debe decidir carga/-descarga comparando con κ_n (el estado committed del paso anterior). La transición entre ramas (loading ↔ unloading) es **discontinua en la tangente**: al cruzar $\varepsilon_{\text{eq}} = \kappa_n$ el segundo término aparece/desaparece bruscamente. En la práctica esto no genera problemas porque el Newton converge antes de oscilar entre ramas; en problemas patológicos puede requerir continuation o arc-length.

5.21 Contrato de implementación

```

name: IsotropicDamage2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: damage
  is_symmetric: false      # tangente consistente no simétrica (ver §8)

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young intacto" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: kappa_0,    type: float, required: true,  desc: "Umbral de deformación
    equivalente; daño nulo mientras _0" }
  - { name: alpha,      type: float, required: true,  desc: "Velocidad de degradación
    exponencial (>0)" }
  - { name: hypothesis, type: str,   required: false, default: "plane_stress", desc: "
    plane_stress | plane_strain (delegado a Elastic2D interno)" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
    ADR 0008)" }

signature:
  compute_state: " (: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_tan: ndarray
    (3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  kappa: "float    _0 - máxima deformación equivalente histórica"
  damage: "float    [0, DAMAGE_MAX] - variable de daño escalar isótropa"

conventions:
  sign: " > 0  tracción; el modelo no distingue tracción de compresión en la activació
    n del daño"
  voigt: "_xy = 2·_xy en deformación; _eq usa norma M=diag(1,1,1/2)"
  damage_irreversibility: " monótono no decreciente vía max(_old, _eq); d() creciente
    "

validity:
  - "E > 0, _0 > 0,    > 0"
  - "    (-1, 0.5) (delegación a Elastic2D)"
  - "hypothesis  {'plane_stress', 'plane_strain'}"

out_of_scope:
  - "split tracción/compresión (Mazars, de Vree, Mises modificado); el modelo daña por
    igual en cualquier régimen"
  - "anisotropía del daño (tensor de daño D vs escalar)"
  - "regularización para evitar mesh-dependency en regime de ablandamiento (longitud
    característica, no-local, gradient)"
  - "acoplamiento con plasticidad"
  - "fatiga / acumulación cíclica del daño"
  - "tangente consistente para IsotropicDamage1D - fuera del alcance de esta spec;
    misma deuda gemela"

numerical_caveats:
  - "Tangente consistente NO simétrica → IS_SYMMETRIC=False obliga al despachador
    algebraico a usar LU (ADR 0003). Costo por paso ~30-50% mayor que Cholesky pero

```

```

    convergencia cuadrática del Newton global lo compensa."
- "Transición loadingunloading es discontinua en la tangente. Newton converge antes
  de oscilar entre ramas en casos típicos. Para problemas patológicos cerca del
  umbral, considerar arc-length."
- "Saturación d=DAMAGE_MAX corta la corrección consistente (d / deja de ser
  efectivamente (1-d) (1/+)); el algoritmo devuelve tangente secante en ese caso
  para evitar términos espurios."
- "Sin regularización, la solución es mesh-dependent en régimen de ablandamiento (
  localización en una banda de elementos). Para benchmark cuantitativo se requiere
  mismo refinamiento de malla; para comparaciones cualitativas (presencia de banda,
  dirección) basta con regularización de gradiente o longitud característica (
  fuera de scope)."
```

acceptance:

```

# Cubierto por tests/test_materials_unit.py · TestIsotropicDamage2D (preexistentes)
# y tests/test_damage2d_consistent_tangent.py (nuevos)
unit_existing_no_regression:
- name: secant_response_below_threshold
  expect: " _0 d=0, = C_e·, tangente = C_e (sin cambio respecto a versión
    secante anterior)"
  tol_rel: 1.0e-12

- name: damage_evolution_exponential
  expect: "para > _0, d cumple ley exponencial dentro de la fórmula"
  tol_rel: 1.0e-10

unit_consistent_tangent:
- name: tangent_equals_secant_on_unloading
  setup: "carga hasta >_0, descarga con de menor norma"
  expect: "C_tan = (1-d)·C_e exacto (segundo término se anula con / = 0)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_below_threshold
  setup: " pequeño, _old = _0"
  expect: "C_tan = C_e (sin daño activo, ni secante con d=0 ni corrección)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_at_saturation
  setup: "carga hasta enorme tal que d alcanza DAMAGE_MAX"
  expect: "C_tan = (1-DAMAGE_MAX)·C_e (el término consistente se corta porque d /
    efectivamente 0 al saturar)"
  tol_rel: 1.0e-12

- name: tangent_consistent_term_on_loading
  setup: "carga activa con > _0 y d < DAMAGE_MAX"
  expect: "C_tan (1-d)·C_e - incluye el término rank-1 -(1-d) (1/+) /_eq · (Ce·)
    (M·)"
  verification: "compara contra fórmula explícita evaluada en el mismo estado"
  tol_rel: 1.0e-9

- name: tangent_not_symmetric_in_loading
  setup: "carga activa con estado de deformación no degenerado (_xx _yy, _xy 0)
    "
  expect: "C_tan - C_tan^T 0 (al menos un elemento off-diagonal con diferencia
    significativa)"
  verification: "||C_tan - C_tan^T||_F / |||C_tan_F 0.01 en el caso de prueba"

- name: tangent_finite_difference_consistency
```

```

setup: "carga activa; calcular tangente por diferencia finita centrada (
    perturbación 1e-7) y comparar con cerrada"
expect: "||C_tan_FD -|| C_tan_closed_form / |||C_tan < 1e-5"
tol_rel: 1.0e-5

unit_arch:
- name: is_symmetric_attribute_is_false
  expect: "IsotropicDamage2D.IS_SYMMETRIC == False (atributo declarado en clase)"

integration:
- name: quad4_damage_newton_converges_in_few_iter
  setup: "Quad4 unitario, IsotropicDamage2D plane stress, carga uniaxial hasta régimen
    plenamente dañado"
  expect: "Newton converge en 6 iteraciones por paso (evidencia de convergencia
    cuadrática con tangente consistente)"

references:
- "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
  Cap. 7 (continuum damage mechanics)."
```

```

- "Oliver J. (1989). A consistent characteristic length for smeared cracking models.
  IJNME 28, 461-474."
```

```

- "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
  Formulation. Int. J. Solids Struct. 23, 821-840 (tangente consistente para daño
  isótropo)."
```

```

- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for
  Plasticity. Wiley. §12 (damage models)."
```

5.22 Implementación

- **Archivo:** solidum/materials/damage_2d.py.
- **Clase:** IsotropicDamage2D, registrada vía @MaterialRegistry.register. Declara IS_SYMMETRIC = False como ClassVar (sobreescribe el default True de Material).
- **Estado:** dict {'kappa': float, 'damage': float}. PRIMARY_STATE_VAR = 'damage'.
- **Algoritmo compute_state:**
 1. Recuperar κ_n y calcular $\varepsilon_{eq}(\varepsilon)$.
 2. Si $\varepsilon_{eq} > \kappa_n$ y $\kappa_{n+1} > \kappa_0$: rama de carga activa.
 3. Calcular d por la ley exponencial, saturar a DAMAGE_MAX.
 4. Calcular $\sigma = (1 - d)\mathbf{C}_e\varepsilon$.
 5. Tangente: rama según §7 (secante o consistente). En la rama consistente, calcular el término rank-1 una sola vez y restar a $(1 - d)\mathbf{C}_e$.
- **admissibility_scale:** sin cambios respecto a versión previa, $E \cdot \kappa_0$ (umbral inicial — la escala no evoluciona con el estado porque el criterio se mide contra κ_0 , no contra κ).
- **Compatibilidad:** el cambio rompe IS_SYMMETRIC para este material. El despachador algebraico (ADR 0003) verifica domain_is_symmetric agregando material.IS_SYMMETRIC sobre todos los materiales del dominio; basta un material con IS_SYMMETRIC=False para que el solver elija LU.

- **Tests:**
- tests/test_materials_unit.py · **TestIsotropicDamage2D** (existente, no regresión) + nueva clase **TestIsotropicDamage2DConsistentTangent**.
- tests/test_solid_2d_damage.py nuevo: integración Quad4 + IsotropicDamage2D + Nonlinear-Solver con tangente consistente.

5.23 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Sin ADR: extiende un material existente añadiendo un término a la tangente; no rompe contratos arquitecturales transversales. Sí cambia **IS_SYMMETRIC** para este material, lo cual el despachador algebraico ya soporta vía agregación sobre el dominio (no requiere refactor).
- **2026-05-14** · Decisión: la transición loading $\leftrightarrow$ unloading se decide con κ_n committed (no con κ trial dentro del Newton). Esto es lo estándar y evita oscilaciones del Newton entre ramas en pasos con ambigüedad. Si el paso converge, κ se compromete y la decisión es consistente con la trayectoria.
- **2026-05-14** · **IsotropicDamage1D** queda con tangente secante. Misma deuda gemela conceptual, pero fuera del alcance pactado (A2 era solo 2D). Si el usuario la prioriza luego, replicar es trivial: misma fórmula reducida a escalar.

5.24 DruckerPrager2D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

5.25 Especificación física

5.25.1 0. Descripción general

Modelo elastoplástico **friccional** independiente de la velocidad. Suelos, hormigón, granulares, materiales cuasi-frágiles cohesivo-friccionales. Suaviza la pirámide hexagonal de Mohr-Coulomb a un **cono circular** en el espacio de tensiones principales, eliminando las aristas que complican el return mapping.

Diferencia clave con J2:

- **Dependencia de la presión.** La resistencia al corte aumenta con la presión de confinamiento (componente friccional).
- **Plasticidad no asociada.** La dilatancia volumétrica real del flujo plástico (ψ) es típicamente menor que la fricción del criterio (ϕ). Forzar $\psi = \phi$ (asociada) sobreestima la dilatancia y el volumen post-fluencia.
- **Retorno al ápice.** El criterio define un cono con vértice; estados de prueba en zona puramente hidrostática extrema retornan al vértice (zona singular en la dirección de flujo).

Hipótesis cinemática soportada: **plane_strain** únicamente en esta spec.

5.25.2 1. Descomposición aditiva y elasticidad

Pequeñas deformaciones: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$. Ley elástica isótropa: $\boldsymbol{\sigma} = \mathbf{C}_e(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$.

5.25.3 2. Criterio de fluencia (Drucker-Prager)

$$f(\boldsymbol{\sigma}, \alpha) = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$$

con:

- $J_2 = \frac{1}{2} \mathbf{s} : \mathbf{s}$, segundo invariante del desviador.
- $I_1 = \text{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz}$, primer invariante.
- η_f , coeficiente friccional del cono (adimensional, ≥ 0).
- $k(\alpha) = k_0 + H_k \alpha$, parámetro cohesivo con endurecimiento isótropo lineal.

La superficie es un cono con eje en la línea hidrostática ($\sigma_{xx} = \sigma_{yy} = \sigma_{zz}$), vértice en $I_1 = k/\eta_f$, abertura controlada por η_f .

5.25.4 3. Calibración con Mohr-Coulomb

Los parámetros (η_f, k_0) se derivan de los parámetros físicos (c_0, ϕ) — cohesión y ángulo de fricción interna — según la variante del cono elegida:

variant	η_f	k_0
plane_strain_matched (default)	$\tan(\phi)/\sqrt{9 + 12 \tan^2(\phi)}$	$3 c_0/\sqrt{9 + 12 \tan^2(\phi)}$
outer_cone	$2 \sin(\phi)/[\sqrt{3} (3 - \sin(\phi))]$	$6 c_0 \cos(\phi)/[\sqrt{3} (3 - \sin(\phi))]$
inner_cone	$2 \sin(\phi)/[\sqrt{3} (3 + \sin(\phi))]$	$6 c_0 \cos(\phi)/[\sqrt{3} (3 + \sin(\phi))]$

`plane_strain_matched` es la variante que coincide exactamente con Mohr-Coulomb en condiciones de plane strain (Drucker, 1953). Las otras dos circunscriben (outer) o inscriben (inner) el hexágono de MC en el plano π . Como esta spec solo soporta plane strain, `plane_strain_matched` es el default natural.

5.25.5 4. Regla de flujo (no asociada por defecto)

Potencial plástico g con coeficiente de **dilatancia** η_g (función del ángulo de dilatancia ψ , calibrado con la misma fórmula que η_f pero usando ψ):

$$g(\boldsymbol{\sigma}) = \sqrt{J_2} + \eta_g I_1$$

$$\dot{\boldsymbol{\epsilon}}^p = \dot{\gamma} \frac{\partial g}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \left(\frac{\mathbf{s}}{2\sqrt{J_2}} + \eta_g \mathbf{I} \right)$$

Descomposición:

- **Parte desviadora:** $\dot{\boldsymbol{\epsilon}}^p = \dot{\gamma} \mathbf{s}/(2\sqrt{J_2})$ — análoga a J2.
- **Parte volumétrica:** $\text{tr}(\dot{\boldsymbol{\epsilon}}^p) = 3\dot{\gamma} \eta_g$ — **dilatación plástica positiva** si $\eta_g > 0$.

Si $\psi = \phi$ (i.e. $\eta_g = \eta_f$) el modelo se vuelve **asociado**; tangente algorítmica simétrica. Para $\psi < \phi$ (típico en suelos: $\psi \approx \phi/2$ o incluso $\psi = 0$, "non-dilatat"), el modelo es **no asociado** y la tangente algorítmica es **asimétrica**.

5.25.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

con α la **deformación plástica equivalente** (convención del proyecto para Drucker-Prager). La superficie crece manteniendo eje y abertura, solo k aumenta linealmente.

Endurecimiento por fricción/dilatancia (cambio de η_f o η_g con α) queda fuera de esta spec — escenario que no necesitamos hoy y que añade no-linealidad fuerte al return mapping.

5.25.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

5.25.8 7. Variables internas

Símbolo	Tipo	Significado
ϵ^p	tensor 4 componentes [xx, yy, zz, xy_tensorial]	deformación plástica
α	escalar ≥ 0	deformación plástica equivalente (= acumulado de $\Delta\gamma$)

alpha es la variable principal exportable (PRIMARY_STATE_VAR = 'alpha').

5.26 Formulación numérica

5.26.1 8. Esquema implícito — Backward Euler

Dado $\{\epsilon_n^p, \alpha_n\}$ y ϵ_{n+1} , predictor elástico:

$$\epsilon^{\text{trial}} = \epsilon_{n+1} - \epsilon_n^p, \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}_{\text{dev}}^{\text{trial}}, \quad p^{\text{trial}} = K \text{tr}(\epsilon^{\text{trial}})$$

con $I_1^{\text{trial}} = 3p^{\text{trial}}$, $\sqrt{J_2^{\text{trial}}} = \|\mathbf{s}^{\text{trial}}\|/\sqrt{2}$.

Función de fluencia trial:

$$f^{\text{trial}} = \sqrt{J_2^{\text{trial}}} + \eta_f I_1^{\text{trial}} - k(\alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol} \rightarrow$ paso elástico. Si no, intentar **return regular**.

5.26.2 9. Return regular (cone surface)

La dirección desviadora $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}}/\sqrt{2J_2^{\text{trial}}} = \mathbf{s}/\sqrt{2J_2}$ es invariante bajo carga radial. Las actualizaciones son lineales en $\Delta\gamma$:

$$\begin{aligned} \sqrt{J_2^{n+1}} &= \sqrt{J_2^{\text{trial}}} - G \Delta\gamma \\ p^{n+1} &= p^{\text{trial}} - 3K \eta_g \Delta\gamma, \quad I_1^{n+1} = 3p^{n+1} \end{aligned}$$

La consistencia $f(\sigma_{n+1}, \alpha_{n+1}) = 0$ con $\alpha_{n+1} = \alpha_n + \Delta\gamma$ y $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$ da:

$$\Delta\gamma = \frac{f^{\text{trial}}}{G + 9K \eta_f \eta_g + H_k}$$

Forma cerrada — sin Newton local. Cada término del denominador:

- G : cambio en $\sqrt{J_2}$ por la parte desviadora del flujo.
- $9K \eta_f \eta_g$: cambio en $\eta_f I_1$ por la parte volumétrica dilatante del flujo (multiplicado por η_f del criterio y η_g del potencial).
- H_k : endurecimiento.

Actualizaciones:

$$\mathbf{s}^{n+1} = \frac{\sqrt{J_2^{n+1}}}{\sqrt{J_2^{\text{trial}}}} \mathbf{s}^{\text{trial}}$$

$$\boldsymbol{\sigma}^{n+1} = \mathbf{s}^{n+1} + p^{n+1} \mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma \left(\frac{\mathbf{s}^{n+1}}{2\sqrt{J_2^{n+1}}} + \eta_g \mathbf{I} \right)$$

Condición de validez del return regular: $\sqrt{J_2^{n+1}} \geq 0$. Si $\Delta\gamma_{\text{reg}} > \sqrt{J_2^{\text{trial}}}/G$, el desviador "se pasa de cero" el algoritmo regular es inválido — toca **return al ápice**.

5.26.3 10. Return al ápice

Cuando el estado de prueba cae en zona puramente hidrostática extrema (p_{trial} muy traccionante respecto a la cohesión), la proyección al cono cae en su vértice, donde $\mathbf{s} = 0$ y la dirección desviadora $\hat{\mathbf{n}}$ no está definida.

En el ápice: $\boldsymbol{\sigma}^{n+1} = (k_{n+1}/\eta_f) \mathbf{I}$ (puramente hidrostático), $\sqrt{J_2^{n+1}} = 0$.

Por conservación volumétrica:

$$p^{n+1} = p^{\text{trial}} - 3K \eta_g \Delta\gamma = \frac{k_{n+1}}{3\eta_f}$$

con $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$. Despejando:

$$\Delta\gamma_{\text{apex}} = \frac{p^{\text{trial}} \eta_f - k(\alpha_n)/3 \cdot \eta_f \cdot 3}{3K \eta_f \eta_g + H_k} = \frac{3p^{\text{trial}} \eta_f - k(\alpha_n)}{3K \cdot 3\eta_f \eta_g + H_k}$$

Reordeno: $3p^{\text{trial}} \eta_f - k(\alpha_n) = I_1^{\text{trial}} \eta_f - k(\alpha_n)$. Denominador: $9K \eta_f \eta_g + H_k$. (Equivalente a la fórmula regular sin el término G , porque la rama de apex es puramente volumétrica.)

$$\Delta\gamma_{\text{apex}} = \frac{I_1^{\text{trial}} \eta_f - k(\alpha_n)}{9K \eta_f \eta_g + H_k}$$

Actualizaciones (apex):

$$\mathbf{s}^{n+1} = 0, \quad p^{n+1} = K(\text{tr}(\boldsymbol{\varepsilon}_{n+1}) - 3\eta_g \Delta\gamma) \equiv \frac{k_{n+1}}{3\eta_f}$$

$$\boldsymbol{\sigma}^{n+1} = p^{n+1} \mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma \eta_g \mathbf{I} + \boldsymbol{\varepsilon}_{\text{trial}}^{p,\text{dev}}$$

(la parte desviadora plástica absorbe completamente el desviador de prueba, ya que $\mathbf{s}^{n+1} = 0$.)

5.26.4 11. Detección del régimen de retorno

Algoritmo de despacho:

1. Calcular $\Delta\gamma_{\text{reg}}$ por la fórmula regular.
2. Si $\sqrt{J_2^{\text{trial}}} - G \Delta\gamma_{\text{reg}} \geq 0 \rightarrow$ return regular.
3. Si no $\rightarrow$ return al ápice con $\Delta\gamma_{\text{apex}}$.

El criterio es geoméricamente: el predictor cae más allá del ápice si la proyección desviadora exigiría reducir $\sqrt{J_2}$ por debajo de cero.

5.26.5 12. Tangente algorítmica consistente

Return regular ($\sqrt{J_2^{n+1}} > 0$):

Por linealización del algoritmo:

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G \left(1 - \frac{G \Delta\gamma}{\sqrt{J_2^{\text{trial}}}} \right) \mathbf{I}_{\text{dev}} + 2G \Theta \hat{\mathbf{n}} \otimes \hat{\mathbf{n}} - 3K \eta_g \mathbf{1} \otimes \mathbf{a}_f - 3K \eta_f \mathbf{a}_g \otimes \mathbf{1} + \text{corrección}$$

donde:

- $\hat{\mathbf{n}} = \mathbf{s}^{n+1} / (2\sqrt{J_2^{n+1}})$ (dirección de flujo desviador, normalizada por convención).
- $\mathbf{1} = (1, 1, 1, 0)$ en notación 4 componentes (hidrostática).
- $\Theta, \mathbf{a}_f, \mathbf{a}_g$: términos cruzados que recogen el acoplamiento entre cambio de $\sqrt{J_2}$ y endurecimiento.

Forma compacta (de Souza Neto §8.3.4):

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G \left(1 - \frac{G \Delta\gamma}{\sqrt{J_2^{\text{trial}}}} \right) \mathbf{I}_{\text{dev}} - \frac{1}{G + 9K\eta_f\eta_g + H_k} \mathbf{b}_g \otimes \mathbf{b}_f$$

con $\mathbf{b}_g = G\hat{\mathbf{n}} + 3K\eta_g \mathbf{1}$, $\mathbf{b}_f = G\hat{\mathbf{n}} + 3K\eta_f \mathbf{1}$. **Asimétrica** si $\eta_f \neq \eta_g$ (no asociada).

Return al ápice ($\sqrt{J_2^{n+1}} = 0$):

$$\mathbf{C}^{\text{alg}} = K \left(1 - \frac{3K\eta_f\eta_g}{3K\eta_f\eta_g + H_k/3} \right) \mathbf{1} \otimes \mathbf{1}$$

— rigidez puramente volumétrica reducida (el material en el ápice se comporta como un fluido con compresibilidad finita; sin rigidez desviadora alguna).

Forma equivalente más limpia: $\mathbf{C}_{\text{apex}}^{\text{alg}} = \frac{K H_k}{9K\eta_f\eta_g + H_k} \mathbf{1} \otimes \mathbf{1}$ (con $H_k = 0$, el ápice se vuelve completamente plástico volumétrico y la tangente colapsa; este caso queda manejado en el código devolviendo un escalar pequeño > 0 para evitar singularidad).

5.26.6 13. Tolerancia de fluencia

ADR 0006: `admissibility_scale = k(α) = k_0 + H_k·α` (escala cohesiva corriente del cono). Tiene unidades de esfuerzo, igual que f .

5.27 Contrato de implementación

```

name: DruckerPrager2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha
  is_symmetric: false      # tangente asimétrica si no asociada ( )

parameters:
- { name: E,          type: float, required: true,  desc: "Módulo de Young" }
- { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
- { name: cohesion,   type: float, required: true,  desc: "Cohesión c_0 inicial ( esfuerzo)" }
- { name: phi_deg,    type: float, required: true,  desc: "Ángulo de fricción interna en grados; típico suelos 20-40°, hormigón 30-37°" }
- { name: psi_deg,     type: float, required: false, default: null, desc: "Ángulo de dilatación en grados. Default = phi_deg (asociada). Típico no asociada: < , e.g . 0 o /2." }
- { name: H,          type: float, required: false, default: 0.0, desc: "Endurecimiento isótropo lineal en cohesión H_k (0). H=0 perfectamente plástico." }
- { name: hypothesis, type: str,   required: false, default: "plane_strain", desc: "plane_strain (único soportado)" }
- { name: variant,     type: str,   required: false, default: "plane_strain_matched", desc: "plane_strain_matched | outer_cone | inner_cone - calibración con Mohr-Coulomb" }
- { name: density,     type: float, required: false, default: null, desc: "Densidad ( ADR 0008)" }

signature:
  compute_state: " (: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_alg: ndarray (3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  eps_p: "ndarray(4,) - [^p_xx, ^p_yy, ^p_zz, ^p_xy_tensorial]"
  alpha: "float 0 - multiplicador plástico acumulado"

conventions:
  sign: "> 0 tracción (Reglas §5)"
  voigt: "_xy = 2·_xy en deformación; ^p_xy en tensorial (sin factor 2)"
  associated_flow: " = asociada (tangente simétrica); no asociada"

validity:
- "E > 0, c_0 > 0, 0 < 90°, 0"
- " (-1, 0.5)"
- "H 0"
- "hypothesis == 'plane_strain' (único soportado en esta spec)"
- "variant {'plane_strain_matched', 'outer_cone', 'inner_cone'}"

out_of_scope:
- "plane_stress - la proyección de Drucker-Prager con _zz=0 acoplada a regla de flujo dilatante es notoriamente delicada; difiere hasta caso real"
- "endurecimiento por cambio de y/o con - no lineal fuerte en el return mapping"
- "endurecimiento cinemático (back-stress hidrostático y desviador)"

```

```

- "cap (modelo cap-cone) para suelos compactables"
- "ablandamiento (H<0) - requiere regularización"
- "grandes deformaciones"

numerical_caveats:
- "Tangente algorítmica asimétrica si → IS_SYMMETRIC=False, el despachador
  algebraico usa LU (ADR 0003)."
```

- "Modos de carga muy traccionantes pueden caer en la rama de ápice. La detección automática del régimen (regular vs apex) está en el algoritmo; si en el futuro aparece oscilación entre ramas en pasos cerca de la transición, considerar smoothing con vertex regularization."

- "Con $\nu = 0$ (sin dilatancia) y $H_k = 0$, en la rama de ápice la tangente volumétrica se anula → singularidad. El código devuelve un valor de respaldo (rigidez volumétrica reducida no nula) para evitar matriz singular; típicamente sale del ápice en el siguiente paso."

- "El return regular puede generar crecimiento con incrementos pequeños cerca de la transición regularapex. Newton global converge igual; si en problema concreto se detecta oscilación, usar arc-length."

```

acceptance:
  unit:
    - name: parameter_validation
      expect: "constructor rechaza variant inválida,  $90^\circ$ ,  $>$ , hypothesis distinto de plane_strain"

    - name: calibration_matched_consistency
      expect: "para variant='plane_strain_matched' con  $\nu = 0$ , asociada →  $\nu_f = \nu_g$ ; con  $\nu = 0$ ,  $\nu_g = 0$  (sin dilatación)"

    - name: paso_elastico_below_yield
      expect: "pequeño;  $\nu = C_e \cdot \epsilon_p$  y alpha intactos"
      tol_rel: 1.0e-10

    - name: regular_return_under_shear
      setup: "cortante puro suficiente para fluir ( $\gamma_{xy} = \gamma/2$ , sin parte hidrostática)"
      expect: "return regular activo,  $\gamma > 0$ ,  $\sqrt{J}$  del estado final =  $\sqrt{(2/3) \cdot k - \nu_f \cdot I}$  0 (cumple criterio)"
      tol_rel: 1.0e-8

    - name: apex_return_under_pure_tension
      setup: "puramente hidrostática traccionante ( $\sigma_{xx} = \sigma_{yy} > 0$ , sin cortante)"
      expect: "return al ápice;  $\sigma_{xx} = \sigma_{yy} = k/(3 \cdot \nu_f)$  tras converger;  $\sqrt{J} = 0$ "
      tol_rel: 1.0e-8

    - name: associated_tangent_is_symmetric
      setup: " $\nu = 0$  (asociada), estado plástico activo"
      expect: "C_alg simétrica  $\|(C_{alg} - C_{alg}^T)\| < tol$ "
      tol_abs: 1.0e-8

    - name: nonassociated_tangent_is_asymmetric
      setup: " $\nu = 0$ ,  $\nu = 30^\circ$ , estado plástico activo no degenerado"
      expect: "C_alg - C_alg^T 0 con norma significativa"

    - name: tangent_matches_finite_difference
      setup: "estado plástico regular; comparar C_alg cerrada con diferencia finita centrada"
      expect: "error relativo < 1e-5"
      tol_rel: 1.0e-5

```



```

- name: alpha_monotonic_under_increasing_load
  expect: "bajo carga creciente no decrece"

- name: unloading_recovers_C_e
  setup: "cargar a plástico, descargar elásticamente"
  expect: "tangente = C_e en el paso de descarga, se conserva"
  tol_rel: 1.0e-10

- name: unit_invariance_MPa_vs_Pa
  expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) idéntico"
  tol_rel: 1.0e-12

integration:
- name: quad4_pipeline_elastico
  setup: "Quad4 unitario plane strain, carga muy por debajo del yield"
  expect: "respuesta elástica pura, =0"
  tol_rel: 1.0e-8

- name: quad4_pipeline_plastico_converges
  setup: "Quad4 unitario plane strain, carga que produce plasticidad activa, 4
    pasos, max_iter=8"
  expect: "Newton converge en cada paso, > 0 al final"

- name: quad4_yield_onset_confined_tension # DP-1 (2026-05-19)
  setup: "Quad4 unitario plane strain con desplazamientos prescritos en los 4 nodos
    para _xx=k·_y, _yy=0, _xy=0"
  expect: "_xx=0.5·_y elástico (_xx coincide con (+2)· exacto, =0); _xx=0.99·_y
    aún =0; _xx=1.5·_y plástico con >0 idéntico en los 4 Gauss points"
  tol_rel: 1.0e-8

- name: quad4_flow_rule_invariant # DP-2 (2026-05-19)
  setup: "Quad4 unitario plane strain, carga monótona en rama regular (5 niveles
    1.2·_y → 2.6·_y), "
  expect: "invariante cinemática tr(_p) = 3·_g· en cada Gauss point y en cada
    nivel"
  tol_rel: 1.0e-8

- name: quad4_apex_under_biaxial_tension # DP-2bis (2026-05-19)
  setup: "Quad4 unitario plane strain, _xx = _yy ~ 1e-2 (predicor hidrostático
    extremo)"
  expect: "estado final cae en ápice: _xx _yy = k()/ (3·_f), _xy 0, en cada
    Gauss point"
  tol_rel: 1.0e-6

references:
- "Drucker D.C., Prager W. (1952). Soil mechanics and plastic analysis or limit
  design. Quart. Appl. Math. 10, 157-165."
- "Drucker D.C. (1953). Coulomb friction, plasticity, and limit loads. J. Appl. Mech.
  21, 71-74. (calibración plane_strain_matched)"
- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §6 (cone
  plasticity, apex return)."
- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for
  Plasticity. Wiley. §8 (Drucker-Prager, tangentes algorítmicas detalladas)."
- "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. §7 (
  cohesivo-friccionales)."

```

5.28 Implementación

- **Archivo:** solidum/materials/drucker_prager_2d.py.
- **Clase:** DruckerPrager2D, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` declarativo (la tangente puede ser asimétrica; el despachador agrega el flag sobre todos los materiales del dominio).
- **Kernel Numba** `_compute_drucker_prager_plane_strain`:
 1. Predictor elástico desviador-volumétrico (paralelo a J2 plane strain, ε_{zz}^p libre).
 2. Check f^{trial} .
 3. Si elástico: devolver σ_{trial} restaurando contribución hidrostática y volumétrica.
 4. Si no: intentar $\Delta\gamma_{\text{reg}}$ cerrado; comprobar $\sqrt{J_2^{n+1}} \geq 0$.
 5. Si regular válido: actualizar σ , ε^p , α , tangente regular (de Souza Neto §8.3.4).
 6. Si regular inválido: rama de ápice — $\Delta\gamma_{\text{apex}}$ cerrado, σ puramente hidrostático, tangente volumétrica reducida.
- **Cálculo de** (η_f, k_0, η_g) : en `__init__` desde $(c_0, \phi, \psi, \text{variant})$ — sin coste runtime. Se valida $\psi \leq \phi$ (físicamente sensato).
- **admissibility_scale**: $k(\alpha) = k_0 + H \cdot \alpha$.
- **Tests**:
 - tests/test_materials_unit.py · `TestDruckerPrager2D`.
 - tests/test_solid_2d_drucker_prager.py.

5.29 Diálogo

- **2026-05-14** · Spec creada. Decisiones clave del alcance MVP:
- Solo `plane_strain` en esta entrega; `plane_stress` con cono en $\sigma_{zz}=0$ es notoriamente delicado y de uso geotécnico raro — se difiere.
- Parámetros físicos (c_0, φ, ψ) en lugar de adimensionales (k_0, η_f, η_g) — más natural para el usuario de geotecnia. Calibración interna por `variant`.
- **No asociada por defecto** (ψ es parámetro independiente con `default = \varphi` que la fuerza a asociada solo si el usuario no especifica). Plasticidad asociada en suelos sobreestima dilatación.
- Endurecimiento solo en cohesión (lineal); fricción/dilatación constantes.
- `IS_SYMMETRIC = False` declarativo a nivel material (independiente del caso particular de uso; conservador y consistente con el patrón del proyecto: el despachador agrega).
- **2026-05-14** · Sin ADR. El modelo encaja en el patrón establecido por `VonMises2D` (kernel Numba, despacho hypothesis aunque solo soporte una, semántica trial/commit) y no rompe contratos transversales.

Capítulo 6

Esquemas de Solución

6.1 ModalSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

6.2 Especificación física

6.2.1 0. Descripción general

Análisis modal lineal de una estructura discretizada por FEM: cálculo de frecuencias naturales y modos de vibración no amortiguados. Hipótesis: linealidad geométrica y material, amortiguamiento nulo ($\mathbf{C} = \mathbf{0}$), masa constante en el tiempo (lagrangeano total).

6.2.2 1. Problema físico

Vibración libre no amortiguada del sistema discreto:

$$\mathbf{M} \ddot{\mathbf{u}}(t) + \mathbf{K} \mathbf{u}(t) = \mathbf{0}.$$

Solución armónica $\mathbf{u}(t) = \boldsymbol{\phi} e^{i\omega t}$. Sustituyendo:

$$(\mathbf{K} - \omega^2 \mathbf{M}) \boldsymbol{\phi} = \mathbf{0}.$$

6.2.3 2. Problema de autovalor generalizado

Solución no trivial requiere $\det(\mathbf{K} - \omega^2 \mathbf{M}) = 0$. Equivalente:

$$\mathbf{K} \boldsymbol{\phi}_n = \omega_n^2 \mathbf{M} \boldsymbol{\phi}_n, \quad n = 1, \dots, N_{\text{dof}}.$$

Autovalores $\lambda_n = \omega_n^2 \geq 0$; autovectores $\boldsymbol{\phi}_n$ son los modos de vibración.

6.2.4 3. Propiedades del par $(\mathbf{K}, \mathbf{M})$

- $\mathbf{K}$ simétrica, positiva (semi)definida tras Dirichlet.
- $\mathbf{M}$ simétrica, **positiva definida** estrictamente para masa consistente (toda fila/columna tiene aporte de algún elemento con $\rho > 0$).
- Espectro real no negativo: $\omega_n \in \mathbb{R}_{\geq 0}$.

6.2.5 4. Salidas

- Frecuencias naturales ω_n (rad/s), $f_n = \omega_n/(2\pi)$ (Hz), períodos $T_n = 1/f_n$ (s).
- Modos ϕ_n M-ortonormales: $\phi_i^\top \mathbf{M} \phi_j = \delta_{ij}$.
- Ordenación ascendente: $\omega_1 \leq \omega_2 \leq \dots \leq \omega_{n_{\text{modos}}}$.

6.3 Formulación numérica

6.3.1 5. Reducción por Dirichlet

Por eliminación directa (ADR 0004) con operador $\mathbf{T} \in \mathbb{R}^{N_{\text{dof}} \times N_{\text{libre}}}$ que selecciona DOFs libres:

$$\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K} \mathbf{T}, \quad \mathbf{M}_{\text{red}} = \mathbf{T}^\top \mathbf{M} \mathbf{T}.$$

El problema reducido $\mathbf{K}_{\text{red}} \phi_{\text{red}} = \omega^2 \mathbf{M}_{\text{red}} \phi_{\text{red}}$ se resuelve sobre los DOFs libres. El término $\mathbf{g}_{\text{indep}}$ de restricciones lineales no homogéneas no aplica: los modos son solución del problema homogéneo (los modos de la estructura con apoyos a desplazamiento prescrito no nulo coinciden con los de apoyos nulos; el campo prescrito solo entra en la respuesta estática). Tras resolver, expansión $\phi = \mathbf{T} \phi_{\text{red}}$; en DOFs prescritos $\phi_i = 0$.

6.3.2 6. Algoritmo numérico: Lanczos con shift-invert

ARPACK vía `scipy.sparse.linalg.eigsh`:

$$\text{eigsh}(\mathbf{K}_{\text{red}}, k = n_{\text{modos}}, \mathbf{M} = \mathbf{M}_{\text{red}}, \sigma, \text{which}='LM', \text{tol}).$$

- **Shift-invert** transforma el problema en $(\mathbf{K}_{\text{red}} - \sigma \mathbf{M}_{\text{red}})^{-1} \mathbf{M}_{\text{red}} \phi = \mu \phi$ con $\mu = 1/(\omega^2 - \sigma)$. Buscar `which="LM"` (mayor magnitud de μ) equivale a buscar autovalores cercanos al shift σ .
- $\sigma = 0$ por defecto: las frecuencias más bajas, que son las relevantes en ingeniería estructural.
- Internamente ARPACK factoriza $(\mathbf{K}_{\text{red}} - \sigma \mathbf{M}_{\text{red}})$ una sola vez y la reusa en cada iteración Lanczos.

6.3.3 7. Normalización

`eigsh` con $\mathbf{M}$ explícita devuelve autovectores M-ortonormales sin reescalado adicional. Signo arbitrario (autovector definido módulo signo); convención de signo se documenta solo si emerge como preferencia (no parte del contrato).

6.3.4 8. Post-procesado

- $\omega_n = \sqrt{\lambda_n}$ (clip a cero los autovalores ligeramente negativos por ruido numérico, $|\lambda| < 10^{-14} \lambda_{\text{max}}$).
- $f_n = \omega_n/(2\pi)$, $T_n = 1/f_n$ (con $T_n = \infty$ si $\omega_n = 0$, modo de cuerpo rígido).
- Reordenación ascendente por ω_n .

6.4 Contrato de implementación

```

name: ModalSolver
kind: solver
status: validated          # draft → implemented → validated

interface:
  yaml_type: modal         # solver: { type: modal }
  output: ModalResult      # campos: frecuencias_hz, frecuencias_rad, periods, modes,
                           n_modes

parameters:
- { name: n_modes,        type: int,    required: true,   desc: "Número de modos a calcular"
  }
- { name: sigma,          type: float,  required: false, default: 0.0,    desc: "Shift (
  rad2/s2) para shift-invert; 0 → frecuencias más bajas" }
- { name: which,          type: str,    required: false, default: "LM",    desc: "
  Estrategia ARPACK: LM | SM | LA | SA | BE" }
- { name: tolerance,      type: float,  required: false, default: 1.0e-9,  desc: "
  Tolerancia ARPACK sobre el residuo del autovalor" }
- { name: lumping,        type: str,    required: false, default: "consistent", desc: "
  Discretización de masa; solo 'consistent' en fase 1" }
- { name: linear_algebra, type: str,    required: false, default: "auto",  desc: "Backend
  para la factorización de (K - M) interna a shift-invert (ADR 0003)" }

requirements:
- "Todos los materiales del modelo declaran `density > 0` (ADR 0008). Densidad cero
  genera M_red singular → eigsh con sigma=0 falla; el solver propaga el error con
  mensaje físico."
- "Al menos un DOF libre tras Dirichlet."
- "n_modes < N_libre (ARPACK exige k < n)."

conventions:
  units:          "heredadas del modelo (ADR 0008); en rad/s, f en Hz, T en s
                  consistentes con kg/N/m o equivalentes."
  frecuencias:    "y f expuestos simultáneamente; el usuario consulta cualquiera."
  modos:          "M-ortonormales  $\Phi$  (  $M \Phi = I$  ); signo arbitrario por modo."
  ordering:        "ascendente en :      ...      _n_modos."
  rigid_body:      "modos de cuerpo rígido aparecen con 0; Tm= se reporta sin error."

out_of_scope:
- "Modos complejos con amortiguamiento no proporcional."
- "Análisis modal sobre estado deformado (modos de pequeña amplitud alrededor de
  configuración prestressed) - diferido."
- "Masa lumped y factores de participación / masas efectivas - implementados (fase 2
  y fase 7 del ADR 0009, cerradas 2026-05-18); accesibles vía `lumping='lumped'` y
  `ResponseSpectrumSolver` respectivamente."

acceptance:
  verification:
- name: barra_axial_empotrada_libre
  setup: |
    Barra 1D elástica empotrada en x=0, libre en x=L. Discretización con
    N=20 elementos Truss2D alineados con el eje x. E=210e9 Pa,  $\rho=7850$  kg/m3,
    A=1e-4 m2, L=1.0 m.
  expect: |
    Frecuencias propias analíticas (ondas axiales en barra empotrada-libre):
     $\omega_n = ((2n-1) / (2L)) \cdot \sqrt{E / \rho}$ , n = 1, 2, 3, ...

```

```

    Los primeros 3 modos calculados coinciden con la solución analítica.
    tol_rel: 0.01
- name: viga_bernoulli_simplemente_apoyada
  setup: |
    Viga Bernoulli-Euler simplemente apoyada en sus dos extremos.
    Discretización con N=20 elementos Frame2DEuler alineados con el eje x.
    E=210e9 Pa,  $\rho=7850$  kg/m3, A=1e-4 m2, I=8.33e-10 m4, L=1.0 m.
  expect: |
    Frecuencias propias analíticas (flexión transversal):
     $\omega_n = (n/L)^2 \cdot \sqrt{E \cdot I / (\rho \cdot A)}$ ,  $n = 1, 2, 3, \dots$ 
    Los primeros 3 modos coinciden con la solución analítica.
    tol_rel: 0.02
specific:
- name: ortonormalidad_de_masa
  setup: "Cualquier modelo bien formado tras resolver el problema modal."
  expect: " $\Phi^T M \Phi = I$  (n_modos  $\times$  n_modos)."
  tol_abs: 1.0e-10
- name: ortogonalidad_de_rigidez
  setup: "Mismo modelo."
  expect: " $\Phi^T K \Phi = \text{diag}(\omega_1^2, \dots, \omega_n^2)$ ."
  tol_rel: 1.0e-8
- name: dirichlet_anula_modos_en_prescritos
  setup: "Barra empotrada-libre; nodo x=0 con ux=0."
  expect: "Componente del modo en el DOF prescrito = 0 exacto."
  tol_abs: 1.0e-14
- name: density_faltante_lanza_value_error
  setup: "Material sin density declarada en el modelo."
  expect: "ModalSolver.solve() lanza ValueError listando los materiales afectados (
    mismo patrón que assemble_self_weight, ADR 0008)."

references:
- "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa consistente)
  y cap. 11 (algoritmos de autovalor)."
- "Cook, Malkus & Plesha, Concepts and Applications of Finite Element Analysis, §
  -11.3§11.5."
- "Hughes T. J. R., The Finite Element Method, cap. 7."
- "Wilson E. L., Three-Dimensional Static and Dynamic Analysis of Structures, cap.
  10."
- "Lehoucq, Sorensen, Yang - ARPACK Users' Guide (Lanczos con shift-invert)."
- "ADR 0003 - Capa algebraica (factorización reutilizable de  $(K - M)$ )."
- "ADR 0008 - Material.density."
- "ADR 0009 - Análisis modal y dinámico (este solver es la fase 1: el análisis modal
  estricto, problema generalizado  $K \cdot \ddot{u} = -M \cdot \ddot{u}$ ; el análisis dinámico transitorio
  entra en fases 3-7)."
```

6.5 Implementación

- **Archivo:** solidum/math/solvers/modal.py (clase ModalSolver, registrada vía @SolverRegistry.register).
- **Backend algebraico:** solidum/math/linalg/eigen.py (clase EigenSolver envolviendo scipy.sparse.linalg).
- **Tipo de resultado:** ModalResult — dataclass frozen con frequencies_rad, frequencies_hz, periods, modes, n_modos, converged. Expone free_vibration(M, u0, u0_dot, t) para reconstruir analíticamente la respuesta temporal de vibración libre sin amortiguamiento por superposición modal — solución cerrada de $M \cdot \ddot{u} + K \cdot u = 0$ proyectada sobre los modos calculados (modos rígidos tratados aparte con evolución lineal $q_n(t) = a_n + b_n \cdot t$).

- **Entrypoint público:** `solidum.run_modal` para uso programático; `solidum.run_yaml` despacha automáticamente a `run_modal` cuando el YAML declara `solver: type: ModalSolver`.
- **Pipeline:** `Assembler.assemble_system()` → `Assembler.assemble_mass_matrix()` → `Assembler.reduce_M()` → `EigenSolver.solve(K_red, M_red, n_modes)` → expansión $\Phi = T \cdot \Phi_{\text{red}}$ → `ModalResult`.
- **Caché:** `Assembler` cachea `M_global` con el lumping usado; reusos posteriores del mismo análisis no recomputan.
- **Validación de density:** pre-chequeo agregado en `Assembler.assemble_mass_matrix` con el mismo patrón de ADR 0008 — `assemble_self_weight`.
- **Tests:**
 - `tests/test_modal.py` — 13 tests cubriendo:
 - **TestModalAxialBar:** barra empotrada-libre, 20 elementos `Truss2D`, frecuencias contra $\omega_n = ((2n-1)\pi/(2L)) \cdot \sqrt{E/\rho}$ (tol 1 %).
 - **TestModalSimplySupportedBeam:** viga biapoyada, 20 elementos `Frame2DEuler`, frecuencias contra $\omega_n = (n\pi/L)^2 \cdot \sqrt{EI/(\rho A)}$ (tol 2 %).
 - **TestModalAlgebraicProperties:** ortonormalidad $\Phi^T M \Phi = I$ (atol 1e-10) y ortogonalidad $\Phi^T K \Phi = \text{diag}(\omega^2)$ (rtol 1e-8) en ambos modelos.
 - **TestModalSolverContract:** errores agregados (`density` faltante, `run_modal` sin `solver` ni `n_modes`) y verificación de que `lumping="lumped"` corre tras ADR 0009 fase 2 (cerrada 2026-05-18).
 - **TestModalYamlPipeline:** pipeline end-to-end desde YAML.
- **Notas de traducción:**
 - El parámetro `linear_algebra` está en el constructor por coherencia con los demás solvers, pero en fase 1 no se usa: ARPACK gestiona internamente la factorización LU de $(K - \sigma M)$ para shift-invert. Cuando se enchufe a la capa algebraica (ADR 0003), se podrá inyectar Cholesky o LDL^T .
 - El bloque `convergence:` del YAML (ADR 0007) se omite silenciosamente para `ModalSolver` igual que para `LinearSolver`: no hay iteración Newton ni norma de residuo configurable; la tolerancia es la propia de ARPACK (`tolerance:`).
 - Matriz de masa consistente "translacional invariante rotacional" para `Truss*` y `Cable*` ($M = \text{kron}([2,1], [1,2]) \cdot \rho A L / 6, I_{\text{dim}}$). Para frames se ensambla la masa local axial + Hermitiana cúbica y se rota con $T^T \cdot M_{\text{local}} \cdot T$. Para `Frame3D` se incluye masa torsional con momento polar geométrico $J_p = I_y + I_z$ (no la constante de Saint-Venant J , que rige la rigidez). Para sólidos 2D, integración por la cuadratura del propio elemento (Tri3 usa fórmula analítica exacta porque su cuadratura 1-punto sería insuficiente).
 - Inercia rotacional propia de sección ($\rho I \cdot L$) **incluida** en los DOFs rotacionales de `Frame2D` (Euler, Timoshenko, EulerCorot) y `Frame3D` — coherente con Timoshenko y con la rigidez `K_local` que tiene términos rotacionales puros (ADR 0009 §1).

6.6 Diálogo

- **2026-05-13** · Validado. Los 13 tests de `tests/test_modal.py` cubren los criterios de **acceptance** (verificación analítica + propiedades algebraicas + contrato de errores + pipeline YAML) y pasan con las tolerancias declaradas. Promovido **status: validated**.

6.7 NewmarkSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

6.8 Especificación física

6.8.1 0. Descripción general

Análisis dinámico transitorio **lineal** de una estructura discretizada por FEM: integración temporal directa de la ecuación semidiscreta de movimiento bajo cargas dependientes del tiempo. Hipótesis: linealidad geométrica y material ($\mathbf{K}$ constante), masa $\mathbf{M}$ constante (lagrangeano total), amortiguamiento Rayleigh proporcional $\mathbf{C} = \alpha \cdot \mathbf{M} + \beta \cdot \mathbf{K}$. Apoyos Dirichlet constantes en el tiempo. La no-linealidad (Newton dentro de cada paso) se difiere a la fase 4 del ADR 0009.

6.8.2 1. Ecuación de movimiento semidiscreta

Tras la discretización espacial por FEM, el problema continuo

$$\rho \ddot{\mathbf{u}} - \operatorname{div} \boldsymbol{\sigma} = \mathbf{b}(\mathbf{x}, t)$$

se reduce al sistema acoplado de ODEs:

$$\mathbf{M} \ddot{\mathbf{u}}(t) + \mathbf{C} \dot{\mathbf{u}}(t) + \mathbf{K} \mathbf{u}(t) = \mathbf{F}(t),$$

con condiciones iniciales $\mathbf{u}(0) = \mathbf{u}_0$, $\dot{\mathbf{u}}(0) = \dot{\mathbf{u}}_0$ y condiciones de contorno Dirichlet $\mathbf{u}_s(t) = \mathbf{g}$ constantes.

6.8.3 2. Amortiguamiento Rayleigh

$$\mathbf{C} = \alpha \mathbf{M} + \beta \mathbf{K},$$

con coeficientes calibrados a partir de dos pares modales (ξ_1, ω_1) y (ξ_2, ω_2) :

$$\alpha = \frac{2\omega_1\omega_2(\xi_1\omega_2 - \xi_2\omega_1)}{\omega_2^2 - \omega_1^2}, \quad \beta = \frac{2(\xi_2\omega_2 - \xi_1\omega_1)}{\omega_2^2 - \omega_1^2}.$$

La razón de amortiguamiento del modo n -ésimo resulta $\xi_n = \frac{1}{2}(\alpha/\omega_n + \beta\omega_n)$ — exacta en ω_1, ω_2 y suavemente sobre-amortiguada fuera del rango. El usuario también puede pasar (α, β) directos.

6.8.4 3. Salidas

- $\mathbf{u}(t_k)$, $\dot{\mathbf{u}}(t_k)$, $\ddot{\mathbf{u}}(t_k)$ para cada paso temporal $k = 0, 1, \dots, N_t$.
- Vector de instantes $t_k = k \Delta t$.
- Diagnósticos: α, β Rayleigh efectivos; número de pasos; tiempo de factorización vs tiempo de pasos.

6.9 Formulación numérica

6.9.1 4. Método de Newmark- β (a-form)

Familia paramétrica con (β, γ) . Predictores:

$$\begin{aligned}\tilde{\mathbf{u}}_{n+1} &= \mathbf{u}_n + \Delta t \dot{\mathbf{u}}_n + \frac{\Delta t^2}{2} (1 - 2\beta) \ddot{\mathbf{u}}_n, \\ \tilde{\dot{\mathbf{u}}}_{n+1} &= \dot{\mathbf{u}}_n + \Delta t (1 - \gamma) \ddot{\mathbf{u}}_n.\end{aligned}$$

Sistema efectivo en aceleraciones:

$$(\mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}) \ddot{\mathbf{u}}_{n+1} = \mathbf{F}_{n+1} - \mathbf{C} \tilde{\dot{\mathbf{u}}}_{n+1} - \mathbf{K} \tilde{\mathbf{u}}_{n+1}.$$

Correctores:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta \Delta t^2 \ddot{\mathbf{u}}_{n+1}, \quad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma \Delta t \ddot{\mathbf{u}}_{n+1}.$$

6.9.2 5. Aceleración inicial

$\ddot{\mathbf{u}}_0$ se calcula consistentemente con la ecuación de movimiento en $t = 0$:

$$\mathbf{M} \ddot{\mathbf{u}}_0 = \mathbf{F}(0) - \mathbf{C} \dot{\mathbf{u}}_0 - \mathbf{K} \mathbf{u}_0.$$

6.9.3 6. Parámetros canónicos

Esquema	β	γ	Propiedades
Average acceleration (default)	1/4	1/2	Incondicionalmente estable, sin amortiguamiento numérico, exactitud $O(\Delta t^2)$
Linear acceleration	1/6	1/2	Condicionally estable ($\Delta t \leq 2\sqrt{3}/\omega_{\max}$), $O(\Delta t^2)$
Fox-Goodwin	1/12	1/2	Condicionally estable, $O(\Delta t^4)$
Diferencias centradas	0	1/2	Explícito; condicionalmente estable ($\Delta t \leq 2/\omega_{\max}$), $O(\Delta t^2)$

6.9.4 7. Imposición de Dirichlet con apoyos constantes

Mismo mecanismo que estática (ADR 0004 fase 1) con operador $\mathbf{T}$ que selecciona DOFs libres y $\mathbf{g}$ que recoge los valores prescritos: $\mathbf{u} = \mathbf{T} \mathbf{u}_{\text{free}} + \mathbf{g}$. Como $\mathbf{g}$ es constante, $\dot{\mathbf{u}} = \mathbf{T} \dot{\mathbf{u}}_{\text{free}}$ y $\ddot{\mathbf{u}} = \mathbf{T} \ddot{\mathbf{u}}_{\text{free}}$. El sistema efectivo se proyecta a DOFs libres:

$$\mathbf{A}_{\text{eff,red}} \ddot{\mathbf{u}}_{\text{free},n+1} = \mathbf{T}^\top (\mathbf{F}_{n+1} - \mathbf{K} \mathbf{g}) - \mathbf{C}_{\text{red}} \tilde{\dot{\mathbf{u}}}_{\text{free}} - \mathbf{K}_{\text{red}} \tilde{\mathbf{u}}_{\text{free}},$$

donde $\mathbf{A}_{\text{eff,red}} = \mathbf{M}_{\text{red}} + \gamma \Delta t \mathbf{C}_{\text{red}} + \beta \Delta t^2 \mathbf{K}_{\text{red}}$. El término $\mathbf{T}^\top \mathbf{K} \mathbf{g}$ se precalcula una vez. La expansión final a globales es $\mathbf{u}_n = \mathbf{T} \mathbf{u}_{\text{free},n} + \mathbf{g}$.

6.9.5 8. Factorización reutilizable

Con Δt constante y problema lineal, $\mathbf{A}_{\text{eff,red}}$ es constante. Se factoriza una vez al inicio (ADR 0003 — **FactorizedSolver**) y cada paso es una resolución triangular barata. Para N_t pasos sobre N_{libre} DOFs: coste $\sim O(\text{factor})_{\text{una vez}} + N_t \cdot O(N_{\text{libre}})_{\text{por paso}}$.

6.10 Contrato de implementación

```

name: NewmarkSolver
kind: solver
status: validated      # draft → implemented → validated

interface:
  yaml_type: NewmarkSolver
  output: TransientResult  # campos: t_history, u_history, udot_history, uddot_history

parameters:
  - { name: t_end,      type: float, required: true,      desc: "Tiempo final
    del análisis (s)" }
  - { name: dt,        type: float, required: true,      desc: "Paso temporal
    constante (s)" }
  - { name: beta,      type: float, required: false, default: 0.25, desc: "Parámetro
    de Newmark (default: average acceleration)" }
  - { name: gamma,     type: float, required: false, default: 0.5,  desc: "Parámetro
    de Newmark" }
  - { name: rayleigh,  type: dict,  required: false, default: null, desc: "
    Amortiguamiento Rayleigh: {alpha, beta} directos o {xi1, omega1, xi2, omega2}
    para calibración modal" }
  - { name: u0,        type: ndarray, required: false, default: null, desc: "
    Desplazamiento inicial (ndof,); cero por defecto" }
  - { name: u0_dot,    type: ndarray, required: false, default: null, desc: "Velocidad
    inicial (ndof,); cero por defecto" }
  - { name: F_func,    type: callable, required: false, default: null, desc: "Callback
    Python F_func(t) -> ndarray (ndof,); cero por defecto" }
  - { name: linear_algebra, type: str, required: false, default: "auto", desc: "Backend
    para factorizar A_eff (ADR 0003)" }
  - { name: lumping,   type: str, required: false, default: "consistent", desc: "
    Discretización de masa; solo 'consistent' en esta fase" }

requirements:
  - "Todos los materiales del modelo declaran `density > 0` (ADR 0008)."
```

"K lineal y constante (hipótesis de pequeñas deformaciones, material elástico)."

"Apoyos Dirichlet constantes en el tiempo."

```

conventions:
  units: "heredadas del modelo (ADR 0008); t en s,  en rad/s, F en N consistente con
    kg/N/m."
  damping: "Rayleigh proporcional: C = ·M + ·K. Si se pasan  (,) ,(,), (,) se
    derivan automáticamente."
  stability: |
    Default (=1/4, =1/2) es incondicionalmente estable. Para =0 (diferencias
    centradas) o =1/6 (lineal) el usuario es responsable de que Δt  2/_max;
    el solver no comprueba esta cota automáticamente.

out_of_scope:
  - "Apoyos dependientes del tiempo (multi-support excitation sísmica) - diferido."
  - "Δt adaptativo - diferido."
  - "Generalized -, Bossak - - diferidos (variantes adicionales de la familia con
    amortiguamiento numérico)."
```

"No-linealidad y disipación numérica controlada - implementados como variantes en clases hermanas (`NewtonNewmarkSolver`, `HHTSolver`, `NewtonHHTSolver`); fases 4 y 3-bis del ADR 0009, cerradas."

```

acceptance:

```

```

verification:
- name: oscilador_1gdl_no_amortiguado
  setup: |
    Sistema 1 GDL:  $M=1$ ,  $K=25$  ( $=5$  rad/s),  $C=0$ . Condiciones iniciales
     $u=1$ ,  $\dot{u}=0$ .  $F(t)=0$ .  $\Delta t = T/100$  (100 pasos por período). Integrar
    durante 4 períodos.
  expect: |
     $u(t) = \cos(\cdot t)$  exacto. Error máximo en  $u$  durante el intervalo
     $< 1\%$  ( $=1/4$  tiene error de período  $O(\Delta t^2)$  bajo control con 100
    pasos por período).
  tol_rel: 0.01
- name: oscilador_1gdl_amortiguado_rayleigh
  setup: |
    Sistema 1 GDL:  $M=1$ ,  $K=25$ ,  $C=2 \cdot \cdot \cdot M$  con  $=0.05$  (sub-amortiguado).
     $u=1$ ,  $\dot{u}=0$ ,  $F=0$ .  $\Delta t = T/100$ ,  $t_{\text{end}} = 4 \cdot T$ . Construir  $C$  vía
    Rayleigh con un único par ( $\cdot$ ,  $\cdot$ ).
  expect: |
     $u(t) = e^{-(\cdot)t} \cdot (\cos(\cdot_d \cdot t) + (\cdot/\cdot_d) \cdot \sin(\cdot_d \cdot t))$  con
     $\cdot_d = \cdot \sqrt{1 - \cdot^2}$ . Error máximo  $< 2\%$ .
  tol_rel: 0.02
- name: viga_voladizo_carga_subita_modal_vs_newmark
  setup: |
    Voladizo Frame2DEuler (20 elementos) con carga puntual aplicada en
    el extremo en  $t=0$  como step function.  $\Delta t = T/40$  ( $T$  período del
    primer modo),  $t_{\text{end}} = 2 \cdot T$ . Sin amortiguamiento.
  expect: |
    La reconstrucción modal de la respuesta (superposición de primeros
    5 modos forzados por  $F(t)=H(t) \cdot F$  con solución analítica conocida)
    coincide con Newmark a  $< 5\%$ .
  tol_rel: 0.05
- name: orden_de_convergencia
  setup: |
    Oscilador 1 GDL no amortiguado integrado con  $=1/4$ ,  $=1/2$  a  $\Delta$ 
     $t$ ,  $\Delta t/2$ ,  $\Delta t/4$ . Calcular el error máximo en  $u$  contra solución
    analítica.
  expect: |
    El cociente de errores  $e\Delta(t)/e\Delta(t/2)$  tiende a 4 (orden 2 exacto).
  tol_rel: 0.5 # tolerancia sobre el cociente:  $4 \pm 2$ 

specific:
- name: rayleigh_calibration
  setup: "Datos ( $=0.02$ ,  $=10$ ), ( $=0.05$ ,  $=100$ ), calibrar ( $\cdot$ ,  $\cdot$ )."
```

$$= 2 \cdot \cdot \cdot (\cdot - \cdot) / (\cdot^2 - \cdot^2) \quad 0.303,$$

$$= 2 \cdot (\cdot - \cdot) / (\cdot^2 - \cdot^2) \quad 9.6e-4.$$

```

  Y verificar ( $\cdot_n$ ) = ( $\cdot/\cdot_n + \cdot \cdot_n$ )/2 reproduce en  $\cdot$  y en  $\cdot$ .
  tol_rel: 1.0e-10

references:
- "Newmark N. M. (1959), A method of computation for structural dynamics, J. Eng.
  Mech. Div. ASCE."
- "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa) y cap. 9.4
  (Newmark)."
```

- "Hughes T. J. R., The Finite Element Method, cap. 9 (algoritmos de integración
temporal)."
- "Chopra A. K., Dynamics of Structures, cap. 5 (1 GDL) y cap. 15 (MDOF Newmark)."
- "ADR 0003 - Capa algebraica (factorización reutilizable de A_{eff})."
- "ADR 0007 - Convergencia y tolerancias."
- "ADR 0009 - Análisis modal y dinámico (este solver es la fase 3)."

6.11 Implementación

- **Archivo:** `solidum/math/solvers/newmark.py` (clase `NewmarkSolver`, registrada vía `@SolverRegistry.register`).
- **Amortiguamiento:** `solidum/math/damping.py` (`rayleigh_from_modes`, `rayleigh_xi`).
- **Tipo de resultado:** `TransientResult` — dataclass frozen con `t_history`, `u_history`, `udot_history`, `uddot_history`, `n_steps`, `alpha_rayleigh`, `beta_rayleigh`, `converged`.
- **Entrypoint público:** `solidum.run_transient` para uso programático; `solidum.run_yaml` despacha automáticamente cuando el YAML declara `solver: type: NewmarkSolver`.
- **Pipeline:** `Assembler.assemble_system()` → `Assembler.assemble_mass_matrix()` → $C = \alpha \cdot M + \beta \cdot K$ → reducción simultánea por Dirichlet (operador T) → cálculo de $\ddot{u}_0$ → factorización única de $A_{\text{eff_red}} = M_{\text{red}} + \gamma \Delta t \cdot C_{\text{red}} + \beta \Delta t^2 \cdot K_{\text{red}}$ → bucle de pasos con resolución triangular reutilizada.
- **Tests:**
 - `tests/test_newmark.py` — 14 tests:
 - `TestRayleighCalibration`: (α, β) reproduce ξ objetivo en ω_1, ω_2 ; errores agregados (ω iguales, no positivos).
 - `TestNewmark1DofUndamped`: vibración libre $u(t) = \cos(\omega t)$ a 1 %; estado inicial preservado.
 - `TestNewmark1DofDamped`: respuesta sub-amortiguada $e^{(-\xi \omega t)} \cdot (\cos(\omega_d t) + \dots)$ a 2 %; coeficientes Rayleigh propagados al `TransientResult`.
 - `TestNewmark1DofStepResponse`: $u(t) = (F_0/K)(1 - \cos(\omega t))$ a 0.1 %.
 - `TestNewmarkConvergenceOrder`: cociente de errores al refinar $\Delta t \in T/40, T/80, T/160$ cae en $[3.5, 4.5]$ → orden 2 confirmado.
 - `TestNewmarkContract`: validaciones de constructor y `run_transient`.
 - `TestNewmarkYamlPipeline`: end-to-end desde YAML.
- **Notas de traducción:**
 - **Factorización única:** con Δt constante y problema lineal, $A_{\text{eff_red}}$ es invariante; se factoriza al inicio (`A_solver.factorize(A_eff)`) y cada paso es una sustitución triangular barata (ADR 0003 — `FactorizedSolver`).
 - **reduce_pair vs reduce triple:** en el problema modal `reduce_pair(K, M)` basta porque g no entra en autovalores. En Newmark se manejan tres matrices (K, M, C) y un término constante $F_{\text{dir}} = T^T K \cdot g$ para apoyos no nulos; se hace explícitamente en `NewmarkSolver.solve()` sin un helper unificado (no aporta valor centralizarlo con un solo consumidor).
 - **Aceleración inicial consistente:** $M \cdot \ddot{u}_0 = F(0) - C \cdot \dot{u}_0 - K \cdot u_0$ resuelve un sistema lineal aparte; se usa el dispatcher de la capa algebraica sobre M_{red} (siempre SPD si la masa es consistente y la densidad estrictamente positiva).

- **F_func opcional:** si es `None`, se trata como vector cero (vibración libre). El callback se invoca una vez por paso con `t_{n+1}`; el usuario es responsable de la coherencia temporal (no se reusan valores entre pasos).
- **El bloque `convergence` del `YAML`** (ADR 0007) se omite para `NewmarkSolver` igual que para `LinearSolver` y `ModalSolver`: la fase 3 es lineal y no hay iteración Newton interna; cuando entre la fase 4 (Newton dentro de cada paso), el bloque se reincorporará.

6.12 Diálogo

- **2026-05-13** · Validado. Los 14 tests de `tests/test_newmark.py` cubren `acceptance.verification` (osciladores 1 GDL no amortiguado / amortiguado / step / orden de convergencia) y `acceptance.specific` (calibración Rayleigh). Promovido `status: validated`.

6.13 NewtonNewmarkSolver

*Variante de NewmarkSolver según la regla de variantes (Reglas.md §4). Documenta **solo lo que cambia** respecto al padre. Implementa la **fase 4 del ADR 0009**; reusa todas las decisiones arquitecturales del ADR padre — sin ADR nuevo.*

6.14 Qué cambia respecto a NewmarkSolver

NewmarkSolver integra $\mathbf{M} \cdot \ddot{\mathbf{u}} + \mathbf{C} \cdot \dot{\mathbf{u}} + \mathbf{K} \cdot \mathbf{u} = \mathbf{F}(t)$ asumiendo **K constante**: factoriza $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma \Delta t \cdot \mathbf{C} + \beta \Delta t^2 \cdot \mathbf{K}$ una vez y cada paso temporal es una resolución triangular barata.

NewtonNewmarkSolver resuelve el mismo problema cuando **la rigidez no es constante**: hay materiales con historia (plasticidad, daño) o no linealidad geométrica. En cada paso temporal se introduce un bucle de **Newton-Raphson** sobre el residuo dinámico, hasta cumplir el criterio de convergencia.

6.14.1 Residuo dinámico en cada paso

En el instante t_{n+1} las incógnitas son $(\mathbf{u}_{n+1}, \dot{\mathbf{u}}_{n+1}, \ddot{\mathbf{u}}_{n+1})$ ligadas por los correctores Newmark:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta \Delta t^2 \ddot{\mathbf{u}}_{n+1}, \quad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma \Delta t \ddot{\mathbf{u}}_{n+1}$$

(con predictores idénticos a la fase 3). El residuo dinámico es:

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) = \mathbf{F}_{\text{ext}}(t_{n+1}) - \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \mathbf{C} \dot{\mathbf{u}}_{n+1} - \mathbf{M} \ddot{\mathbf{u}}_{n+1} - \mathbf{F}_{\text{dir}}$$

donde $\mathbf{F}_{\text{int}}(\mathbf{u})$ es el vector de fuerzas internas no lineales (plástico, dañado, etc.) que el `Assembler.assemble_non_` ya calcula para los solvers estáticos (ADR 0003 + ADR 0006 + ADR 0008).

6.14.2 Jacobiano dinámico tangente

Derivando $\mathbf{R}$ respecto a $\ddot{\mathbf{u}}_{n+1}$ usando los correctores:

$$\mathbf{J} = -\frac{\partial \mathbf{R}}{\partial \ddot{\mathbf{u}}_{n+1}} = \mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}_t$$

donde $\mathbf{K}_t = \partial \mathbf{F}_{\text{int}} / \partial \mathbf{u}$ es la rigidez tangente del estado corriente. La iteración es:

$$\mathbf{J} \delta \ddot{\mathbf{u}} = \mathbf{R}, \quad \ddot{\mathbf{u}}_{n+1}^{(k+1)} = \ddot{\mathbf{u}}_{n+1}^{(k)} + \delta \ddot{\mathbf{u}}$$

Los desplazamientos y velocidades se actualizan con los mismos correctores Newmark tras cada update de $\ddot{\mathbf{u}}_{n+1}$.

6.14.3 Amortiguamiento Rayleigh — heredado, constante en el tiempo

$\mathbf{C} = \alpha \mathbf{M} + \beta_R \mathbf{K}_0$ con $\mathbf{K}_0$ la rigidez **elástica de referencia** (la primera ensamblada en $t = 0$, $\mathbf{u} = 0$). No se actualiza con la plasticidad: el Rayleigh es modelo aproximado de amortiguamiento estructural, no consecuencia de la cinemática elastoplástica. Mantenerlo constante evita un acoplamiento ad-hoc que ningún código de referencia (Abaqus, ANSYS, OpenSees) realiza por defecto.

6.14.4 Reducción y commit de estado

- **Reducción por Dirichlet** heredada de la fase 3 (ADR 0004): mismo operador $\mathbf{T}$ y $\mathbf{g}$ constantes; el residuo y el jacobiano se proyectan a DOFs libres antes de cada solve.
- **Commit de variables internas**: al converger el paso, se llama `assembler.commit_all_states()` — los estados ($\epsilon^p, \alpha, \kappa, d, \dots$) pasan de *trial* a *committed*. Mismo patrón que `NonlinearSolver` y `ArcLengthSolver`. Si el paso no converge, los estados trial se descartan (no se llama commit) y se reporta error.

6.14.5 Convergencia (ADR 0007)

Criterio dual fuerza + desplazamiento, calibrado al primer paso temporal con la escala del ensamblaje inicial:

- $\|\mathbf{R}\|/(\text{atol} + \text{rtol} \|\mathbf{F}\|) \leq 1$
- $\|\delta\mathbf{u}\|/(\text{atol} + \text{rtol} \|\mathbf{u}\|) \leq 1$

Idéntico al `NonlinearSolver` estático. La calibración se hace una vez al inicio del análisis; las tolerancias son adimensionales (invariantes bajo cambio de unidades) gracias a la calibración con `force_scale/disp_scale`.

6.14.6 Factorización por iteración

A diferencia de `NewmarkSolver` (que factoriza $\mathbf{J}$ una sola vez al inicio), aquí $\mathbf{K}_t$ cambia con cada iteración, así que $\mathbf{J}$ debe re-factorizarse. Se mantiene el patrón de **Newton modificado opcional** (ADR 0003 fase 2): si `freeze_tangent_after_iter` es `int`, se factoriza fresco las primeras N iteraciones del paso y se reusa la factorización en las siguientes. `None` $\Rightarrow$ Newton estándar (re-factoriza cada iteración).

6.14.7 Recuperación del comportamiento lineal

Si todos los materiales del modelo son lineales, $\mathbf{K}_t \equiv \mathbf{K}_0$ constante y el residuo se anula en una iteración de Newton (Δx exacto en el primer solve). Por tanto **NewtonNewmarkSolver** con materiales lineales reproduce exactamente `NewmarkSolver` (sin discretización extra ni error adicional). Esto se valida en los acceptance.

6.15 Contrato de implementación

```
name: NewtonNewmarkSolver
kind: solver
status: validated
extends: NewmarkSolver          # variante según Reglas §4

interface:
  type_yaml: NewtonNewmarkSolver
  pipeline_kind: transient      # mismo que NewmarkSolver

parameters_extra_to_NewmarkSolver:
```



```

- { name: convergence,                type: ConvergenceCriterion, required: false,
  desc: "Política ADR 0007. Default: ConvergenceCriterion() con rtols de solidum.
  constants." }
- { name: max_iter,                    type: int,                      required: false,
  default: 20, desc: "Máximo iteraciones Newton por paso temporal." }
- { name: freeze_tangent_after_iter, type: int or None,                required: false,
  default: null, desc: "Si int, Newton modificado: factoriza fresco las primeras N
  iter y reusa después (ADR 0003 fase 2)." }

parameters_inherited:
# Idénticos a NewmarkSolver: t_end, dt, beta, gamma, rayleigh, u0, u0_dot,
# F_func, linear_algebra, lumping. Ver docs/specs/NewmarkSolver.md.

state_handling:
commit: "assembler.commit_all_states() tras converger cada paso"
rollback: "estado trial descartado si el paso no converge → RuntimeError"

acceptance:
recovery_linear:
- name: linear_material_matches_NewmarkSolver
  setup: "Mismo modelo y carga que un caso analítico de NewmarkSolver (oscilador 1
  GDL, viga), pero con NewtonNewmarkSolver"
  expect: "u_history idéntico (rtol 1e-10) al de NewmarkSolver - convergencia en
  1-2 iter por paso"

nonlinear:
- name: oscilador_1gdl_elastoplastico_libre
  setup: "1 nodo, Truss2D con Elastoplastic1D, masa concentrada, condición inicial
  u_0 más allá del yield, vibración libre (F=0)"
  expect: "u(t) refleja decaimiento por disipación plástica; (deformación plá
  stica acumulada) > 0 al final"

- name: viga_voladizo_elastoplastica_pulso
  setup: "Frame2DEuler con varios elementos, masa distribuida, pulso de carga
  concentrada en el extremo libre"
  expect: "respuesta no lineal con redistribución plástica; converge en pocas iter
  por paso (max 6 típicamente)"

convergence:
- name: newton_converges_in_few_iter
  setup: "paso plástico activo en un benchmark"
  expect: "número de iteraciones Newton por paso max_iter; raramente excede 8 con
  tangente algorítmica consistente"

references:
- "ADR 0009 $Fase 4 - Transitorio no lineal Newmark + Newton"
- "Hughes T.J.R. (2000). The Finite Element Method, cap. 7-9 (Newmark, Newton dentro
  de paso temporal)"
- "Crisfield M.A. (1991). Non-Linear Finite Element Analysis of Solids and Structures
  , vol. 2 cap. 24 (dinámica no lineal)"
- "Spec padre: [docs/specs/NewmarkSolver.md](NewmarkSolver.md)"

```

6.16 Implementación

- Archivo: solidum/math/solvers/newmark.py — junto al padre.
- Clase: `NewtonNewmarkSolver(NewmarkSolver)`, registrada con `@SolverRegistry.register`.

Hereda `__init__` con kwargs adicionales `convergence`, `max_iter`, `freeze_tangent_after_iter`. Sobrescribe `solve()`.

■ **Flujo de `solve()`:**

1. Ensamblar $\mathbf{M}$, ensamblar sistema no lineal inicial para obtener $\mathbf{K}_0$ y $\mathbf{F}_{\text{int},0}$.
2. Calibrar amortiguamiento Rayleigh con $\mathbf{K}_0$ (constante en el tiempo).
3. Reducir por Dirichlet (mismo $\mathbf{T}$, $\mathbf{g}$, $\mathbf{F}_{\text{dir}}$ que la fase 3).
4. Aceleración inicial consistente: $\mathbf{M} \ddot{\mathbf{u}}_0 = \mathbf{F}_{\text{ext}}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C} \dot{\mathbf{u}}_0 - \mathbf{F}_{\text{dir}}$.
5. Bucle temporal con Newton interno: predictor $\rightarrow$ iteración ($\mathbf{R}, \mathbf{J} = \mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}_t$), factoriza/solve, actualiza correctores; converger; commit.
6. Volcar historial; reportar éxito.

- **Refactor del padre:** la fase 3 (`NewmarkSolver`) sigue intacta. La subclase no extrae métodos del padre por ahora (el flujo no lineal es lo suficientemente distinto como para que la herencia sea estructural, no compartición de pasos internos). Cuando emergió la variante HHT- α (2026-05-18, `NewtonHHTSolver` como subclase de `NewtonNewmarkSolver`) la herencia estructural funcionó tal cual — los métodos `solve()` se sobrescriben íntegramente en cada subclase sin compartir pasos internos. Si en el futuro entran generalized- α u otras variantes y se observa que comparten *este* flujo no lineal específico, entonces se refactorizará al patrón con `_solve_step` virtual.

- **Despacho YAML:** `solver: type: NewtonNewmarkSolver` activado vía `SolverRegistry` y `run_yaml`. Como en `NewmarkSolver`, `PIPELINE_KIND = "transient"` declarativo: si futura regla disparo `C` reemplaza el `isinstance` en `entry.run_yaml`, no se rompe.

■ **Tests:**

- `tests/test_newmark_nonlinear.py` nuevo: acceptance.
- Cobertura de regresión de los tests de `NewmarkSolver` permanece intacta (la subclase no toca el padre).

6.17 Diálogo

- **2026-05-14** · Spec creada como **variante** de `NewmarkSolver` aplicando la nueva regla §4 (registrada en el mismo commit que esta spec). Sin ADR nuevo: reusa las decisiones de ADR 0009 fase 4 (residuo, jacobiano, conmutar Newton dentro del paso). Sin entrada propia en el manual estructural — la subclase es invisible al usuario API (excepto por el `type` en YAML y los parámetros extra).
- **2026-05-14** · Decisión: **subclase** sobre `NewmarkSolver` en lugar de bandera `nonlinear=True` en la misma clase. Razones: (i) más explícito en el código y en el YAML (`type` distinto para análisis distinto); (ii) evita condicionales que ensucian el flujo lineal de la fase 3; (iii) deja sitio para variantes adicionales (HHT- α no lineal, generalized- α) por composición. El ADR 0009 fila "Fase 4" decía "`NewmarkSolver` (mismo, con `nonlinear=True`)– interpretación textualista vs. arquitectural: la regla §4 nueva legitima la elección de subclase, y el patrón conserva el espíritu (mismo análisis, mismas decisiones, mismo entripoint) sin la implementación literal.

- **2026-05-14** · Amortiguamiento Rayleigh queda fijado con $\mathbf{K}_0$ (rigidez elástica de referencia) y constante en el tiempo. Alternativa rechazada: Rayleigh con $\mathbf{K}_t$ corriente — no es la convención estándar y crea coupling artificial entre la disipación viscosa y la plástica. Documentado en sección `.Amortiguamiento Rayleigh — heredado`”.

Capítulo 7

Elementos — catálogo

Referencia rápida de los elementos implementados. Una entrada por elemento. Para detalles físicos/numéricos → código fuente.

> **Convenciones:** *STRAIN_DIM* = dimensión Voigt esperada del material asociado (1 = axial escalar, 3 = 2D [ε_{xx} , ε_{yy} , γ_{xy}], 6 = 3D). DOFs por nodo = *DOF_NAMES*.

7.1 Truss2D — barra axial 2D

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos; transmite exclusivamente fuerza axial.

7.1.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.

7.1.2 Formulación

Cosenos directores del eje: $c = (x_2 - x_1)/L$, $s = (y_2 - y_1)/L$.

$$\begin{aligned} &= B \cdot u_e, & B &= (1/L) \cdot [c, -s, c, s] \\ K_e &= (E \cdot A / L) \cdot dd, & d &= -[c, -s, c, s] \\ F_{int} &= -A \cdot d \end{aligned}$$

7.1.3 Convenciones

- Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).
- *STRAIN_DIM* = 1: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (K_e y F_{int} son invariantes bajo su permutación).

7.1.4 Parámetros

- A — área de la sección transversal.

7.1.5 Cargas distribuidas

- `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{A} \cdot L_0 / 2$ por nodo en cada componente global (forma cerrada exacta para $\mathbf{b}$ uniforme). Útil para peso propio: $\mathbf{b} = (0, -\rho \cdot \mathbf{g})$.

7.1.6 Régimen de validez

- Pequeñas deformaciones ($|\varepsilon| \lesssim 10^{-2}$) y pequeños desplazamientos.
- Cargas exclusivamente axiales. Si la carga transversal sobre la barra no es despreciable $\rightarrow$ `Frame2DEuler` / `Frame2DTimoshenko`.

7.1.7 Validación

- Tests: `tests/test_truss.py · TestTruss2D`.
- Archivo fuente: `solidum/elements/truss.py · clase Truss2D`.
- Spec: `docs/specs/Truss2D.md`.
- Referencia: Bathe, *Finite Element Procedures*, §4.2.1.

7.2 Truss2DCorot — armadura 2D corotacional

Barra axial 2D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss2D`; comparte DOFs, parámetros y contrato con el material.

7.2.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.

7.2.2 Formulación

```
d = -[ c_, -s_, c_, s_]      (dirección corriente del eje)
n = -[ s_,  c_, s_, -c_]    (perpendicular al eje)
K_M  = (E·A / L) · d·d
K_G  = (N / l) · n·n        (rigidez geométrica)
K_T  = K_M + K_G
F_int = N · d
```

7.2.3 Cargas distribuidas

- Heredado de `Truss2D`: `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{A} \cdot L_0 / 2$ por nodo, evaluado en geometría de referencia (aproximación estándar para cargas conservadoras en formulaciones corotacionales).

7.2.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud.
- Cargas exclusivamente axiales.
- **Fuera de alcance:** pandeo por bifurcación (se capta ablandamiento precrítico pero no el punto crítico; para snap-through usar arc-length).

7.2.5 Validación

- Tests: tests/test_truss.py · TestTruss2DCorot (3 tests, uno por criterio de aceptación).
- Archivo fuente: solidum/elements/truss.py · clase Truss2DCorot.
- Spec: docs/specs/Truss2DCorot.md.
- Referencias: Crisfield §3.3, Belytschko §4.5.

7.3 Truss3D — barra axial 3D

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite exclusivamente fuerza axial. Régimen estrictamente de **linealidad geométrica**.

7.3.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.
- Configuración inicial fija: L_0 , c , c_y , c_z no se actualizan con los desplazamientos.

7.3.2 Formulación

Cosenos directores del eje: $c = (x_2 - x_1)/L$, $c_y = (y_2 - y_1)/L$, $c_z = (z_2 - z_1)/L$.

$$\begin{aligned} &= B \cdot u_e, & B &= (1/L) \cdot [-c, -c_y, -c_z, c, c_y, c_z] \\ K_e &= (E \cdot A / L) \cdot d \cdot d, & d &= [-c, -c_y, -c_z, c, c_y, c_z] \quad (6 \times 6, \text{ rango } 1) \\ F_{int} &= -A \cdot d \end{aligned}$$

7.3.3 Convenciones

- Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).
- STRAIN_DIM = 1: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (K_e y F_{int} son invariantes bajo su permutación).
- Acepta nodos con 2 ó 3 coordenadas (completa con $z = 0$ si la tercera falta).

7.3.4 Parámetros

- A — área de la sección transversal.

7.3.5 Cargas distribuidas

- `compute_body_load(b)` reparte $\mathbf{b} \cdot A \cdot L_0 / 2$ por nodo en cada componente global (forma cerrada exacta). Útil para peso propio: $\mathbf{b} = (0, 0, -\rho \cdot g)$.

7.3.6 Régimen de validez

- Pequeñas deformaciones ($|\varepsilon| \lesssim 10^{-2}$), pequeños desplazamientos y pequeñas rotaciones.
- Cargas exclusivamente axiales.
- Para grandes rotaciones en 3D $\rightarrow$ `Truss3DCorot`.

7.3.7 Validación

- Tests: `tests/test_truss.py · TestTruss3D`.
- Archivo fuente: `solidum/elements/truss.py · clase Truss3D`.
- Spec: `docs/specs/Truss3D.md`.
- Referencias: Bathe §4.2.1; Cook §2.3.

7.4 Truss3DCorot — barra axial 3D corotacional

Barra axial 3D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss3D`; comparte DOFs, parámetros y contrato con el material.

7.4.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (1 - L_0)/L_0$.
- La dirección perpendicular al eje es un **plano** (dimensión 2), no un vector único.

7.4.2 Formulación

$\mathbf{d} = -[c, -c_y, -c_z, c, c_y, c_z]$	(dirección corriente del eje)
$\hat{\mathbf{e}} = (c, c_y, c_z), \quad \mathbf{P} = \mathbf{I} - \hat{\mathbf{e}} \cdot \hat{\mathbf{e}}$	(proyector 3×3 al plano perpendicular)
$\mathbf{K}_M = (\mathbf{E} \cdot A / L) \cdot \mathbf{d} \cdot \mathbf{d}$	(6×6, rango 1)
$\mathbf{K}_G = (\mathbf{N} / L) \cdot [[\mathbf{P}, -\mathbf{P}], -[\mathbf{P}, \mathbf{P}]]$	(6×6, rango 2)
$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G$	
$\mathbf{F}_{int} = \mathbf{N} \cdot \mathbf{d}$	

7.4.3 Cargas distribuidas

- Heredado de `Truss3D`: `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo, evaluado en geometría de referencia.

7.4.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud en el espacio.
- Cargas exclusivamente axiales.

7.4.5 Validación

- Tests: `tests/test_truss.py` · `TestTruss3DCorot` (3 tests, uno por criterio de aceptación; el de rigidez geométrica verifica dos direcciones transversas independientes del plano perpendicular).
- Archivo fuente: `solidum/elements/truss.py` · clase `Truss3DCorot`.
- Spec: `docs/specs/Truss3DCorot.md`.
- Referencias: Crisfield §3.3; Belytschko §4.5.

7.5 Cable2DCorot — cable 2D corotacional

Elemento 1D inmerso en el plano que modela un cable: dos nodos, transmite solo tensión. Cinemática corotacional; la unilateralidad la aporta el material (típicamente `CableMaterial1D`).

7.5.1 Cinemática

- Idéntica a una barra corotacional: longitud y cosenos directores se recalculan en configuración corriente en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.
- La respuesta física de cable aparece **solo** si el material devuelve $\sigma = 0$ en compresión.

7.5.2 Formulación

```
d = -[ c_, -s_, c_, s_]
n = -[ s_, c_, s_, -c_]

K_M = (E_t · A / L) · d · d      (0 si material destensado)
K_G = (N / l) · n · n           (0 si N = 0)
K_T = K_M + K_G
F_int = N · d                   (0 si N = 0)
```

7.5.3 Cargas distribuidas

- `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo en cada componente global, evaluado en geometría de referencia. Útil para peso propio.

7.5.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el plano.
- Cables completamente destensados aportan $K_T = 0$ al sistema global: otros elementos deben garantizar la estabilidad numérica.

7.5.5 Independencia del diseño

El elemento **no hereda** de `Truss2DCorot`. La cinemática corotacional se implementa dentro de la clase para que futuras modificaciones de las armaduras no afecten a los cables, y viceversa.

7.5.6 Validación

- Tests: `tests/test_cable_elements.py · TestCable2DCorot` (4 tests de aceptación: tensado, destensado, rotación rígida, cruce por cero).
- Archivo fuente: `solidum/elements/cable.py · clase Cable2DCorot`.
- Spec: `docs/specs/Cable2DCorot.md`.
- Referencias: Crisfield §3.3; Irvine §1.2.

7.6 Cable3DCorot — cable 3D corotacional

Elemento 1D inmerso en el espacio que modela un cable: dos nodos, transmite solo tensión, puede rotar libremente en el espacio. Cinemática corotacional 3D; unilateralidad aportada por el material (típicamente `CableMaterial1D`).

7.6.1 Cinemática

- Idéntica en filosofía a una barra corotacional 3D: longitud y cosenos directores se recalculan en configuración corriente.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.
- La dirección perpendicular al eje es un **plano** (dimensión 2).

7.6.2 Formulación

```
d = -[ c, -c_y, -c_z,  c,  c_y,  c_z]
ê = ( c,  c_y,  c_z),  P = I - ê·ê      (proyector 3×3)

K_M   = (E_t · A / L) · d · d          (0 si material destensado)
K_G   = (N / l) · [[P, -P], -[P, P]]   (0 si N = 0)
K_T   = K_M + K_G
F_int = N · d                          (0 si N = 0)
```

7.6.3 Cargas distribuidas

- `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo en cada componente global, evaluado en geometría de referencia.

7.6.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el espacio.
- Cables completamente destensados aportan $K_T = 0$ al sistema global.

7.6.5 Independencia del diseño

No hereda de `Cable2DCorot` ni de `Truss3DCorot`. La maquinaria cinemática 3D se implementa íntegra dentro de la clase.

7.6.6 Validación

- Tests: `tests/test_cable_elements.py` · `TestCable3DCorot` (4 tests de aceptación).
- Archivo fuente: `solidum/elements/cable.py` · clase `Cable3DCorot`.
- Spec: `docs/specs/Cable3DCorot.md`.
- Referencias: Crisfield §3.3; Belytschko §4.5; Irvine §1.2.

7.7 Frame2DEuler — marco/viga 2D Euler-Bernoulli

Viga esbelta 2D con hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje neutro deformado. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector.

7.7.1 Cinemática

- Interpolación lineal del desplazamiento axial, Hermite cúbica del desplazamiento transverso.
- Pequeños desplazamientos y rotaciones (régimen geoméricamente lineal).
- Configuración inicial fija: L_0 , $\mathbf{c}$, $\mathbf{s}$, $\mathbf{T}$ se calculan una vez y no se actualizan.

7.7.2 Formulación

Matriz de transformación global→local $\mathbf{T}$ (6×6) con cosenos directores del eje. Matriz local analítica:

```
K_local = [[ EA/L,      0,      0, -EA/L,      0,      0 ],
            [  0, 12EI/L3, 6EI/L2,  0, -12EI/L3, 6EI/L2 ],
            [  0,  6EI/L2,  4EI/L,   0, -6EI/L2,  2EI/L ],
            [-EA/L,      0,      0,  EA/L,      0,      0 ],
            [  0, -12EI/L3, -6EI/L2,  0, 12EI/L3, -6EI/L2 ],
            [  0,  6EI/L2,  2EI/L,   0, -6EI/L2,  4EI/L ] ]
K_global = T · K_local · T
F_int     = T · F_int_local      (componente axial corregida con ·A del material)
```

7.7.3 Cargas distribuidas

- `compute_body_load(b)` distribuye $\mathbf{q} = \mathbf{A} \cdot \mathbf{b}$ (carga por unidad de longitud) con la fórmula consistente Hermite cúbica: axial mitad-mitad por nodo, transversal $\mathbf{q} \cdot L/2$ en fuerza y $\pm \mathbf{q} \cdot L^2/12$ en momento (signo positivo en nodo i, negativo en j). Transformación local $\leftrightarrow$ global vía la misma $\mathbf{T}$ del elemento. Útil para peso propio.

7.7.4 Régimen de validez

- Vigas esbeltas ($L/h \gtrsim 10$); peraltadas $\rightarrow$ `Frame2DTimoshenko`.
- Pequeños desplazamientos, pequeñas rotaciones.
- No captura **plasticidad por flexión** distribuida en la sección: `E_tangent` escala toda la matriz de rigidez por igual. La fluencia que ocurre primero en la fibra inferior bajo flexión no se modela hasta que entre `FiberSection` (deuda técnica documentada).

7.7.5 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a `Frame2DTimoshenko` y `Frame2DEulerCorot`, sin herencia entre ellas. La construcción de longitud, cosenos directores y matriz de transformación 6×6 se delega a `build_geometry_2d` en `solidum/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye $\mathbf{T}$ desde `alpha0` por su lógica propia).

7.7.6 Validación

- Tests: `tests/test_frame.py` · `TestFrame2DEulerAcceptance` (3 tests de aceptación + registro, incluye flecha analítica del voladizo $PL^3/(3EI)$).
- Archivo fuente: `solidum/elements/frame/euler.py` · clase `Frame2DEuler`.
- Spec: `docs/specs/Frame2DEuler.md`.
- Referencias: Bathe cap. 5; Cook §2.7.

7.8 Frame2DTimoshenko — marco/viga 2D Timoshenko

Viga 2D con deformación por cortante transversal explícita. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector. Apropiado para vigas peraltadas o cortas ($L/h \lesssim 10$).

7.8.1 Cinemática

- Secciones planas, **no** perpendiculares al eje neutro deformado (rotación independiente de la pendiente de la elástica).
- Distorsión angular $\gamma = dv/ds - \theta$ tratada como campo independiente.
- Configuración inicial fija (régimen geoméricamente lineal).

7.8.2 Formulación

Factor de corrección contra shear locking:

$$\Phi = 12 \cdot E \cdot I / (G \cdot A_s \cdot L^2), \quad G = E / [2(1 + \nu)]$$

Matriz local con coeficientes ajustados:

$$\begin{aligned} a &= 12 \cdot EI / [L^3 \Phi(1+)], & b &= 6 \cdot EI / [L^2 \Phi(1+)] \\ c &= \Phi(4+) \cdot EI / [L \Phi(1+)], & d &= \Phi(2-) \cdot EI / [L \Phi(1+)] \end{aligned}$$

$$K_{\text{local}} = \begin{bmatrix} EA/L, & 0, & 0, & -EA/L, & 0, & 0, \\ 0, & a, & b, & 0, & -a, & b, \\ 0, & b, & c, & 0, & -b, & d, \\ -EA/L, & 0, & 0, & EA/L, & 0, & 0, \\ 0, & -a, & -b, & 0, & a, & -b, \\ 0, & b, & d, & 0, & -b, & c \end{bmatrix}$$

Para $\Phi \rightarrow 0$ (viga esbelta) se recupera la matriz Euler-Bernoulli.

7.8.3 Parámetros

- A, I, A_s (área efectiva de cortante).
- `nu` (opcional): se toma de `material.nu` si está disponible; en su defecto, del parámetro del elemento con default 0.3 y aviso por consola.

7.8.4 Cargas distribuidas

- `compute_body_load(b)` usa la misma fórmula consistente Hermite cúbica que `Frame2DEuler` (idéntica para carga uniforme — la diferencia entre formulaciones aparece solo con cargas no uniformes o concentradas).

7.8.5 Régimen de validez

- Vigas gruesas/peraltadas ($L/h \lesssim 10$); para esbeltas $\rightarrow$ `Frame2DEuler` (más simple y sin shear locking).
- Pequeños desplazamientos, pequeñas rotaciones.
- No captura plasticidad distribuida: `E_tangent` escala toda la matriz.

7.8.6 Independencia del diseño

Vive en el paquete `solidum/elements/frame/`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `solidum/elements/frame/_shared.py` — la construcción de la matriz de transformación 6×6 ya no se duplica.

7.8.7 Validación

- Tests: `tests/test_frame.py` · `TestFrame2DTimoshenkoAcceptance` (convergencia a Euler en viga esbelta, axial puro, simetría).
- Archivo fuente: `solidum/elements/frame/timoshenko.py` · clase `Frame2DTimoshenko`.

- Spec: docs/specs/Frame2DTimoshenko.md.
- Referencias: Reddy cap. 5; Bathe §5.4.

7.9 Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional

Viga 2D esbelta Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Grandes desplazamientos y grandes rotaciones rígidas del elemento; deformaciones axiales pequeñas y rotaciones nodales deformacionales moderadas.

7.9.1 Cinemática corotacional

- Rotación rígida α_e del eje separada de las rotaciones deformacionales de las secciones $\bar{\theta}_1, \bar{\theta}_2$.
- DOFs deformacionales locales: $[u_l = l - L_0, \bar{\theta}_1, \bar{\theta}_2]$.
- Todo en configuración corriente: c, s, l, α se recalculan por evaluación.

7.9.2 Formulación

```
Rigidez local (3×3) en DOFs deformacionales:
K_ll = diag-block([EA/L], [[4EI/L, 2EI/L], [2EI/L, 4EI/L]])

Matriz B (3×6) en configuración corriente acopla DOFs locales y globales.

Rigidez tangente:
K_T = B · K_ll · B + K_
K_ = (N/l) · z · z - ((M+M)/l²) · (r · z + z · r)

con r = -[c-, s, 0, c, s, 0], z = -[s, c, 0, s-, c, 0].
```

7.9.3 Cargas distribuidas

- `compute_body_load(b)` evalúa la integral consistente en la configuración de referencia (misma fórmula Hermite cúbica que **Frame2DEuler**). Para grandes rotaciones con cargas conservadoras (gravedad) la diferencia frente a la integración sobre la geometría corriente es de segundo orden y se asume aceptable.

7.9.4 Régimen de validez

- $|\varepsilon_{\text{axial}}| \lesssim 10^{-2}$.
- Rotaciones rígidas del elemento ilimitadas entre commits (hasta $|\alpha_e| < \pi$ por paso).
- Rotaciones deformacionales de las secciones moderadas ($|\theta| \lesssim 30^\circ$).
- Rotaciones continuas $> \pi$ requerirían tracking de vueltas (fuera de alcance).

7.9.5 Dominios que abre

Pandeo por flexión de columnas esbeltas, post-pandeo, snap-through de arcos, brazos flexibles, cables con rigidez a flexión. Problemas que la viga lineal **Frame2DEuler** no puede atacar.

7.9.6 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a las otras vigas 2D, **sin herencia**: la cinemática corotacional se implementa íntegra dentro de la clase (reconstruye T desde `alpha0` en lugar de usar `build_geometry_2d` compartido). Sí comparte con `Frame2DEuler` y `Frame2DTimoshenko` los helpers libres de `_shared.py` para carga de cuerpo (`_frame2d_consistent_body_load`), masa (`_frame2d_consistent_mass_local`) y traducción a `ElementForces` (`_frame2d_forces_from_local`).

7.9.7 Validación

- Tests: `tests/test_frame.py` · `TestFrame2DEulerCorotAcceptance` (4 criterios físicos + **chequeo por diferencias finitas de K_T contra F_{int}** + registro).
- Archivo fuente: `solidum/elements/frame/euler_corot.py` · clase `Frame2DEulerCorot`.
- Spec: `docs/specs/Frame2DEulerCorot.md`.
- Referencias: Crisfield §7.3; Belytschko §4.11; Wriggers cap. 4.

7.10 Frame3D — marco/viga 3D Euler-Bernoulli

Viga 3D esbelta con 6 DOFs por nodo (`ux`, `uy`, `uz`, `rx`, `ry`, `rz`). Transmite axial + cortante en dos planos + flexión en dos planos + torsión. Régimen geoméricamente lineal.

7.10.1 Cinemática

- Secciones planas perpendiculares al eje neutro deformado (Euler-Bernoulli).
- Torsión de Saint-Venant pura (sin alabeo).
- Flexiones en ejes principales desacopladas.
- Configuración inicial fija (no hay variante corotacional 3D por ahora).

7.10.2 Ejes locales y orientación de la sección

El eje $\hat{x}$ local va del nodo 1 al nodo 2. Los ejes $\hat{y}, \hat{z}$ de la sección se definen proyectando un **vector de referencia** (`ref_vector`) sobre el plano perpendicular al eje. Default: `[0, 0, 1]` (eje z global); fallback a `[1, 0, 0]` con advertencia si la barra es casi vertical.

7.10.3 Formulación

Matriz de transformación T 12×12 en bloques de λ (3×3). Matriz local 12×12 desacoplada:

```
Axial      (2×2) : EA/L
Torsión    (2×2) : GJ/L      con G = E / [2(1+)]
Flexión xy (4×4) : 12EIz/L3, 6EIz/L2, 4EIz/L, 2EIz/L
Flexión xz (4×4) : 12EIy/L3, 6EIy/L2, 4EIy/L, 2EIy/L (signos invertidos)

K_global = T K_local T
```

7.10.4 Parámetros

- A , I_y , I_z (área y momentos de inercia respecto a ejes locales y , z).
- J (constante torsional de Saint-Venant).
- ν (opcional, del material si lo expone, sino del elemento con default 0.3).
- `ref_vector` (opcional, default $[0, 0, 1]$).

7.10.5 Cargas distribuidas

- `compute_body_load(b)` proyecta $\mathbf{q} = \mathbf{A} \cdot \mathbf{b}$ sobre los ejes locales y reparte con la fórmula Hermite cúbica en ambos planos de flexión: $\mathbf{q} \cdot L / 2$ en fuerzas, $\pm \mathbf{q} \cdot L^2 / 12$ en momentos (con signos + en nodo i y - en nodo j para M_z , y signos opuestos para M_y por la regla de la mano derecha). No genera torsión (carga uniforme sin excentricidad respecto al centro de cortadura). Útil para peso propio: $\mathbf{b} = (0, 0, -\rho \cdot g)$.

7.10.6 Régimen de validez

- Vigas esbeltas ($L/h \gtrsim 10$).
- Pequeños desplazamientos, pequeñas rotaciones.
- Sin alabeo (secciones macizas o cerradas).
- Para grandes rotaciones en 3D: pendiente `Frame3DCorot`.

7.10.7 Masa lumped: limitación en orientación oblicua

La masa lumped es estrictamente diagonal en ejes locales ($\rho A L / 2$ traslacional, $\rho I \cdot L / 2$ rotacional con I específica por DOF). Tras la rotación $\mathbf{T}^T \cdot \mathbf{T} \cdot \mathbf{M} \cdot \mathbf{T}$ a globales, el bloque traslacional 3×3 por nodo permanece diagonal porque $\mathbf{m}_t \cdot \mathbf{I}_3$ es invariante bajo $SO(3)$, pero el bloque rotacional 3×3 queda **lleno** porque $\mathbf{m}_{rx} = \rho J_p \cdot L / 2$, $\mathbf{m}_{ry} = \rho I_y \cdot L / 2$, $\mathbf{m}_{rz} = \rho I_z \cdot L / 2$ son valores distintos para una sección real. `M_global` resulta entonces **bloque-diagonal por nodo** (no estrictamente diagonal). Es la limitación estándar del lumping en frames 3D (Cook-Malkus-Plesha §11.4); aceptable para Newmark/HHT pero `CentralDifferenceSolver` rechaza con `ValueError` y orienta al usuario hacia Newmark o hacia un eje del elemento alineado con un eje global.

7.10.8 Independencia del diseño

Archivo propio. No hereda ni comparte helpers con `Frame2DEuler`, `Frame2DTimoshenko`, ni con los trusses 3D.

7.10.9 Validación

- Tests: `tests/test_frame3d.py` · `TestFrame3DAcceptance` (5 criterios físicos + validación de `ref_vector` + registro).
- Archivo fuente: `solidum/elements/frame3d.py` · clase `Frame3D`.
- Spec: `docs/specs/Frame3D.md`.
- Referencias: Przemieniecki Tabla 11.3; Cook §2.8; Bathe cap. 5.

7.11 Quad4 — cuadrilátero bilineal 2D

- **Propósito:** continuo 2D isoparamétrico; problema mecánico plano.
- **DOFs por nodo:** ['ux', 'uy'] · 4 nodos (antihorario) · STRAIN_DIM = 3.
- **Cinemática:** matriz $B(\xi, \eta)$ calculada por mapeo isoparamétrico estándar; deformación Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$.
- **Integración:** por defecto Gauss 2×2 (4 puntos). Configurable vía `quadrature` desde `QuadratureRegistry`. **Aviso:** integración reducida 1×1 → riesgo de hourglassing.
- **Parámetros:** `thickness` (espesor para estado plano), `quadrature` (opcional).
- **Cargas distribuidas:** `compute_body_load(b)` integra $\int N^T \mathbf{b} dV$ sobre el elemento; `compute_edge_traction( $\bar{\mathbf{t}}$ )` reparte una tracción uniforme sobre un borde (índices 0..3 según conectividad nodal). Tracción siempre en globales; sin soporte aún para tracción variable o presión normal.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve σ, ε y coordenadas (naturales y globales) en cada punto de Gauss. Habilita el suavizado nodal del exporter VTK (`Sigma*_nodal`).
- **Implementación:** kernels `_compute_kinematics` y `_compute_integrands` con `@njit` (Numba) — JIT en primera llamada. Viven en `solidum/elements/solid_2d/_shared.py`, compartidos con Tri3.
- **Limitaciones:** bloqueo volumétrico con materiales casi-incompresibles ($\nu \rightarrow 0.5$); en ese régimen usar formulación mixta (no implementada).
- **Archivo:** `solidum/elements/solid_2d/quad4.py`

7.12 Tri3 — triángulo lineal 2D (CST)

- **Propósito:** triángulo de deformación constante; útil para transiciones de malla.
- **DOFs por nodo:** ['ux', 'uy'] · 3 nodos · STRAIN_DIM = 3.
- **Cinemática:** matriz B constante (deformación uniforme dentro del elemento).
- **Integración:** 1 punto central, peso 0.5 (área triángulo en coordenadas naturales).
- **Parámetros:** `thickness`.
- **Cargas distribuidas:** `compute_body_load(b)` reparte 1/3 a cada nodo (exacto para b uniforme); `compute_edge_traction(edge,  $\bar{\mathbf{t}}$ )` reparte $L/2$ a cada uno de los 2 nodos del borde (índices 0..2). Tracción en globales.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve el único σ, ε y centroide del elemento (mismo formato que Quad4).
- **Limitaciones:** shear locking severo; convergencia lenta. **Preferir Quad4** salvo en transiciones donde Quad4 no encaja geométricamente.
- **Implementación:** kernel `_compute_kinematics_tri3` con `@njit` en `solidum/elements/solid_2d/_shared`
- **Archivo:** `solidum/elements/solid_2d/tri3.py`

7.13 Quad8 — cuadrilátero serendípito 2D de orden 2

- **Propósito:** continuo 2D cuadrático sin shear locking severo en flexión; reproduce campos cuadráticos exactamente.
- **DOFs por nodo:** ['ux', 'uy'] · 8 nodos (4 vértices antihorarios + 4 medios de borde) · STRAIN_DIM = 3.
- **Funciones de forma:** serendípitas (sin término $\xi^2\eta^2$).
- **Integración:** Gauss 3×3 (9 puntos) por defecto. Reducida 2×2 disponible vía `quadrature` (con riesgo de modos espurios).
- **Cargas:** body load por cuadratura; tracción de borde uniforme reparte 1/6, 4/6, 1/6 (vértice, medio, vértice).
- **Salida por Gauss:** `compute_gauss_state(U)` con 9 puntos por defecto.
- **Spec:** docs/specs/Quad8.md.
- **Archivo:** solidum/elements/solid_2d/quad8.py (subclase de la base interna `_HigherOrderSolid2D` definida en `_shared.py`, compartida con Quad9 y Tri6).

7.14 Quad9 — cuadrilátero Lagrangiano 2D de orden 2

- **Propósito:** continuo 2D cuadrático con espacio polinómico completo Q_2 ; añade un noveno nodo central interior al Quad8.
- **DOFs por nodo:** ['ux', 'uy'] · 9 nodos · STRAIN_DIM = 3.
- **Funciones de forma:** producto tensorial Lagrange 1D-1D.
- **Integración:** Gauss 3×3.
- **Cargas y salida por Gauss:** idénticas a Quad8 (el nodo 8 es interior y no participa en bordes).
- **Spec:** docs/specs/Quad9.md.
- **Archivo:** solidum/elements/solid_2d/quad9.py (subclase de la base interna `_HigherOrderSolid2D`, compartida con Quad8 y Tri6).

7.15 Tri6 — triángulo 2D cuadrático completo P_2

- **Propósito:** triángulo isoparamétrico cuadrático que cura el shear locking del Tri3; reproduce campos cuadráticos exactamente.
- **DOFs por nodo:** ['ux', 'uy'] · 6 nodos (3 vértices + 3 medios de borde) · STRAIN_DIM = 3.
- **Integración:** 3 puntos en los puntos medios (cuadratura `tri_3`).
- **Cargas:** body load por cuadratura; tracción de borde reparte 1/6, 4/6, 1/6.

- **Spec:** docs/specs/Tri6.md.
- **Archivo:** solidum/elements/solid_2d/tri6.py (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con Quad8 y Quad9; declara `_MASS_QUADRATURE = "tri_6"` para integrar exactamente la masa consistente, que la cuadratura del elemento subintegraría).

Sólidos tridimensionales con convención Voigt 6D del proyecto (Reglas.md §5): $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$ con $\gamma_{ij} = 2 \cdot \varepsilon_{ij}$. Esta familia **no expone internal_forces** — la primitiva natural de salida en sólidos es `compute_gauss_state(U)`, que devuelve `{stress, strain, points_*}` por punto de integración (ADR 0012: cierre del contrato `internal_forces` por dominio explícito).

7.16 Hex8 — hexaedro trilineal 3D

- **Propósito:** sólido 3D isoparamétrico de primer orden, espejo natural de Quad4.
- **DOFs por nodo:** `['ux', 'uy', 'uz'] · 8 nodos (orden VTK_HEXAHEDRON) · STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 8` (default Gauss $2 \times 2 \times 2$).
- **Cinemática:** matriz $B(\xi, \eta, \zeta)$ 6×24 por mapeo isoparamétrico estándar; deformación Voigt 3D.
- **Integración:** por defecto Gauss $2 \times 2 \times 2$ (8 puntos). Configurable vía `quadrature (hex_3x3x3` para no-lineales severos, `hex_1x1x1` reducido con riesgo de hourglass).
- **Caras (ADR 0012):** 6 caras numeradas con normal saliente — 0 ($-\zeta$), 1 ($+\zeta$), 2 ($-\eta$), 3 ($+\xi$), 4 ($+\eta$), 5 ($-\xi$). Ver `Hex8.FACE_NODES`.
- **Cargas distribuidas:** `compute_body_load(b)` integra $\int N^T b \, dV$; `compute_face_traction(face,  $\bar{t}$ )` reparte tracción uniforme sobre una cara con cuadratura 2D Gauss 2×2 (4 puntos por cara). $\bar{t}$ en globales.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve `{points_natural, points_global, strain (n_g, 6), stress (n_g, 6)}`. Sin `internal_forces` (ADR 0012).
- **Implementación:** `kernels_compute_kinematics_hex8, _det_jacobian_hex8, _shape_functions_hex8` con `@njit` en `solidum/elements/solid_3d/_shared.py`. Mass lumping vía `lump_hrz` con `n_translational_dirs=3` y `_expand_scalar_mass_3d`.
- **Limitaciones declaradas:**
- **Locking volumétrico** con $\nu \rightarrow 0.5$ — sin mitigación (política idéntica al Quad4 2D). Blindado por `tests/test_volumetric_locking_3d.py`.
- **Hourglass** con integración reducida `hex_1x1x1`: 12 modos espurios; sin estabilización Flanagan-Belytschko.
- **Shear locking** con malla coarse 1 capa en sección: documentado en `tests/validation/test_macneal_beam_3d.py`.
- **Spec:** docs/specs/Hex8.md.
- **Archivo:** solidum/elements/solid_3d/hex8.py.

7.17 Tet4 — tetraedro lineal 3D (CST 3D)

- **Propósito:** sólido 3D isoparamétrico de primer orden, espejo natural de Tri3. Útil para mallas no estructuradas y transiciones.
- **DOFs por nodo:** ['ux', 'uy', 'uz'] · 4 nodos (orden VTK_TETRA, volumen positivo) · STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 1.
- **Cinemática:** B constante 6×12 (deformación uniforme dentro del elemento — CST 3D). Reproduce campos lineales exactamente.
- **Integración:** 1 punto baricéntrico con peso 1/6 (exacto por linealidad de B).
- **Caras (ADR 0012):** 4 caras triangulares numeradas con normal saliente; cara i opuesta al nodo i. Ver Tet4.FACE_NODES.
- **Cargas distribuidas:** compute_body_load(b) reparte $b \cdot V_e / 4$ a cada nodo (exacto); compute_face_traction($\bar{\mathbf{t}}$) reparte $A_{\text{cara}} \cdot \bar{\mathbf{t}} / 3$ a cada uno de los 3 nodos de la cara, cero al nodo opuesto.
- **Salida por Gauss:** compute_gauss_state(U) con 1 punto (centroide); σ y ε constantes sobre el elemento.
- **Implementación:** kernel_compute_kinematics_tet4 con @njit en _shared.py. Masa consistente analítica $M_{ij} = \rho \cdot V_e \cdot (1 + \delta_{ij}) / 20$; lumped HRZ canónico (= row-sum por simetría).
- **Limitaciones declaradas:**
 - **Shear locking severo** en flexión — peor que Hex8 por la pobreza del espacio lineal. Recomendación: usar Hex8 en mallas hexaédricas; Tet4 para transiciones de malla no estructurada o cuando geometría compleja impide hexaedros.
 - **Locking volumétrico** con $\nu \rightarrow 0.5$ — aún peor que en Hex8.
 - **Spec:** docs/specs/Tet4.md.
 - **Archivo:** solidum/elements/solid_3d/tet4.py.

Subfamilia de elementos con **DOFs enriquecidos elementales** y **condensación estática local**: el ensamblador no ve los grados de libertad del salto. Cuando se cumple un criterio de activación (Rankine en fase 1), el elemento materializa una discontinuidad interna Γ_d y enriquece su cinemática con un salto $[[u]]$ gobernado por un material cohesivo (CohesiveMaterial, ver el catálogo de materiales §"Materiales cohesivos"). La activación se evalúa en el hook Element.prepare_step(U_committed) que los solvers no lineales invocan **una vez por paso**, antes del Newton (anti-chattering, ADR 0010 §5).

7.18 CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS)

- **Propósito:** introducir fractura computacional en aproximación discreta sobre el CST padre. Fiel a Retama (2010), Caps. 2, 5, 6 y 7: cinemática KOS, condensación estática local, longitud efectiva $l_d = (A_e/h) \cdot \cos(\theta - \alpha)$.

- **DOFs por nodo:** ['ux', 'uy'] · 3 nodos · STRAIN_DIM = 3 · N_INTEGRATION_POINTS = 1.
- **DOFs enriquecidos (elementales, no globales):** $[[u]] \in \mathbb{R}^2$ en frame local (**n**, **s**) de Γ_d . Se condensan dentro de `compute_element_state` y nunca llegan al ensamblador.
- **Estado intacto:** bit-exact con Tri3 (mismo B, mismo material, misma cuadratura).
- **Estado agrietado:** Newton local sobre $[[u]]$ hasta $R^{\{[[u]]\}} = 0$; después la condensación $K_{\text{cond}} = K_{\text{dd}} - K_{\text{du}} \cdot K_{\{[[u]]\}}^{-1} \cdot K_{\text{du}}^T$ se devuelve al ensamblador. El bulk descarga elásticamente conforme $[[u]]$ crece (discrete approach); la disipación va al cohesivo en Γ_d .
- **Activación:** criterio Rankine ($\sigma_I > \sigma_{t0}$ del cohesivo) en el centroide del CST, con el estado convergido del paso anterior. **Irreversible:** una vez activada, la discontinuidad persiste aunque el siguiente paso descargue.
- **Materiales aceptados:** bulk solo **Elastic2D** en fase 1 (la discrete approach presupone bulk elástico, ADR 0010); cohesivo cualquier **CohesiveMaterial** con `JUMP_DIM = 2`.
- **Estado de la discontinuidad:** **DiscontinuityState** (en `solidum/core/discontinuity_state.py`) — paralela a **ElementState**, semántica `trial/commit`.
- **YAML:**

```
cohesive_materials:
  - {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
      K_e: 1.0e13, softening: linear}
elements:
  - {id: 1, type: CST_Embedded2D, nodes: [1, 2, 3],
      material: 1, cohesive_material: 1}
```

- **Post-procesamiento:** `compute_gauss_state(U)` añade clave 'discontinuity' con {normal, tangent, centroid, solitary_node, l_d, jump, traction, damage} cuando el elemento está agrietado.
- **Limitaciones declaradas** (`out_of_scope` en la spec): modo mixto I-II (fase G del ADR 0010), reorientación de **n** tras activación (tracking no trivial, fase F), múltiples discontinuidades por elemento, contacto unilateral en compresión, bulks no elásticos, orden superior (**Tri6_Embedded**), 3D (**Tet4_Embedded**).
- **Spec:** `docs/specs/CST_Embedded2D.md`.
- **Archivo:** `solidum/elements/solid_2d/embedded_cst.py`.

7.19 Cómo añadir un elemento nuevo

1. **Spec primero** — el usuario crea `docs/specs/<Nombre>.md` a partir de `docs/specs/_template_element.md` (especificación física + formulación + contrato YAML). Sin spec, la IA no escribe código.
2. **Scaffolding** — `/solidum-new element <Nombre>` genera archivo en `solidum/elements/`, decorador `@ElementRegistry.register` y esqueleto de test.

3. **Implementación + validación** — la IA codifica contra la spec; los tests cubren los casos de **acceptance** declarados.
4. **Catálogo** — cuando la spec pasa a **status: validated**, se añade aquí una entrada breve siguiendo el formato de arriba (la spec sigue siendo la referencia detallada).

Capítulo 8

Modelos constitutivos — catálogo

Referencia rápida de los modelos constitutivos implementados. Una entrada por material. Para detalles del algoritmo → código fuente.

>**Convenciones:** `STRAIN_DIM` = dimensión Voigt de la deformación de entrada (1 = escalar, 3 = 2D [`εxx`, `εyy`, `γxy`], 6 = 3D). `PRIMARY_STATE_VAR` = variable interna que el `VtkExporter` lleva al output.

>**Densidad (ADR 0008):** todos los materiales aceptan un parámetro opcional *density* (kg/m³ en las unidades del problema). No requerido al construir — análisis estáticos sin peso propio funcionan sin declararla. Cuando un consumidor que requiere masa la pide (`Assembler.assemble_self_weight(g)` y futuras `assemble_mass_matrix`, etc.) y la encuentra como `None`, **falla con `ValueError`** listando los materiales sin densidad. Valor 0.0 declarado explícitamente cubre el caso legítimo de material sin masa física (penalty, restricción) y emite warning informativo.

8.1 Elastic1D — elástico lineal 1D

- **Ley:** $\sigma = E \cdot \varepsilon$ (Hooke 1D).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young).
- **Variables internas:** ninguna (sin historia).
- **Tangente:** constante = E.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko.
- **Spec:** docs/specs/Elastic1D.md
- **Archivo:** solidum/materials/elastic.py

8.2 Elastic2D — elástico lineal 2D

- **Ley:** $\sigma = C \cdot \varepsilon$, con C el tensor constitutivo isótropo en notación Voigt [`xx`, `yy`, `xy`].
- **STRAIN_DIM:** 3.
- **Parámetros:** E, nu, hypothesis ∈ {'plane_stress', 'plane_strain'}.

- **Variables internas:** ninguna.
- **Tangente:** constante = $\mathbf{C}$.
- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9 (todos los sólidos 2D con STRAIN_DIM = 3).
- **Spec:** docs/specs/Elastic2D.md
- **Archivo:** solidum/materials/elastic_2d.py

8.3 Elastic3D — elástico lineal isótropo 3D

- **Ley:** $\sigma = \mathbf{C} \cdot \varepsilon$, con $\mathbf{C}$ el tensor constitutivo isótropo 6×6 en notación Voigt 3D del proyecto [xx, yy, zz, xy, yz, xz] (ADR 0012, Reglas.md §5).
- **STRAIN_DIM:** 6.
- **Parámetros:** E (>0), nu $\in (-1, 0.5)$, density (opcional, ADR 0008).
- **Variables internas:** ninguna.
- **Tangente:** constante = $\mathbf{C}$.
- **Hipótesis:** no aplican plane_stress/plane_strain en 3D — toda la deformación es activa.
- **Compatible con:** Hex8, Tet4 (todos los sólidos 3D con STRAIN_DIM = 6).
- **Caveat de compatibilidad:** ABAQUS usa orden Voigt [11, 22, 33, 12, 13, 23] (permutación yz $\leftrightarrow$ xz respecto al proyecto). Importar datos de ABAQUS requiere intercambiar los componentes 5 $\leftrightarrow$ 6 una vez en el preprocesador.
- **Spec:** docs/specs/Elastic3D.md
- **Archivo:** solidum/materials/elastic_3d.py

8.4 IsotropicDamage1D — daño isótropo 1D con softening exponencial

- **Modelo:** daño escalar $d \in [0, \text{DAMAGE_MAX}]$, esfuerzo $\sigma = (1 - d) \cdot E \cdot \varepsilon$. Versión 1D de IsotropicDamage2D.
- **Evolución del daño** (Kuhn-Tucker):
 - Deformación equivalente: $\varepsilon_{eq} = |\varepsilon|$.
 - Variable histórica: $\kappa = \max(\kappa_{old}, \varepsilon_{eq})$.
 - Si $\kappa \leq \kappa_0 \rightarrow d = 0$. Si no: $d = 1 - (\kappa_0/\kappa) \cdot \exp(-\alpha \cdot (\kappa - \kappa_0))$, saturado en DAMAGE_MAX.
- **Tangente:** **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d) \cdot E$ en descarga, sin daño activo o al saturar.

- Fórmula consistente: $E_{\text{tan}} = -(1-d) \cdot E \cdot \alpha \cdot \kappa$ — **negativa** durante toda la fase de softening (refleja la pendiente descendente σ - ε post-pico).
- Derivación: de $(1-d) \cdot E - E \cdot \varepsilon \cdot (\partial d / \partial \kappa) \cdot \text{sign}(\varepsilon)$ con $\partial d / \partial \kappa = (1-d)(1/\kappa + \alpha)$ y $\text{sign}(\varepsilon) \cdot \varepsilon = |\varepsilon| = \kappa$ en carga.
- **STRAIN_DIM**: 1 · **PRIMARY_STATE_VAR**: 'damage' · **IS_SYMMETRIC**: True (la tangente escalar produce contribución elemental simétrica aunque pueda ser indefinida; el despachador algebraico ADR 0003 detecta no-PD y degrada Cholesky→LU automáticamente).
- **Parámetros**: E, kappa_0 (umbral elástico), alpha (velocidad de degradación), density (opcional, ADR 0008).
- **Variables internas**: kappa, damage.
- **Admisibilidad (ADR 0006)**: `admissibility_scale = E · kappa_0` — esfuerzo umbral inicial. Garantiza invariancia bajo cambio de unidades del check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$.
- **Limitaciones**: no distingue tracción/compresión en la activación del daño; control de carga global puede ser inestable tras el pico (requiere `ArcLengthSolver` para seguir la rama de softening).
- **Compatible con**: Truss2D, Truss3D.
- **Referencia**: ver docs/specs/IsotropicDamage1D.md y la hermana 2D para derivación detallada. Lemaitre & Chaboche, *Mechanics of Solid Materials*. Simó & Ju (1987, IJSS 23) marco de tangente consistente.
- **Archivo**: solidum/materials/damage_1d.py

8.5 IsotropicDamage2D — daño isótropo 2D con softening exponencial

- **Modelo**: extensión 2D de IsotropicDamage1D. Esfuerzo nominal $\sigma = (1 - d) \cdot C_e \cdot \varepsilon$; daño escalar $d \in [0, \text{DAMAGE_MAX}]$.
- **Deformación equivalente**: $\varepsilon_{\text{eq}} = \sqrt{(\varepsilon_{xx})^2 + (\varepsilon_{yy})^2 + \frac{1}{2} \gamma_{xy}^2}$ (norma simple sin distinguir tensión/compresión).
- **Evolución**: κ máxima histórica (Kuhn-Tucker $\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}})$), ley exponencial $d(\kappa) = 1 - (\kappa_0/\kappa) \cdot \exp(-\alpha(\kappa - \kappa_0))$ para $\kappa > \kappa_0$, saturado a `DAMAGE_MAX`.
- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d) \cdot C_e$ en descarga, sin daño activo ($\kappa \leq \kappa_0$) o al saturar.
- Fórmula consistente: $C_{\text{alg}} = (1-d) \cdot C_e - [(1-d) \cdot (1/\kappa + \alpha) / \varepsilon_{\text{eq}}] \cdot (C_e \cdot \varepsilon) \quad (M \cdot \varepsilon)$ con $M = \text{diag}(1, 1, 1/2)$.
- Derivación: $\partial d / \partial \kappa = (1-d) \cdot (1/\kappa + \alpha)$ por la ley exponencial; $\partial \kappa / \partial \varepsilon = M \cdot \varepsilon / \varepsilon_{\text{eq}}$ en carga activa, 0 en descarga.
- **No simétrica en general** ($(C_e \cdot \varepsilon)$ y $(M \cdot \varepsilon)$ no son proporcionales). Recupera convergencia cuadrática del Newton global frente a la secante (que daba convergencia lineal).

- **STRAIN_DIM:** 3 · **PRIMARY_STATE_VAR:** 'damage' · **IS_SYMMETRIC:** False (la tangente consistente no es simétrica; el despachador algebraico ADR 0003 elige LU en lugar de Cholesky para el sistema global).
- **Parámetros:** E, nu, kappa_0, alpha, hypothesis $\in$ {'plane_stress' (default), 'plane_strain'}, density (opcional, ADR 0008).
- **Variables internas:** kappa, damage.
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = E \cdot \kappa_0$ — idéntica al caso 1D, el criterio de daño tiene el mismo umbral característico independientemente de la dimensión.
- **Limitación:** la ε_{eq} simétrica no distingue daño en tensión vs compresión; para hormigón usar modelo de Mazars o split tensión/compresión (out-of-scope). Sin regularización por longitud característica $\rightarrow$ mesh-dependency en régimen de ablandamiento (banda localizada en una fila de elementos).
- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9.
- **Referencia:** ver docs/specs/IsotropicDamage2D.md. Simó & Ju (1987, IJSS 23) tangente consistente para daño isótropo. Lemaitre & Chaboche (1990) marco de continuum damage mechanics.
- **Archivo:** solidum/materials/damage_2d.py

8.6 Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal

- **Modelo:** plasticidad asociativa con criterio $f = |\sigma_{trial}| - (\sigma_y + H \cdot \alpha) \leq 0$.
- **Algoritmo:** return mapping clásico 1D.
- **Predictor elástico:** $\sigma_{trial} = E \cdot (\varepsilon - \varepsilon_{p_old})$.
- **Corrector:** $\Delta\gamma = f / (E + H)$, $\sigma = \sigma_{trial} - \Delta\gamma \cdot E \cdot \text{sign}(\sigma_{trial})$.
- **Tangente algorítmica consistente:** $E_t = E \cdot H / (E + H)$.
- **STRAIN_DIM:** 1 · **PRIMARY_STATE_VAR:** 'alpha'.
- **Parámetros:** E, sigma_y (fluencia inicial), H (módulo de endurecimiento; 0 = perfectamente plástico).
- **Variables internas:** eps_p (deformación plástica), alpha (acumulada equivalente).
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = \sigma_y + H \cdot \alpha$ — fluencia corriente (adaptativa al endurecimiento). El check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$ se hace contra la superficie de fluencia *en el estado de entrada del paso*.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko (en estos últimos solo se aplica al esfuerzo axial σ).
- **Referencia:** Simo & Hughes, *Computational Inelasticity*, cap. 1.

- **Spec:** docs/specs/Elastoplastic1D.md
- **Archivo:** solidum/materials/plastic_1d.py

8.7 CableMaterial1D — cable axial 1D con elasticidad unilateral

- **Ley:** $\sigma = E \cdot \varepsilon$ si $\varepsilon > 0$; $\sigma = 0$ si $\varepsilon \leq 0$ (sin rigidez en compresión).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young en tensión, estrictamente positivo).
- **Variables internas:** ninguna (respuesta memoryless: la rigidez depende solo del signo instantáneo de ε).
- **Tangente:** $E_t = E$ en tensión, $E_t = 0$ en compresión o sin tensión.
- **Implicación numérica:** la rigidez tangente colapsa a cero cuando el cable se afloja, lo que requiere preconditionamiento o regularización en el solver para no degenerar la matriz global. El uso típico es a través del elemento Cable2DCorot/Cable3DCorot, que detecta la situación y la maneja correctamente.
- **Compatible con:** Cable2DCorot, Cable3DCorot.
- **Referencia:** ver docs/specs/CableMaterial1D.md.
- **Archivo:** solidum/materials/cable_1d.py

8.8 DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain

- **Modelo:** cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio $f = \sqrt{J_2} + \eta_f \cdot I_1 - k(\alpha) \leq 0$ con $k(\alpha) = k_0 + H \cdot \alpha$ (endurecimiento isótropo lineal en cohesión).
- **Hipótesis:** solo plane_strain en esta entrega. plane_stress declarado *out-of-scope* (proyección con $\sigma_{zz}=0$ acoplada a flujo dilatante es notoriamente delicada).
- **Plasticidad no asociada por defecto:** ángulo de dilatación ψ parámetro independiente, default $\psi = \varphi$ (asociada). Para $\psi \neq \varphi$ la tangente algorítmica es **asimétrica** $\rightarrow$ IS_SYMMETRIC = False.
- **Calibración con Mohr-Coulomb (variant):**
- **plane_strain_matched** (default): $\eta_f = \tan(\varphi)/\sqrt{(9 + 12\tan^2(\varphi))}$, $k_0 = 3c_0/\sqrt{(9 + 12\tan^2(\varphi))}$. Coincide exactamente con MC en plane strain.
- **outer_cone:** $\eta_f = 2 \cdot \sin(\varphi)/[\sqrt{3} \cdot (3 - \sin(\varphi))]$. Circumscribe MC.
- **inner_cone:** $\eta_f = 2 \cdot \sin(\varphi)/[\sqrt{3} \cdot (3 + \sin(\varphi))]$. Inscribe MC.
- **Algoritmo — dos ramas cerradas con detección automática:**

- **Return regular** (cone surface): $\Delta\gamma = f_trial / (G + 9K\cdot\eta_f\cdot\eta_g + H)$, dirección desviadora preservada. Tangente algorítmica $C_alg = K\cdot v\ v + 2G\cdot(1-\beta)\cdot I_dev + 4G\cdot\beta\cdot\hat{n}\ \hat{n} - (1/A)\cdot b_g\ b_f$ con $\beta = G\cdot\Delta\gamma/\sqrt{J2_trial}$, $b_g = 2G\cdot\hat{n} + 3K\cdot\eta_g\cdot v$, $b_f = 2G\cdot\hat{n} + 3K\cdot\eta_f\cdot v$.
- **Return al ápice** (vértice del cono, zona puramente hidrostática): $\Delta\gamma_apex = (I1_trial\cdot\eta_f - k(\alpha))/(9K\cdot\eta_f\cdot\eta_g + H)$, $\sigma_n+1 = (k/3\eta_f)\cdot I$ puramente hidrostático. Tangente reducida a $K\cdot H/(9K\cdot\eta_f\cdot\eta_g + H) \cdot v\ v$ (simétrica, sin rigidez desviadora).
- Detección regular $\leftrightarrow$ apex: si tras $\Delta\gamma_regular$ resulta $\sqrt{J2_new} < 0 \rightarrow$ cae en apex; cambia a la fórmula apex.
- **STRAIN_DIM**: 3 · **PRIMARY_STATE_VAR**: 'alpha' · **IS_SYMMETRIC**: False (declarativo, conservador; el despachador algebraico ADR 0003 elige LU).
- **Parámetros** (físicos):
 - E, nu: elásticos.
 - cohesion: cohesión inicial c_0 (esfuerzo).
 - phi_deg: ángulo de fricción interna (grados, $0 \leq \varphi < 90$).
 - psi_deg: ángulo de dilatación (grados, $0 \leq \psi \leq \varphi$; default $\varphi \Rightarrow$ asociada).
 - H (≥ 0 , default 0): endurecimiento isótropo lineal en cohesión.
 - variant: calibración con MC (default 'plane_strain_matched').
 - density (opcional, ADR 0008).
- **Variables internas**: eps_p (tensorial 4 componentes [xx, yy, zz, xy_tensorial]), alpha (multiplicador plástico acumulado, adimensional).
- **Admisibilidad (ADR 0006)**: $admissibility_scale = k(\alpha) = k_0 + H\cdot\alpha$ (cohesión efectiva corriente, unidades de esfuerzo igual que f).
- **Limitaciones** (out-of-scope):
 - Sin plane_stress, endurecimiento por fricción/dilatancia, endurecimiento cinemático, cap (modelo cap-cone), ablandamiento ($H < 0$ requiere regularización), grandes deformaciones.
 - Validación inicial cubierta por tests unitarios (incluyendo FD numérica de la tangente) y un benchmark de pipeline Quad4 (rama elástica + rama apex + caso asociado). El régimen rama regular pura.^{en} geometría confinada es difícil de calibrar sin caer en apex; se cubre por los tests unitarios de cortante puro.
- **Compatible con**: Quad4, Tri3, Tri6, Quad8, Quad9 (todos con material 2D plane strain).
- **Referencia**: ver docs/specs/DruckerPrager2D.md. Drucker & Prager (1952). de Souza Neto, Perić & Owen (2008) cap. 8 (Drucker-Prager, tangentes algorítmicas).
- **Archivo**: solidum/materials/drucker_prager_2d.py

8.9 VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal

- **Modelo:** J2 (Von Mises) con regla de flujo asociada y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes (selección por kwarg `hypothesis` al construir): `plane_strain` ($\varepsilon_{zz} = 0$) y `plane_stress` ($\sigma_{zz} = 0$). Cada hipótesis usa un kernel Numba especializado.
- **Algoritmo plane strain** (Simó-Hughes §3.3): return mapping radial cerrado sobre la parte desviadora 3D extendida. Predictor elástico desviador $\mathbf{s}_{\text{trial}} = 2G \cdot (\mathbf{e}_{\text{dev}} - \mathbf{e}_p)$ con presión $p = K \cdot \text{tr}(\varepsilon)$; criterio $f = \|\mathbf{s}_{\text{trial}}\| - \sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)} \leq 0$; corrector radial $\Delta\gamma = f_{\text{trial}} / (2G + \cdot H)$ con $\mathbf{N} = \mathbf{s}_{\text{trial}} / \|\mathbf{s}_{\text{trial}}\|$; actualización $\alpha_{\text{new}} = \alpha + \sqrt{(2/3) \cdot \Delta\gamma}$. Tangente algorítmica consistente cerrada.
- **Algoritmo plane stress** (Simó-Hughes §3.4.1, "plane stress projected algorithm"): predictor $\sigma_{\text{trial}} = C_{\text{e_ps}} \cdot (\varepsilon - \mathbf{e}_p)$ en subespacio plane stress; función de fluencia proyectada $\bar{f} = \frac{1}{2} \cdot \sigma \cdot P \cdot \sigma - R^2/3$ con operador $P = (1/3) \cdot [[2, -1, 0], [-1, 2, 0], [0, 0, 6]]$. Newton local escalar sobre $\Delta\gamma$ en base de autovectores ortonormales $\mathbf{v}_1 = [1, 1, 0]/\sqrt{2}$, $\mathbf{v}_2 = [1, -1, 0]/\sqrt{2}$, $\mathbf{v}_3 = [0, 0, 1]$ de $C_{\text{e_ps}} \cdot P$ con autovalores $\mu_1 = E/(3(1-\nu))$, $\mu_2 = \mu_3 = 2G$. Converge en 3-6 iteraciones (tangente cerrada). Actualización de α con la regla físicamente correcta $\alpha_{\text{new}} = \alpha + \Delta\gamma \cdot \sqrt{(2\sigma P \sigma)/3}$ (la heurística $\sqrt{(2/3) \cdot \Delta\gamma}$ válida en plane strain rompe la invariancia bajo cambio de unidades en plane stress, donde $\Delta\gamma$ tiene unidades $[1/\text{esfuerzo}]$). Incompresibilidad plástica cierra $\mathbf{e}_{p_zz} = -(\mathbf{e}_{p_xx} + \mathbf{e}_{p_yy})$. Tangente algorítmica consistente cerrada con corrección por $d\alpha/d\Delta\gamma \neq \sqrt{(2/3)}$.
- **STRAIN_DIM:** 3 · **PRIMARY_STATE_VAR:** 'alpha'.
- **Parámetros:** $E, \nu, \sigma_y, H (\geq 0, \text{default } 0), hypothesis \in \{'plane_strain' (\text{default}), 'plane_stress'\}, density$ (opcional, ADR 0008).
- **Variables internas:** `eps_p` (tensorial 4 componentes `[xx, yy, zz, xy_tensorial]`), `alpha` (acumulada equivalente, adimensional en ambas hipótesis).
- **Admisibilidad (ADR 0006):**
 - `plane_strain`: `admissibility_scale` = $\sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)}$ — radio desviador de la superficie J2.
 - `plane_stress`: `admissibility_scale` = $(\sigma_y + H \cdot \alpha)^2/3$ — escala de $\bar{f}$ proyectada.
 - En ambos, la tolerancia se precomputa fuera del kernel `@njit` y se pasa como `float`.
- **Limitaciones declaradas** (ver `out_of_scope` en spec): no endurecimiento cinemático/Voce, no ablandamiento ($H < 0$ requiere regularización), no regla de flujo no asociada, no daño acoplado, no viscoplasticidad, no grandes deformaciones, no anisotropía. Locking volumétrico en plane strain con elementos de bajo orden cuando $\nu \rightarrow 0.5$ o dominio plástico (mitigaciones B-bar/F-bar diferidas).
- **Implementación:** kernels `_compute_j2_plane_strain` y `_compute_j2_plane_stress` con `@njit`. Despacho por `hypothesis` en construcción (sin coste runtime).

- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9 (configurados en cualquiera de las dos hipótesis).
- **Referencia:** ver docs/specs/VonMises2D.md. Simó & Hughes, *Computational Inelasticity* (1998), §3.3 (plane strain), §3.4 (plane stress projected, Box 3.1). de Souza Neto, Perić & Owen, *Computational Methods for Plasticity* (2008), §9.4.
- **Archivo:** solidum/materials/von_mises_2d.py

Los materiales cohesivos son una **jerarquía paralela e independiente** de los materiales continuos: operan sobre el salto de desplazamientos $[[u]]$ sobre una superficie de discontinuidad Γ_d y devuelven tracciones t , no esfuerzos sobre ε . Viven en `solidum/cohesive_materials/`, heredan de `solidum.core.cohesive_material.CohesiveMaterial`, se registran vía `CohesiveMaterialRegistry` y se declaran en YAML bajo la sección `cohesive_materials` (separada de `materials`). El bulk del elemento sigue siendo un `Material` continuo; el cohesivo entra al sistema sólo cuando el elemento activa una discontinuidad embebida (ver ADR 0010).

Convenciones de la familia: *JUMP_DIM* = dimensión del vector de salto (2 en 2D, 3 en 3D); *PRIMARY_STATE_VAR* = variable interna que se exporta al post-proceso; *IS_SYMMETRIC* = simetría de la contribución a la tangente del elemento. Parámetros físicos típicos: *sigma_t0* (Pa), *G_f* (N/m), *K_e* (Pa/m). No tienen densidad — la inercia es del bulk.

8.10 CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial

- **Modelo:** daño escalar $\omega \in [0, 1]$ sobre el salto. Tracción $t_n = (1 - \omega) \cdot K_e \cdot [[u_n]]$ en dirección normal, $t_s = 0$ en tangencial (Modo-I puro). Activación tipo Rankine ($f = \langle [[u_n]] \rangle - \kappa$); historial monótono $\kappa_{n+1} = \max(\kappa_n, \langle [[u_n]] \rangle)$ con $\kappa_0 = \sigma_{t0}/K_e$. Energía de fractura *G_f* cierra la curva analíticamente:
- **Lineal:** $T_{\text{soft}}(\kappa) = \sigma_{t0} \cdot (w_c - \kappa) / (w_c - \kappa_0)$ con apertura crítica $w_c = 2 \cdot G_f / \sigma_{t0}$. $\omega(\kappa) = 1 - \sigma_{t0} \cdot (w_c - \kappa) / [K_e \cdot \kappa \cdot (w_c - \kappa_0)]$. $\omega = 1$ en $\kappa \geq w_c$.
- **Exponencial:** $T_{\text{soft}}(\kappa) = \sigma_{t0} \cdot \exp[-\sigma_{t0} \cdot (\kappa - \kappa_0) / H]$ con $H = G_f - \sigma_{t0} \cdot \kappa_0 / 2$. $\omega(\kappa) = 1 - T_{\text{soft}}(\kappa) / (K_e \cdot \kappa)$. Asintótica a 1.
- **Tangente:** simétrica por construcción (rank-1 sobre $n \cdot n$). Algorítmica consistente en carga activa: $T_{\text{tan}} = K_e \cdot [(1 - \omega) - [[u_n]] \cdot d\omega/d\kappa] \cdot (n \cdot n)$. Secante reducida $(1 - \omega) \cdot K_e \cdot (n \cdot n)$ en descarga y sin daño. Cap por `DAMAGE_MAX` aplicado **sólo a la rigidez tangente** cuando $\omega \geq \text{DAMAGE_MAX}$; el valor de ω reportado y la tracción usan el valor físico (puede llegar a 1.0 exacto) — esto distingue al material de `IsotropicDamage2D`, donde *K_e* permite truncar ω sin sesgo.
- **JUMP_DIM:** 2 · **PRIMARY_STATE_VAR:** 'damage' · **IS_SYMMETRIC:** True.
- **Parámetros:** *sigma_t0* (resistencia a tracción, Pa), *G_f* (energía de fractura, N/m), *K_e* (rigidez del salto / penalty, Pa/m; sin default automático, ver §12 de la spec para guía $K_e \approx 10 \cdot E_{\text{bulk}} / w_c$), *softening* $\in \{'linear', 'exponential'\}$.
- **Variables internas:** *kappa* (historial del salto equivalente, [m]), *damage* (ω).

- **Validación energética:** por construcción $\int_0^{\{w_c\}} t \cdot d[[u_n]] = G_f$ (lineal) o $\int_0^\infty \approx G_f$ (exponencial); cubierto por tests con cuadratura trapezoidal.
- **Limitaciones declaradas** (`out_of_scope` en la spec): sólo Modo-I (mixto I–II diferido a fase G del ADR 0010), sin contacto unilateral en compresión (la grieta dañada transmite compresión con rigidez $(1-\omega) \cdot K_e$, no rígida), sin anisotropía del daño, sin acoplamiento viscoso/cíclico, sin regularización para mesh-objectivity (se aborda a nivel del elemento `CST_Embedded2D`, no del material).
- **Compatible con:** pendiente de fase 2 del ADR 0010 (elemento `CST_Embedded2D`). En aislamiento, testeable con historial prescrito de `[[u]]`.
- **Referencia:** ver `docs/specs/CohesiveDamageIsotropic.md`. Retama (2010) Cap. 3; Hillerborg, Modéer & Petersson (1976); Simó & Ju (1987).
- **Archivo:** `solidum/cohesive_materials/damage_isotropic.py`

8.11 Cómo añadir un material nuevo

`/solidum-new material <Name>` — genera archivo en `solidum/materials/`, decorador `@MaterialRegistry.register` esqueleto de test. Declarar `STRAIN_DIM` y, si tiene historia, `PRIMARY_STATE_VAR` (la variable que aparecerá en el VTK). Tras implementar el modelo constitutivo, **añadir una entrada a este catálogo** siguiendo el formato de arriba.

Para un material **cohesivo** nuevo: el patrón es análogo pero el archivo vive en `solidum/cohesive_materials/`, la clase hereda de `CohesiveMaterial` (no `Material`) y se registra con `@CohesiveMaterialRegistry.register`. Declarar `JUMP_DIM`, `PRIMARY_STATE_VAR` y `IS_SYMMETRIC`. Añadir la entrada en la sección "Materiales cohesivos" de este mismo catálogo.

Capítulo 9

Solvers — catálogo

Referencia rápida de los métodos de solución implementados. Una entrada por solver. Para detalles del algoritmo → código fuente.

9.1 LinearSolver — solución lineal directa en un paso

- **Propósito:** resolver $K \cdot U = F$ en un único `spsolve`.
- **Esquema:** ensamblaje único → eliminación directa de DOFs prescritos (ADR 0004) → `scipy.sparse.linalg.spsolve`.
- **Parámetros:** `linear_algebra` (default auto; ADR 0003 §4).
- **Cuándo usarlo:** problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **No converge / no aplica:** cualquier no-linealidad material (daño, plasticidad) o geométrica (corotacional).
- **Spec:** docs/specs/LinearSolver.md
- **Archivo:** solidum/math/solvers/linear.py

9.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

- **Propósito:** resolver problemas no lineales con control de carga, dividiendo la carga total en pasos y aplicando Newton-Raphson dentro de cada paso.
- **Esquema:**
 - Bucle externo: incrementar factor de carga λ desde 0 hasta 1 en `num_steps` pasos.
 - Bucle interno: Newton-Raphson con $K_t \cdot \Delta U = R = \lambda \cdot F_{\text{ext}} - F_{\text{int}}$.
- **Criterio de convergencia dual:** `max(err_disp, err_force) < tol`, con

`err_disp =  $\|\Delta U\| / (\|U\| + \varepsilon)$` y `err_force =  $\|R\| / \max(\|\lambda \cdot F_{\text{ext}}\|, \|F_{\text{int}}\|, \varepsilon)$` .

- **Adaptatividad** (`adaptive=True`): si converge en <5 iteraciones, agranda el siguiente paso ($\times 1.5$); si no converge, biseca ($\div 2$). Falla si $\Delta\lambda < \text{min_delta_lambda}$.
- Tras converger un paso $\rightarrow \text{assembler.commit_all_states}()$ (trial $\rightarrow$ committed en todos los elementos).
- **Parámetros**: `convergence`, `max_iter`, `num_steps`, `adaptive`, `min_delta_lambda`, `linear_algebra`, `freeze_tangent_after_iter`, `line_search` (default `False`, ADR 0011).
- **Cuándo usarlo**: la opción por defecto para no-linealidad material o geométrica suave (sin snap-back).
- **Cuándo activar `line_search=True`**: cuando se observe oscilación del residuo entre iteraciones (síntoma clásico: ratios alternados sin descenso monótono). Caso documentado: plasticidad perfecta con carga cerca/sobre la capacidad. **Default `False`** porque en problemas con tangente consistente cuasi-cuadrática (daño activo, plasticidad estándar) el line search rechaza pasos correctos del Newton donde el residuo sube transitoriamente — ver ADR 0011 §.Enmiendas”.
- **Diagnóstico al diverger**: lanza una subclase tipada de `RuntimeError` (`OscillatingNewtonError`, `SingularTangentError`, `LoadExceedsCapacityError`, `UnknownDivergenceError`) con métricas (último $\|R\|$, último $\|\delta U\|$, factor de carga, bisecciones consumidas) y `hint` textual. Ver `solidum/math/solvers/diagnostics.py`.
- **Limitación**: con bisección adaptativa y cinemática no lineal (corotacional) puede atravesar puntos límite suaves; no atraviesa snap-back con $du/d\lambda < 0$. Para eso $\rightarrow$ `ArcLengthSolver`. Hallazgo de la auditoría fase A: el solver es más capaz de lo asumido históricamente (ver `docs/auditorias/solvers_robustez_fase_A.md`).
- **Referencia**: Crisfield, *Non-linear Finite Element Analysis of Solids and Structures*, vol. 1, cap. 9. Line search: Grippo-Lampariello-Lucidi 1986 (variante de descenso no monótono).
- **Spec**: `docs/specs/NonlinearSolver.md`
- **Archivo**: `solidum/math/solvers/nonlinear.py`

9.3 ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield)

- **Propósito**: trazar curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga.
- **Esquema**:
- **Predictor tangente**: $du_t = K_t^{-1} \cdot F_{ext_ref}$; $d\lambda = \text{sign} \cdot dl / \|du_t\|$.

El `sign` se elige por proyección con el incremento del paso anterior (evitar regresar).

- **Corrector iterativo**: en cada iteración resuelve dos sistemas ($du_R = K^{-1} \cdot R$ y $du_t = K^{-1} \cdot F_{ext}$) y aplica la restricción cuadrática cilíndrica de Crisfield: $\|dU\|^2 = dl^2$.
- De las dos raíces se elige la que mantiene el ángulo positivo con el incremento previo.

- **Caso especial — final_step**: si el predictor sobrepasaría `max_lambda`, fija λ exactamente a `max_lambda` y resuelve solo desplazamientos (Newton-Raphson puro).
- **Auto-ajuste de dl**: <4 iter $\rightarrow$ ampliar ($\times 1.5$, tope `5·dl_initial`); >8 iter $\rightarrow$ reducir ($\times 0.6$); no converge $\rightarrow$ biseca ($\div 2$).
- **Convergencia**: mismo criterio dual que `NonlinearSolver`.
- **Parámetros**: `tol`, `max_iter`, `max_lambda`, `initial_dl`, `max_steps`, `dl_grow_factor`, `dl_max_factor`, `dl_shrink_factor`, `dl_grow_iter_threshold`, `dl_shrink_iter_threshold`.
- **Cuándo usarlo**: problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando `NonlinearSolver` diverge cerca de un punto límite.
- **Limitación**: más caro por paso (dos `spsolve` por iteración); requiere ajuste de `initial_dl` para problemas nuevos.
- **Referencia**: Crisfield, ^A fast incremental/iterative solution procedure that handles snap-through” (Computers & Structures, 1981); Crisfield vol. 1, cap. 9.
- **Spec**: docs/specs/ArcLengthSolver.md
- **Archivo**: solidum/math/solvers/arclength.py

9.4 DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004)

- **Propósito**: trazar la rama post-pico de problemas con **softening severo** controlando la disipación incremental de energía por paso en lugar de la longitud de arco euclídea. Más robusto que el cilíndrico cuando la curva U - λ tiene tramos casi-verticales en el plano (U, λ) .
- **Esquema**:
 - Subclase de `ArcLengthSolver`; reusa predictor tangente, corrector Newton, ajuste de paso, Dirichlet/MPC y backend algebraico.
 - **Restricción de paso**: $g(\Delta U, \Delta \lambda) = \frac{1}{2} \cdot (\lambda_n \cdot F \cdot \Delta U - \Delta \lambda \cdot F \cdot U_n) = \tau$. Lineal en $(\Delta U, \Delta \lambda) \Rightarrow$ corrector con una sola raíz en $dd\lambda$, sin selección por menor ángulo ni patología de raíces imaginarias.
 - **Switching automático cilíndrico $\leftrightarrow$ disipación** según se detecte disipación neta sobre el umbral relativo `dissipation_threshold·||F_ref||·||U||`. Arranque obligado en cilíndrico ($\alpha = \frac{1}{2} \cdot (\lambda_n \cdot F \cdot du_t - F \cdot U_n)$ es 0 cuando $\lambda_n = U_n = 0$).
 - **Sign-of-pivot tracking aproximado** vía signo del `slogdet(K_t)`. Distingue 0 vs número impar de pivots negativos — diagnóstico de paso por punto límite simple. Tracking exacto requiere LDL^T Bunch-Kaufman (deuda técnica #7 STATUS.md).
 - **Salvaguarda contra final_step prematuro**: si $|d\lambda_{pred}| > 3 \cdot (\max_lambda - \lambda_{curr})$, bisecta dl o τ antes de aceptar el paso.
 - **Detección de $\alpha \approx 0$** : threshold relativo $|\alpha| < 1e-6 \cdot \text{escala}$; revierte temporalmente a cilíndrico (en problemas lineales monotónicos $\alpha = 0$ exactamente por construcción).

- **Parámetros heredados:** todos los de `ArcLengthSolver` (`max_iter`, `max_lambda`, `initial_dl`, `max_steps`, `factores_dl_*`, `linear_algebra`). **Nuevos:** `initial_tau` (obligatorio), `tau_grow_factor`, `tau_max_factor`, `tau_shrink_factor`, `tau_grow_iter_threshold`, `tau_shrink_iter_threshold`, `dissipation_threshold`.
- **Cuándo usarlo:** problemas con softening continuo donde `ArcLengthSolver` cilíndrico bisecta excesivamente cerca del pico — daño bulk 1D/2D, plasticidad con localización progresiva, post-pandeo con descarga elástica.
- **Cuándo NO usarlo (limitación documentada):** cohesivo+embedded discontinuity con `K_e` stiff ($\sim 1e13$). La activación discreta del Rankine introduce un salto en `F_int/K_t` que el `sign` por producto escalar no maneja graciosamente; el predictor del paso siguiente reorienta hacia descarga. Resolverlo requiere LDL^T Bunch-Kaufman real (item #7 deuda STATUS.md) o control por CMOD/CTOD del salto (spec separada). En problemas lineales puramente elásticos coincide con `ArcLengthSolver` cilíndrico a paridad de bits (regresión validada).
- **Referencias:** Gutiérrez (2004), *Communications in Numerical Methods in Engineering* 20, 19-29; Verhoosel, Remmers, Gutiérrez (2009), *IJNME* 77, 1290-1321; Crisfield (1991) vol. 1 §9.4.
- **Spec:** docs/specs/DissipationArcLengthSolver.md (status `implemented`, validación parcial).
- **Archivo:** solidum/math/solvers/dissipation_arclength.py

9.5 ModalSolver — análisis modal por autovalores generalizados (ADR 0009)

- **Propósito:** calcular frecuencias naturales ω_n y modos φ_n de vibración no amortiguados resolviendo $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$.
- **Esquema:**
- Ensamblaje de `K` (rigidez lineal en `u = 0`) y `M` (masa consistente, ADR 0009 §1) reusando la topología COO cacheada del `Assembler`.
- Reducción simultánea por `Dirichlet Assembler.reduce_pair(K, M) → K_red, M_red, T`.
- `EigenSolver` envuelve `scipy.sparse.linalg.eigsh` con `shift-invert` en `sigma` (`sigma=0` → frecuencias más bajas) y `which="LM"`. ARPACK factoriza $(K_{red} - \sigma \cdot M_{red})$ una sola vez y la reusa en todas las iteraciones Lanczos.
- Expansión de modos al espacio completo: $\varphi = T \cdot \varphi_{red}$. En DOFs prescritos por `Dirichlet` la componente es 0.
- Salida en `ModalResult`: `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (`M`-ortonormales), `n_modes`, `converged`.
- **Parámetros:** `n_modes` (obligatorio), `sigma` (default 0.0), `which` (default "LM"), `tolerance` (default 1.0e-9), `lumping` (default `consistent`"; fase 1 solo admite ese valor), `linear_algebra` (reservado; ADR 0003).

- **Firma del solver:** `solve()` sin argumentos (no consume `F_applied`). El entrypoint público es `solidum.run_modal(domain, n_modes=N)` o YAML con `solver: type: ModalSolver`. `solidum.run_yaml` despacha automáticamente al detectar el tipo.
- **Cuándo usarlo:** caracterización dinámica de la estructura (frecuencias naturales, formas modales) antes de pasar a análisis transitorio o sísmico. Validación de la matriz de masa contra fórmulas analíticas (barra empotrada-libre, viga Bernoulli-Euler).
- **Limitaciones:** lineal (K evaluada en $u = 0$); sin amortiguamiento; modos complejos no soportados. Cálculos derivados (factores de participación, masas efectivas, combinación espectral SRS-S/CQC) viven en `solidum/math/modal_response.py` y los consume `ResponseSpectrumSolver` (ADR 0009 fase 7 cerrada).
- **Referencia:** Bathe, *Finite Element Procedures*, cap. 9 (masa) y cap. 11 (algoritmos de autovalor); Hughes, *The Finite Element Method*, cap. 7; ARPACK Users' Guide (Lehoucq-Sorensen-Yang).
- **Spec:** `docs/specs/ModalSolver.md`
- **Archivos:** `solidum/math/solvers/modal.py` (clase `ModalSolver`), `solidum/math/linalg/eigen.py` (clase `EigenSolver`), `solidum/results.py` (`ModalResult`).

9.6 NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3)

- **Propósito:** integrar en el tiempo la ecuación de movimiento semidiscreta $M\ddot{u} + C\dot{u} + Ku = F(t)$ con amortiguamiento Rayleigh $C = \alpha M + \beta K$ y apoyos Dirichlet constantes.
- **Esquema:**
- Familia Newmark- β con (β, γ) parametrizables; default $(1/4, 1/2)$ — average acceleration, incondicionalmente estable, sin amortiguamiento numérico, error $O(\Delta t^2)$.
- **Predictores:** $\tilde{u} = u_n + \Delta t \cdot \dot{u}_n + (\Delta t^2/2)(1-2\beta) \cdot \ddot{u}_n$, $\tilde{u} = \dot{u}_n + \Delta t \cdot (1-\gamma) \cdot \ddot{u}_n$.
- **Sistema efectivo:** $A_{\text{eff}} \cdot \ddot{u}_{n+1} = F_{n+1} - C \cdot \tilde{u} - K \cdot \tilde{u}$ con $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$.
- **Correctores:** $u_{n+1} = \tilde{u} + \beta \Delta t^2 \cdot \ddot{u}_{n+1}$, $\dot{u}_{n+1} = \tilde{u} + \gamma \Delta t \cdot \ddot{u}_{n+1}$.
- Reducción por Dirichlet (operador T) aplicada a (K, M, C) simultáneamente; factorización única de $A_{\text{eff_red}}$ reusada en todos los pasos.
- Aceleración inicial consistente con la ecuación de movimiento en $t=0$.
- **Parámetros:** `t_end`, `dt` (obligatorios), `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (`None` | `{alpha, beta}` directos | `{xi1, omega1, xi2, omega2}` para calibración modal), `u0`, `u0_dot` (default ceros), `F_func` (callback Python $t \rightarrow F(t)$; default vibración libre), `linear_algebra`, `lumping`.
- **Firma del solver:** `solve()` sin argumentos, retorna `TransientResult` con historiales `t_history`, `u_history`, `udot_history`, `uddot_history` (shape $(n_{\text{dof}}, n_{\text{steps}} + 1)$).

- **Cuándo usarlo:** respuesta a cargas dependientes del tiempo (sísmica time-history, impacto, vibración forzada), evolución transitoria desde condiciones iniciales arbitrarias, validación de respuesta libre/forzada de estructuras lineales.
- **Limitaciones:**
 - Solo lineal (K , M constantes). La extensión a no-linealidad (Newton dentro de cada paso) es fase 4 del ADR 0009.
 - Apoyos Dirichlet constantes en el tiempo. Multi-support excitation sísmica diferida.
 - Paso Δt fijo. Adaptativo diferido.
 - HHT- α y generalized- α (con amortiguamiento numérico controlado) no incluidos.
- **Referencia:** Newmark (1959), J. Eng. Mech. Div. ASCE; Bathe, *FEP* cap. 9.4; Hughes, *FEM* cap. 9; Chopra, *Dynamics of Structures* cap. 5 y 15.
- **Spec:** docs/specs/NewmarkSolver.md
- **Archivos:** solidum/math/solvers/newmark.py (clase `NewmarkSolver`), solidum/math/damping.py (Rayleigh), solidum/results.py (`TransientResult`).

9.7 NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4)

- **Propósito:** integrar en el tiempo $M\ddot{u} + C\dot{u} + F_{\text{int}}(u) = F_{\text{ext}}(t)$ con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores y reducción de Dirichlet; añade Newton-Raphson dentro de cada paso temporal.
- **Esquema:**
 - Predictores Newmark idénticos a la fase 3.
 - **Bucle interno de Newton-Raphson** sobre el residuo dinámico

$R(\ddot{u}_{\{n+1\}}) = F_{\text{ext}}(t_{\{n+1\}}) - F_{\text{int}}(u_{\{n+1\}}) - C\dot{u}_{\{n+1\}} - M\ddot{u}_{\{n+1\}}$, con jacobiano $J = M + \gamma\Delta t \cdot C + \beta\Delta t^2 \cdot K_t$ y K_t la rigidez tangente corriente.

- **Convergencia dual** (ADR 0007): $R/(\text{atol} + \text{rtol} \cdot F_{\text{ref}}) \leq 1$ y $\delta u/(\text{atol} + \text{rtol} \cdot \|u\|) \leq 1$.
- **Commit de estado:** tras converger cada paso, `assembler.commit_all_states()` (trial $\rightarrow$ committed). Si no converge en `max_iter`, lanza `RuntimeError` y se descarta el state trial.
- **Amortiguamiento Rayleigh constante en el tiempo**, calibrado con la **rigidez elástica de referencia** K_0 (al inicio del análisis, $u = 0$). Convención estándar (Abaqus, ANSYS, OpenSees); evita acoplamiento ad-hoc entre disipación viscosa y plástica.
- **Newton modificado opcional** (`freeze_tangent_after_iter`, ADR 0003 fase 2): factoriza fresco las primeras N iter de cada paso y reusa la factorización en las siguientes.

- **Recuperación del caso lineal:** con materiales lineales ($K_t = K_0$), el residuo se anula en 1 iter y el resultado coincide exactamente con `NewmarkSolver`. Validado en tests.
- **Parámetros:** heredados de `NewmarkSolver` (`t_end`, `dt`, `beta`, `gamma`, `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`) más `convergence` (`ConvergenceCriterion`, ADR 0007), `max_iter` (default 20), `freeze_tangent_after_iter` (default None), `line_search` (default False, ADR 0011 — mismo criterio que `NonlinearSolver`).
- **Diagnóstico al diverger:** lanza una subclase tipada de `RuntimeError` con métricas y `hint` (mismo patrón que `NonlinearSolver`, ADR 0011).
- **Firma del solver:** `solve()` sin argumentos, retorna `TransientResult` (mismo formato que `NewmarkSolver`).
- **Cuándo usarlo:** respuesta dinámica de estructuras con plasticidad transitoria, fatiga de bajo ciclo, impacto en hormigón friccional, vibraciones de marcos con disipación plástica.
- **Limitaciones:** igual que `NewmarkSolver` en lo lineal (apoyos constantes, paso fijo). Además: no resuelve snap-back en problemas con softening severo — combinar con `ArcLengthSolver` si emerge.
- **Despacho YAML:** `solver.type: NewtonNewmarkSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por ese atributo declarativo (regla C del ADR 0009, aplicada 2026-05-18) — sin `isinstance` ni cadenas hardcoded.
- **Referencia:** ADR 0009 §Fase 4; Hughes, *FEM* cap. 7-9 (Newton dentro de paso temporal); Crisfield vol. 2 cap. 24 (dinámica no lineal).
- **Spec:** docs/specs/NewtonNewmarkSolver.md
- **Archivo:** solidum/math/solvers/newmark.py (clase `NewtonNewmarkSolver`, junto al padre).

9.8 HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α)

- **Propósito:** integrar $M\ddot{u} + C\dot{u} + Ku = F(t)$ con **amortiguamiento numérico controlado** que disipa selectivamente los modos de alta frecuencia espurios sin afectar los modos resueltos. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores, reducción Dirichlet y factorización única de A_{eff} .
- **Esquema:**
- Ecuación de equilibrio temporal: $M\ddot{u}_{\{n+1\}} + (1+\alpha)C\dot{u}_{\{n+1\}} - \alpha C\dot{u}_n + (1+\alpha)Ku_{\{n+1\}} - \alpha Ku_n = (1+\alpha)F_{\{n+1\}} - \alpha F_n$.
- **Auto-derivación de β , γ desde α** (Hilber 1977): $\beta = (1-\alpha)^2/4$, $\gamma = (1-2\alpha)/2$. Preserva orden 2 y estabilidad incondicional.
- **Sistema efectivo:** $A_{\text{eff}} = M + (1+\alpha)\gamma\Delta t C + (1+\alpha)\beta\Delta t^2 K$. Constante en el tiempo $\rightarrow$ factorización única reutilizable (como Newmark).

- **Radio espectral en alta frecuencia:** $\rho_{\infty} = (1+\alpha)/(1-\alpha)$. Con $\alpha=0 \rightarrow \rho_{\infty}=1$ (sin disipación, recupera Newmark trapezoidal). Con $\alpha=-0.05 \rightarrow \rho_{\infty}\approx 0.905$. Con $\alpha=-1/3 \rightarrow \rho_{\infty}=0.5$ (disipación máxima manteniendo orden 2).
- **Disipación selectiva:** modos con $\omega\Delta t > \pi$ se atenúan significativamente; modos con $\omega\Delta t \ll 1$ casi intactos. Es lo que distingue HHT- α de subir γ Newmark (que disipa en todas las frecuencias y baja a orden 1).
- **Parámetros:** `t_end`, `dt` obligatorios. `alpha` (default -0.05 , rango $[-1/3, 0]$). `beta`, `gamma` autoderivados desde `alpha` salvo override explícito (no recomendado fuera de experimentación). Resto heredado de `NewmarkSolver`: `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`.
- **Cuándo usarlo:** cuando la respuesta transitoria contenga modos de alta frecuencia espurios (mallas con modos altos no de interés, impacto, contacto, propagación con $\omega\Delta t$ cerca o por encima de π) que enmascaren la señal. Si quieres exactamente Newmark trapezoidal sin disipación, usa `NewmarkSolver`.
- **Default de alpha:** -0.05 es el valor canónico (Hilber 1977; Abaqus, OpenSees, ANSYS): disipación $>9\%$ por ciclo en altas frecuencias, $<0.5\%$ en modos de interés. Si pides `HHTSolver` con `alpha=0.0`, es funcionalmente idéntico a `NewmarkSolver(beta=0.25, gamma=0.5)`.
- **Despacho YAML:** `solver.type: HHTSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por el atributo declarativo (regla C ADR 0009).
- **Referencia:** Hilber, Hughes & Taylor (1977), *Earthquake Eng. Struct. Dyn.* 5(3) 283-292; Hughes *FEM* §9.3; Chopra *Dynamics of Structures* §15.4.
- **Spec:** docs/specs/HHTSolver.md
- **Archivo:** solidum/math/solvers/newmark.py (clase `HHTSolver`).

9.9 NewtonHHTSolver — análisis dinámico transitorio no lineal HHT- α

- **Propósito:** combinar HHT- α (disipación numérica) con Newton-Raphson interno para problemas con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewtonNewmarkSolver` (Reglas §4) que aplica el esquema temporal HHT- α al residuo dinámico no lineal.
- **Esquema:**
 - Residuo dinámico: $R(\ddot{u}_{n+1}) = (1+\alpha) \cdot F_{n+1} - \alpha \cdot F_n - [(1+\alpha) \cdot F_{\text{int}}(u_{n+1}) - \alpha \cdot F_{\text{int}}(u_n)] - [(1+\alpha) \cdot C \cdot \dot{u}_{n+1} - \alpha \cdot C \cdot \dot{u}_n] - M \cdot \ddot{u}_{n+1}$.
 - Jacobiano: $J = M + (1+\alpha) \cdot \gamma \Delta t \cdot C + (1+\alpha) \cdot \beta \Delta t^2 \cdot K_t$.
- Todo lo demás idéntico a `NewtonNewmarkSolver`: convergencia dual (ADR 0007), commit/-rollback, Rayleigh con `K_0`, Newton modificado opcional, `line_search` opt-in (ADR 0011), telemetría tipada al diverger.
- **Recuperación:** con materiales lineales, el residuo de Newton se anula en una iteración y el resultado coincide con `HHTSolver`. Validado en tests.

- **Parámetros:** `alpha` (default `-0.05`) + heredados de `NewtonNewmarkSolver` (`convergence`, `max_iter`, `freeze_tangent_after_iter`, `line_search`, ...).
- **Cuándo usarlo:** respuesta dinámica de estructuras con plasticidad transitoria + presencia de modos altos espurios que el Newmark trapezoidal no atenuaría.
- **Despacho YAML:** `solver.type: NewtonHHTSolver`. Vía atributo `PIPELINE_KIND="transient"` heredado de `NewmarkSolver` → `run_transient`.
- **Spec:** `docs/specs/NewtonHHTSolver.md` (variante corta sobre la spec padre `HHTSolver.md`).
- **Archivo:** `solidum/math/solvers/newmark.py` (clase `NewtonHHTSolver`).

9.10 CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5)

- **Propósito:** integrar $M\ddot{u} + C\dot{u} + F_{\text{int}}(u) = F(t)$ con esquema **explícito** de segundo orden — la aceleración se actualiza en cada paso por $M^{-1} \cdot \text{rhs}$ con M diagonal lumped, sin sistema lineal ni Newton interno. Cubre análisis lineal y no lineal con un solo solver (parámetro `nonlinear: bool`).
- **Esquema (leapfrog Belytschko-Liu-Moran):**
 - Inicialización: $a_0 = M^{-1} \cdot (F(0) - F_{\text{int}}(u_0) - C \cdot v_0 - F_{\text{dir}})$, $v_{\{1/2\}} = v_0 + (\Delta t/2) \cdot a_0$.
 - Paso $n \rightarrow n+1$: $u_{\{n+1\}} = u_n + \Delta t \cdot v_{\{n+1/2\}}$; reevaluar $F_{\text{int}}(u_{\{n+1\}})$; $a_{\{n+1\}} = M^{-1} \cdot (F_{\{n+1\}} - F_{\text{int}} - C \cdot v_{\{n+1/2\}} - F_{\text{dir}})$; $v_{\{n+1\}} = v_{\{n+1/2\}} + (\Delta t/2) \cdot a_{\{n+1\}}$.
 - Modo lineal: $F_{\text{int}} = K \cdot u$ (K constante, ensamblada una vez). Modo no lineal: `Assembler.assemble_non_line` cada paso (descarta la tangente).
- **Estabilidad condicional CFL:** $\Delta t < 2/\omega_{\text{max}}$. Para barras: $\Delta t < L_{\text{el}}/c_p = L_{\text{el}} \cdot \sqrt{(\rho/E)}$. Si se excede, el solver detecta la explosión exponencial a posteriori y aborta con `RuntimeError` informativo. **El solver no calcula Δt_{crit} automáticamente** — responsabilidad del usuario en esta versión inicial (power iteration sobre $M^{-1}K$ queda diferida).
- **Parámetros:** `t_end`, `dt`, `nonlinear` (default `False`), `rayleigh`, `u0`, `u0_dot`, `F_func`, `lumping` (default `"lumped"`; `"consistent"` rechazado), `divergence_threshold` (default `1e8`).
- **Cuándo usarlo:** wave propagation, impacto, dinámica de respuesta rápida — situaciones donde el Δt natural del problema cae por debajo del paso de estabilidad incondicional de Newmark. Para problemas con plasticidad transitoria de respuesta moderadamente rápida, sigue siendo competitivo frente a Newton-Newmark por evitar la factorización de la tangente cada iteración.
- **Rechazos defensivos:** `lumping=consistent` → `ValueError` (la inversión de M consistente cada paso anula la ventaja); `Frame3D` con eje oblicuo a globales → `ValueError` (M_{red} no es estrictamente diagonal; ADR 0009 fase 2 documenta la bloque-diagonalidad inherente).
- **Despacho YAML:** `solver.type: CentralDifferenceSolver`. Vía atributo `PIPELINE_KIND="transient"` → `run_transient`. **Primer solver fuera de la jerarquía de Newmark**, lo que disparó la regla C de refactor (atributo declarativo en lugar de cadena de `isinstance`).

- **Spec:** docs/specs/CentralDifferenceSolver.md.
- **Archivo:** solidum/math/solvers/central_difference.py.

9.11 HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6)

- **Propósito:** resolver $(-\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K})\cdot\hat{\mathbf{u}} = \mathbf{F}$ para un barrido de frecuencias y devolver la amplitud compleja $\hat{\mathbf{u}}(\omega)$ en cada frecuencia. Lineal, en estado estacionario — sin transitorio. Apropiado para FRFs, diagnóstico de resonancias y respuesta forzada periódica.
- **Esquema:** ensamble único de $\mathbf{K}$, $\mathbf{M}$, $\mathbf{C} = \alpha\cdot\mathbf{M} + \beta\cdot\mathbf{K}$; reducción de Dirichlet; barrido `for  $\omega$  in omega: spsolve(Z( $\omega$ ), F_red)` con $\mathbf{Z}(\omega) = -\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K}$ compleja. Factorización LU compleja por frecuencia (no se cachea — $\mathbf{Z}$ cambia con ω).
- **Barrido configurable:** lineal (`scale="linear"`), logarítmico (`scale="log"`), o vector explícito (`omega=array`). Default `n_omega=100`.
- **Parámetros:** `omega` o (`omega_min`, `omega_max`, `n_omega`, `scale`); `F_amplitude` (default ceros; YAML inyecta automáticamente las `point_loads`); `rayleigh`; `lumping`.
- **Cuándo usarlo:** diagnóstico de resonancias, FRFs, respuesta forzada armónica de máquinas rotantes, viento turbulento descompuesto en armónicos. Si el problema requiere no linealidad (geométrica o material), usar `NewtonNewmarkSolver` con excitación armónica.
- **Caveats:** resonancia exacta sin amortiguamiento $\Rightarrow \mathbf{Z}(\omega_n)$ singular (`spsolve` puede fallar). Mitigación: añadir Rayleigh. Cargas con fase compleja requieren construcción desde código — YAML no soporta tipos complejos nativos.
- **Despacho YAML:** `solver.type: HarmonicSolver`. Vía atributo `PIPELINE_KIND="harmonic"` $\rightarrow$ `run_harmonic`. Tercer valor del literal (después de `"static"`, `"modal"`, `"transient"`).
- **Spec:** docs/specs/HarmonicSolver.md.
- **Archivo:** solidum/math/solvers/harmonic.py.

9.12 ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)

- **Propósito:** combinar las respuestas máximas de los primeros `n_modes` modos del sistema bajo un espectro de respuesta dado ($\mathbf{S}_d(\omega)$ o $\mathbf{S}_a(\omega)$) en una envolvente máxima. Análisis estadístico, no temporal — apropiado para diseño sísmico normativo (ASCE 7, Eurocódigo 8, NCh 433, NTC-DS).
- **Esquema:**
- Análisis modal interno vía `ModalSolver` para obtener (`modes`, `frequencies`).
- Factores de participación $\Gamma_n = \varphi_n^\top \cdot \mathbf{M} \cdot \mathbf{r}$ con $\mathbf{r}$ el vector de excitación rígida en la dirección sísmica.

- Respuesta máxima por modo: $u_n^{\text{max}} = \Gamma_n \cdot \varphi_n \cdot S_d(\omega_n)$.
- Combinación SRSS (default) o CQC con coeficientes de correlación de Der Kiureghian.
- **Parámetros:** `n_modes`, `direction` (ndarray o `{"dof_name": "ux"}`), `spectrum` (callable o dict tabulado/constante), `combination` ("SRSS"/"CQC"), `damping` (sólo CQC, default 0.05), `sigma`, `lumping`.
- **Cuándo usarlo:** diseño sísmico contra espectro normativo; diagnóstico de modos dominantes en una dirección de excitación. Para no-linealidad usar transitorio (Newton-Newmark con acelerograma).
- **Caveats:** precisión depende del número de modos incluidos — verifica `cumulative_effective_mass_ratio` ≥ 0.9 . CQC requerido con modos cercanos en frecuencia (cociente < 1.3). Excitación multi-direccional simultánea (CQC3, 100/30/30) requiere combinación adicional fuera del solver.
- **Despacho YAML:** `solver.type: ResponseSpectrumSolver`. Vía atributo `PIPELINE_KIND="spectrum"` $\rightarrow$ `run_response_spectrum`. Cuarto valor del literal (después de "static", "modal", "transient", "harmonic").
- **Algoritmos centralizados:** `solidum/math/modal_response.py` — `response_spectrum_srss`, `response_spectrum_cqc`, `participation_factors`, `spectrum_from_sa`, `spectrum_tabulated`. Compartido con `ModalResult.free_vibration` (wrapper delgado tras regla D 2026-05-18).
- **Spec:** `docs/specs/ResponseSpectrumSolver.md`.
- **Archivo:** `solidum/math/solvers/response_spectrum.py`.

9.13 Cómo añadir un solver nuevo

`/solidum-new solver <Name>` — genera archivo en `solidum/math/solvers/<snake>.py`, decorador `@SolverRegistry.register`, esqueleto de test.

Convenciones de interfaz:

- **Solvers estáticos** (lineales, no lineales, arc-length): `PIPELINE_KIND = "static"`. Constructor recibe `assembler` + parámetros; método `solve(F_ext_global, step_callback=None)` $\rightarrow$ `U_final`. Comprometen los estados internos vía `assembler.commit_all_states()` al converger cada paso. Retornan el campo de desplazamientos completo.
- **Solvers modales / autovalor** (modal — ADR 0009 — y futuros pandeo lineal): `PIPELINE_KIND = "modal"`. Constructor recibe `assembler` + parámetros; método `solve()` $\rightarrow$ `ModalResult` (u otro tipo específico). No consumen vector de cargas. `run_yaml` despacha a `run_modal`.
- **Solvers transitorios** (Newmark, HHT, central differences): `PIPELINE_KIND = "transient"`. Constructor recibe `assembler`, `t_end`, `dt` + parámetros; método `solve()` $\rightarrow$ `TransientResult`. `run_yaml` despacha a `run_transient`.
- **Solvers en frecuencia / espectrales** (harmonic con `PIPELINE_KIND = "harmonic"`, response spectrum con `"spectrum"`). Constructor recibe `assembler` + parámetros del análisis; método `solve()` devuelve el resultado específico (`HarmonicResult`, `ResponseSpectrumResult`). `run_yaml` despacha a `run_harmonic` o `run_response_spectrum` según el atributo.

El **dispatch en run_yaml** se hace por el atributo de clase `PIPELINE_KIND` (regla C, 2026-05-18). Solvers nuevos no clásicos (que no hereden de los existentes) sólo declaran su `PIPELINE_KIND` y quedan automáticamente cableados — no requieren tocar `entry.py`.

Tras implementar, **añadir una entrada a este catálogo** siguiendo el formato de arriba y promover la spec a `status: validated`.

Capítulo 10

Anexos técnicos

*Esta sección es referencia técnica del subsistema interno que resuelve el sistema lineal $K\mathbf{x} = \mathbf{b}$ que aparece en el corazón de cada solver no lineal. La elección del backend es **automática**; el usuario no decide. Esta sección documenta cómo se decide y cuáles son los puntos de override para diagnóstico.*

>Diseño completo: ver ADR 0003 en `docs/adr/0003-despachador-de-solver-algebraico.md`.

10.1 Dos capas que se llaman ambas «solver»

Conviene distinguirlas:

- **Solver no lineal** — `LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`. Orquesta la estrategia de paso, las iteraciones de Newton, el control de longitud de arco, los criterios de convergencia. Es lo que el usuario elige en el YAML con `solver.type`.
- **Capa algebraica** — `solidum.math.linalg`. Resuelve el sistema lineal $K \cdot \delta U = \mathbf{R}$ que aparece dentro de cada iteración del solver no lineal. Tiene varios *backends* numéricos y un despachador interno que elige el adecuado según las propiedades de K . Es plumbing automático: el usuario no la ve salvo que pida diagnóstico.

10.2 Backends disponibles y criterio de selección

El despachador (`solidum.math.linalg.dispatcher.select_solver`) elige el backend en función de tres flags declarativos sobre K : simetría, positividad y talla.

Régimen de K	Backend elegido	Notas
Simétrica + positiva definida	Cholesky (CHOLMOD)	2× más rápido y 2× menos memoria que LU. Requiere la dependencia opcional <code>scikit-sparse</code> . Si falta, degrada a LU con un <i>warning</i> una sola vez.
Simétrica indefinida	LU general (placeholder LDL^T)	Ideal para LDL^T con conteo de pivotes negativos (Sturm sequence) en pando y snap-through. La interfaz está reservada; mientras no haya backend nativo, se usa LU sin pérdida de corrección, solo sin diagnóstico de bifurcación.
No simétrica	LU general (SuperLU)	Backend universal. Cubre plasticidad no asociada, follower loads, contacto con fricción.

Origen de los flags, todos derivables del modelo:

- `is_symmetric`: AND lógico de `material.IS_SYMMETRIC` y `element.PRESERVES_SYMMETRY` sobre todos los componentes del dominio. Defaults True.
- `is_positive_definite`: arranca en True para `LinearSolver` y `NonlinearSolver`; en False para `ArcLengthSolver`. Se degrada automáticamente si Cholesky reporta no-positividad.
- `size`: número total de DOFs.

Ningún flag se pide al usuario.

EigenSolver (ADR 0009, problema generalizado). La capa algebraica incluye además `solidum.math.linalg.eigen.EigenSolver`, que resuelve el problema generalizado simétrico $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ envolviendo `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). No comparte el Protocol `LinearAlgebraSolver` de los backends de $K \cdot x = b$ porque su firma natural es `solve(K, M, n_modes) → (λ, φ)`. El `ModalSolver` lo invoca para el análisis modal; los cálculos internos de shift-invert generan una factorización de $(K - \sigma \cdot M)$ reutilizada en cada iteración Lanczos, así que la palanca de optimización es la misma — Cholesky enchufado en lugar de SuperLU bajaría 2× el coste del modal en problemas SPD (pendiente, ver memoria de cierre).

10.3 Fallback automático SPD → LU

Si en algún paso K deja de ser positiva definida (paso a régimen postcrítico, daño con reblandecimiento), Cholesky lanza `CholeskyNotPositiveDefiniteError`. Los tres solvers no lineales lo capturan y reinstancian el backend con LU general para el resto del análisis. La transición es silenciosa — solo se imprime un mensaje informativo en stdout.

Esto significa que **forzar Cholesky desde YAML como diagnóstico nunca rompe el análisis**: si la matriz se vuelve incompatible, el sistema se autocorrigie.

10.4 Factorización reusable y Newton modificado

La interfaz expone `solver.factorize(K)` que retorna un objeto `FactorizedSolver` con `solve(b)` reutilizable. Habilita:

- **Newton modificado:** el `NonlinearSolver` admite el parámetro `freeze_tangent_after_iter: int` que congela la factorización tras N iteraciones del paso. Útil cuando la tangente cambia poco y el coste dominante es la factorización.
- **Dinámica implícita lineal** (ADR 0009 fase 3): el `NewmarkSolver` factoriza una sola vez la matriz efectiva $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$ al inicio del análisis y reutiliza la factorización en cada paso temporal. Con Δt constante y problema lineal, los cientos o miles de pasos del análisis se reducen a sustituciones triangulares baratas. Es el mismo `FactorizedSolver` del despachador.
- **Análisis futuros** (pandeo lineal, dinámica implícita no lineal con factorización congelada entre pasos cuando la tangente cambia poco) reutilizan la misma interfaz sin extensiones adicionales.

Ejemplo YAML:

```
solver:
  type: NonlinearSolver
  num_steps: 10
  tol: 1.0e-8
  freeze_tangent_after_iter: 2
```

10.5 Override desde YAML — herramienta de diagnóstico

Solo en caso de necesidad de *diagnóstico* o *benchmarking*, el bloque `solver` admite el campo opcional `linear_algebra`:

```
solver:
  type: LinearSolver
  linear_algebra: auto           # default; el despachador decide
  # linear_algebra: cholesky    # forzar Cholesky (requiere scikit-sparse)
  # linear_algebra: lu          # forzar LU (siempre disponible)
  # linear_algebra: ldlt        # placeholder; degrada a LU con warning
```

Cuándo usar el override:

- Comparar tiempos entre Cholesky y LU en un mismo problema.
- Forzar LU si se sospecha un bug en la auto-detección de simetría tras añadir un material nuevo.
- Reproducir un caso de regresión bit-a-bit con un backend específico.

Cuándo NO usarlo: como parte del setup habitual de un caso de análisis. No es decisión de modelado; el default `auto` cubre todos los regímenes correctamente.

10.6 Activación de Cholesky (opcional)

```
conda install -c conda-forge scikit-sparse
```

Una vez instalado, los problemas estáticos lineales y los Newton estables empiezan a usar Cholesky automáticamente — sin tocar YAML, sin recompilar, sin reescribir specs. Si la dependencia no está disponible, Solidum funciona idéntico con LU.